ATAP . . . 3
TrustX - Adaptive Trust Assurance Platform . . . 4
Detailed Serverless Architecture Diagram . . . 6
Workflow for the AI Compliance Assistant (Trustx Bot) . . . 8
Mockup Outline . . . 10
Scope of Work - Privileged Access Management with User Footprint Knowledge Graph . . . 11
Next Steps & Focus Area . . . 15
1 Data Ingestion & API Integrations . . . 16
2 Graph Database Schema (Knowledge Graph) & Mock Data . . . 18
Mock - TrustX Chatbot Graph Visualization Code . . . 21
Graph DB Modeling . . . 25
3 Risk Analytics & AI/ML Models . . . 27
4 Chatbot & Graph Visualization . . . 28
Chatbot + Knowledge Graph Demo Plan . . . 30
Next Step: Build the Demo . . . 32
5 Compliance & Reporting Features . . . 34
Hybrid Approach (LightRAG + GraphDB) . . . 35
Queries That Require Proper Reasoning (Beyond Simple NL-to-Cypher Conversion) . . . 38
Profile360 Requirements Doc (Draft) . . . 40
Set up Cloudflare Pages with Auth0 authentication . . . 44
TrustX AWS Infrastructure Documentation . . . 46
Lambda Development Guide with Auth0 and Dependencies Layers . . . 49
Auth0 Authentication Flow for Cloudflare Pages . . . 54
TrustX ATAP Multi-Tenant Architecture (Shared Database, Separate Schemas) . . . 56
Schema for TrustX Multi-Tenant Architecture (Shared Database, Separate Schemas) . . . 60
Database Schema Design . . . 62
Compliance Database Model for TrustX . . . 70
Data Pipeline for Compliance and Profile360 Modules . . . 73
TrustX ATAP Integration Security & Storage Standards . . . 75
TrustX GitHub Integration . . . 78
GitHub Integration Testing - ATAP API & UI . . . 81
TrustX Adaptive API - Hybrid API Structure (REST + GraphQL) . . . 84
Archived - TrustX Adaptive API Gateway . . . 86
AI-Assisted Policy Management . . . 89
Draft - Policy Management API - Swagger . . . 93
Design & Functional Specification for Policy Editing Feature . . . 96
Comprehensive List of Policy Templates for TrustX ATAP . . . 98
Acceptable Use Policy (Template) . . . 101
Information Security Policy (Template) . . . 103
TrustX Control ID Naming Convention and Standard . . . 106
About TrustX Manufacturing, Inc. . . . 109
Draft - BRD Profile360 - AI Insights & Risk Intelligence Expansion . . . 111
Knowledge Graph Project Timeline . . . 113
Production-Ready PostgreSQL 15.2 with Apache AGE on AWS ECS Fargate . . . 114
Profile 360 Refactor Documentation . . . 117

Profile 360 System Overview 118
Profile 360 Multi-Tenant Architecture Guide 122
Profile 360 Deployment Checklist 128
Profile 360 S3 Permissions Fix 132
Profile 360 Migration Guide 136
Profile 360 Multi-Tenant Deployment Guide 141
Neel Hand-off 145

ATAP

Description

In a sentence or two, describe the purpose of this space.

Project Tracker

Recently updated content

This list below will automatically update each time somebody in your space creates or updates content.

Neel Hand-off
21 minutes ago • contributed by Neel Gali

Profile 360 Multi-Tenant Deployment Guide
Jun 16, 2025 • contributed by Neel Gali

Profile 360 Multi-Tenant Architecture Guide
Jun 16, 2025 • contributed by Neel Gali

Profile 360 Migration Guide
Jun 11, 2025 • contributed by Neel Gali

Profile 360 S3 Permissions Fix
Jun 11, 2025 • contributed by Neel Gali

Profile 360 Deployment Checklist
Jun 11, 2025 • contributed by Neel Gali

Profile 360 System Overview
Jun 11, 2025 • contributed by Neel Gali

Profile 360 Refactor Documentation
Jun 11, 2025 • contributed by Neel Gali

Production-Ready PostgreSQL 15.2 with Apache AGE on AWS ECS Fargate
Jun 04, 2025 • contributed by Rama Kesani

Knowledge Graph Project Timeline
Jun 02, 2025 • contributed by Rama Kesani

Contributors

This list below will automatically update each time somebody in your space creates or updates content.

Bhanu Nagarur, Rama Kesani, Bhanu Nagarur (Unlicensed), Neel Gali, avaneesh (Unlicensed), Sai Vijay Nagarur (Unlicensed)

TrustX - Adaptive Trust Assurance Platform

TrustX is an innovative AI-driven SaaS platform designed to redefine trust and compliance in the digital age. The platform provides real-time, adaptive security and compliance assurance by integrating advanced AI insights, user behavior analytics, integrated threat intelligence, and a dynamic trust center. TrustX aims to proactively identify and mitigate risks, ensuring continuous compliance and enhancing transparency for customers and stakeholders.

KEY FEATURES

1. **AI-Driven Insights and Recommendations**
   - Utilize AI to analyze real-time data and provide actionable insights for improving security posture.
   - Offer tailored recommendations based on the company's industry, size, and specific threat landscape.
2. **Dynamic Trust Center**
   - Interactive, customizable trust center that evolves with the company's security landscape.
   - Real-time updates and notifications about changes in compliance status, security incidents, and remediation actions.
3. **User Behavior Analytics**
   - Monitor and analyze user behavior to detect anomalies and potential security threats.
   - Provide insights into how employees and users interact with the platform, identifying risky behaviors and suggesting training or policy adjustments.
4. **Integrated Threat Intelligence**
   - Incorporate threat intelligence feeds to provide context-aware security updates and alerts.
   - Display the latest threats relevant to the company's environment and how the company is protected against them.
5. **Automated Compliance Management**
   - Automate evidence collection and audit preparation for various certifications (SOC 2, ISO 27001, etc.).
   - Continuously monitor compliance controls and provide alerts for any deviations.
6. **Customer and Stakeholder Engagement**
   - Provide a transparent view of the company's security and compliance status to customers and stakeholders.
   - Offer a secure portal where customers can request specific documents or evidence, which can be shared with granular access controls.
7. **Gamified Security Training**
   - Implement gamified training modules that adapt based on user performance and behavior analytics.
   - Encourage employees to improve their security awareness and practices through engaging and interactive content.
8. **Adaptive Security Policies**
   - Use AI to recommend and implement adaptive security policies that change based on real-time threat data and user behavior.
   - Ensure that security policies are always up-to-date and relevant to the current threat landscape.

UNIQUE SELLING POINTS

1. **Real-Time Adaptation**
   The platform's ability to adapt and evolve based on real-time data and analytics makes it a proactive rather than reactive solution.
2. **Personalized Insights**
   Tailored recommendations and insights ensure that the platform adds significant value to each specific user and organization.

3. **Engagement and Transparency**
   Enhanced engagement with customers and stakeholders through an interactive trust center and secure document sharing.
4. **Behavioral Analysis**
   Incorporating user behavior analytics to preemptively identify and mitigate risks sets it apart from traditional static security platforms.

MARKET POSITIONING

- **Target Audience:** Mid to large-sized enterprises that prioritize security and compliance, especially those in regulated industries like finance, healthcare, and technology.
- **Value Proposition:** A dynamic, intelligent, and user-centric approach to security and compliance that not only protects but also empowers organizations through real-time adaptation and insights.

By focusing on adaptive and real-time features, this platform offers a unique and valuable proposition in the competitive security and compliance SaaS market.

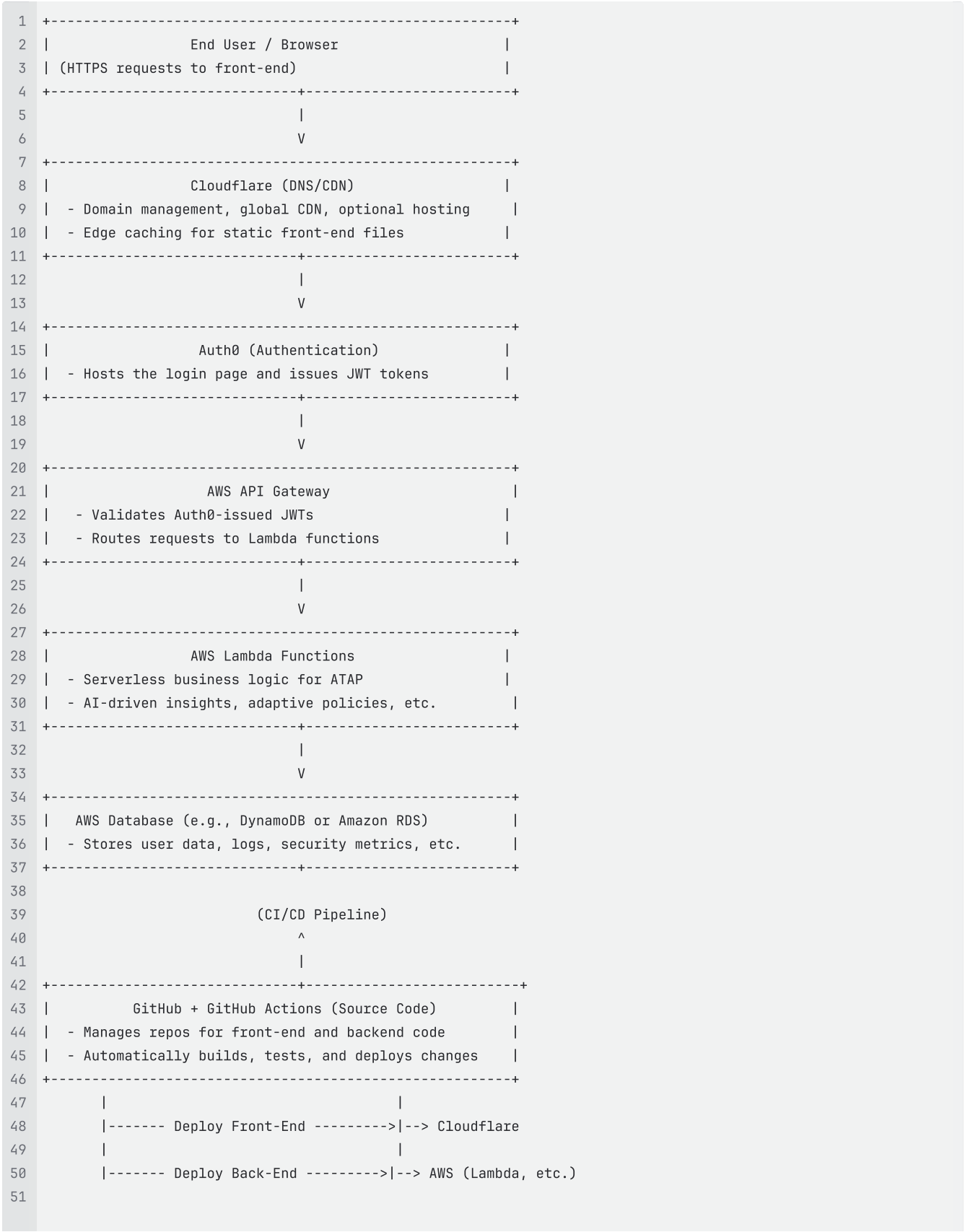
Market Opportunity:

- The global cybersecurity market is expected to reach $345.4 billion by 2026.
- The compliance management market is projected to grow to $64.7 billion by 2026.
- The estimated addressable market for TrustX is between $25-50 billion by 2026, combining elements of cybersecurity and compliance management markets.

Target Audience:

- Mid to large-sized enterprises in highly regulated industries such as finance, healthcare, and technology.
- Security-conscious organizations prioritize proactive security and compliance measures.
- Service providers (MSSPs) seeking advanced tools for their clients.

Detailed Serverless Architecture Diagram



```
52
53
```

How It Works

1. **User Access via Cloudflare**
   - End users navigate to your custom domain, which is managed by **Cloudflare** for DNS and CDN.
   - Cloudflare can serve static files (e.g., an SPA front-end) from its global edge network.
2. **Authentication with Auth0**
   - Users attempting to log in are redirected to **Auth0**, where they authenticate (via username/password, social login, SSO, etc.).
   - Upon successful login, Auth0 issues a **JWT (JSON Web Token)** to the user.
3. **API Requests via AWS API Gateway**
   - The front-end includes the Auth0-issued JWT in requests to **AWS API Gateway**.
   - API Gateway verifies the JWT signature and ensures the token is valid before forwarding the request.
4. **Serverless Logic on AWS Lambda**
   - **AWS Lambda** hosts the core Adaptive Trust Assurance Platform logic, including AI-driven analytics, user behavior monitoring, and adaptive policy updates.
5. **Data Storage in AWS**
   - A database service such as **AWS DynamoDB** (NoSQL) or **Amazon RDS** (relational) stores user profiles, security data, logs, and compliance information.
6. **CI/CD with GitHub + GitHub Actions**
   - All source code (front-end and back-end) lives in **GitHub**.
   - **GitHub Actions** automates building, testing, and deployment:
     - **Front-End** updates are pushed to Cloudflare (or another hosting provider if preferred).
     - **Back-End** updates are deployed to AWS (updating Lambda functions, API configurations, etc.).
7. **Monitoring & Logging** (Optional)
   - **AWS CloudWatch** or other monitoring services can capture logs and metrics from Lambda and API Gateway.
   - **Auth0** provides dashboards and logs for authentication events and user insights.

This design leverages **Auth0** for secure, standards-based authentication (with minimal custom code), **Cloudflare** for high-speed global content delivery, **AWS** for serverless infrastructure, and **GitHub Actions** for a streamlined development workflow. The result is a **scalable, flexible, and secure** architecture well-suited for an AI-driven Adaptive Trust Assurance Platform.

Workflow for the AI Compliance Assistant (Trustx Bot)

1. User Login and Role Detection

- **User Journey**:
  - The user logs into TrustX.
  - System identifies the user's role (e.g., SOC Analyst, Compliance Manager, CISO) and customizes the dashboard and chatbot accordingly.
- **Backend**:
  - Authenticate via **Amazon Cognito** or **Auth0**.
  - Use Role-Based Access Control (RBAC) to fetch user-specific data.

---

2. Dynamic Compliance Dashboard

- **Components**:
  - **Certification Status Overview**:
    - Progress bars for SOC 2, ISO 27001, PCI DSS, etc.
    - High-level compliance percentage (e.g., 85% compliant).
  - **Task List**:
    - Actionable items tied to controls (e.g., "Submit evidence for Control 3.2").
  - **Audit Status**:
    - Active audits with deadlines and flagged items requiring immediate attention.
  - **Alerts**:
    - Notifications for overdue tasks, expiring certifications, or non-compliant controls.
- **Tech Stack**:
  - **Frontend**: React.js or Angular for dynamic cards and updates.
  - **Backend**: AWS Lambda or Node.js API fetching data from a central compliance database.

---

3. Chatbot Interaction Workflow

- **User Interaction**:
  - A **persistent chatbot window** is available on the dashboard.
  - Example Questions:
    - *"What's the current status of our SOC 2 audit?"*
    - *"How many controls are pending for ISO 27001?"*
    - *"Can you upload this document as evidence for Control 3.5?"*
- **Chatbot Capabilities**:
  - **Query Compliance Data**:
    - Integrate APIs to fetch real-time information about frameworks, controls, and evidence.
  - **Document Management**:
    - Allow users to upload or link documents directly via the chatbot.
  - **Task Automation**:
    - Enable users to mark controls as complete or assign tasks (e.g., *"Assign Control 3.2 to John Doe"*).
  - **Proactive Alerts**:
    - Notify users of upcoming deadlines or auditor requests.

4. AI Backend Integration

- **NLP Model**:
  - Fine-tune a GPT model for compliance-specific language (e.g., SOC 2, ISO 27001 terminologies).
  - Preload the model with framework definitions, control requirements, and past audit examples.
- **Knowledge Base**:
  - Centralized repository of:
    - Compliance frameworks (SOC 2, ISO 27001).
    - Organizational policies and evidence documents.
    - Past audit outcomes and action items.
- **Tech Stack**:
  - **AI Framework**: OpenAI API or Hugging Face Transformers.
  - **Data Storage**: Amazon DynamoDB for compliance data.

---

5. Evidence Submission Workflow

- **User Journey**:
  - Auditor requests evidence for a specific control.
  - SOC analyst uploads the document directly via the chatbot (e.g., *"Upload SOC2_Report_2025.pdf for Control 3.2"*).
- **Integration**:
  - Documents are stored in Amazon S3 or a similar secure repository.
  - Control statuses are automatically updated in the compliance database.

---

6. Multi-Framework Mapping

- **Use Case**:
  - Users want to compare overlapping controls between frameworks.
  - Example: *"Show me controls common between SOC 2 and ISO 27001."*
- **Implementation**:
  - Map controls across frameworks in a **graph database** (e.g., Neo4j).
  - Allow the bot to fetch and visualize overlapping controls.

Mockup Outline

```
+-------------------------------------------------------------------------------------------------+
|                            TrustX Compliance Assistant                                          |
|-------------------------------------------------------------------------------------------------|
| TrustX Logo                         SOC Analyst                                 User Profile  |
|-------------------------------------------------------------------------------------------------|
|                                                                                    Alerts Section   |
|                                                                                    ---------------- |
| Certification Status Cards:                                                      |  - 3 overdue       |
| -------------------------                                                         |    controls (SOC2) |
| [ SOC 2 Compliance         ]  [ ISO 27001 Compliance  ]  [ PCI DSS Compliance ] |  - 2 deadlines    |
| [ 85% Complete ]              [ 70% Complete ]           [ 60% Complete ]        |    approaching    |
| --------------------------                                                 ----------------------|
|                                                                                 Task List Section|
| Tasks:                                                                        --------------------|
| - Submit evidence for Control 3.2 (SOC 2)                                     |                    |
| - Update Control 5.1 with the latest policy                                   |  TrustX Bot Chat     |
| - Respond to auditor request (ISO 27001)                                      | -------------------|
|                                                                               |  > Ask me anything |
| Alerts:                                                                       |  Examples:         |
| - High-risk flagged controls                                                  |  - Status of SOC2  |
| - Expiring compliance documents                                               |  - Submit Evidence |
|                                                                               |  - Compare SOC2... |
|                                                                               --------------------|
| Compliance Insights:                                                                              |
| --------------------                                                                              |
| [ Framework Overlap: SOC 2 & ISO 27001 ]                                                          |
| [ Click here for detailed view ]                                                                  |
+-------------------------------------------------------------------------------------------------+

```

Landing Page (Dynamic Dashboard)

- **Header**:
  - Company logo and user profile (role-based).
- **Main Section**:
  - **Certification Status Cards**:
    - SOC 2: 85% Complete (Progress bar).
    - ISO 27001: 70% Complete (Risk level: Medium).
  - **Task Section**:
    - Actionable items (e.g., “Submit evidence for Control 5.3 by Jan 20”).
  - **Alerts**:
    - Notifications (e.g., “3 overdue controls for SOC 2”).
- **Chatbot Window**:
  - Fixed at the bottom-right corner.
  - Example: *“How can I help you today?”*

Chatbot Interface

- **Opening Prompts**:
  - *“Ask me about compliance status, tasks, or audit requirements.”*
- **Interactions**:
  - User: *“What’s the status of our SOC 2 framework?”*
    Chatbot: *“85% of controls are implemented. 3 are overdue. Click here for details.”*
  - User: *“Upload this document for Control 5.1.”*
    Chatbot: *“Document uploaded and linked to Control 5.1. Anything else?”*

Evidence Submission Flow

- Chatbot prompts the user to upload evidence.
- Evidence automatically links to the requested control and updates the dashboard.

Future Features

1. **Custom Reporting**:
   - Generate tailored reports for stakeholders.
   - Example: *“Show me a risk report for Q1 2025.”*
2. **AI-Powered Remediation Suggestions**:
   - If a control is flagged as non-compliant, the bot suggests remediation steps.
3. **Continuous Monitoring**:
   - Integrate with security tools (e.g., AWS Config, Splunk) to monitor control compliance in real time.

Scope of Work - Privileged Access Management with User Footprint Knowledge Graph

---

Marketing

> **Feature Name**
>
> **Profile360™ :** “360° Access Intelligence: Know Who, What, and Why.”
>
> **(once registered Profile360®)**

1. **Objective**:
   - Analyze and map user privileges across all connected applications.
   - Build a **knowledge graph** to represent a user's **access footprint**, highlighting privileged accounts and their associated risks.
   - Enable SOC analysts to interact with the knowledge graph via the **TrustX Chatbot**, offering real-time answers and actionable insights.
2. **Key Functionalities**:
   - **Access Aggregation**:
     - Collect and normalize access data from multiple applications (e.g., AWS, Azure, Jira, Salesforce).
     - Identify roles, permissions, and privileged access.
   - **Risk Analysis**:
     - Flag high-risk users or permissions (e.g., users with admin access to critical systems).
     - Detect unusual privilege escalations or dormant accounts.
   - **Knowledge Graph Visualization**:
     - Display a dynamic, interactive graph showing:
       - User roles.
       - Applications accessed.
       - Privilege levels.
       - Connections to shared or privileged accounts.
   - **Real-Time Queries via TrustX Chatbot**:
     - SOC analysts can ask:
       - *“Show me the footprint of user John Doe across all applications.”*
       - *“Who has admin access to AWS?”*
       - *“Are there any dormant privileged accounts?”*
     - Responses are displayed as visual graphs with contextual data.

Workflow Example

1. **Data Ingestion**:
   - Integrate with applications using APIs (e.g., AWS IAM, Azure AD, Okta, Salesforce).
   - Collect data on users, roles, permissions, and access logs.

- Use a **graph database** (e.g., Neo4j) to build relationships between entities.

2. **User Query via Chatbot**:
   - **SOC Analyst Input**:
     - *"Show me Alice's access footprint."*
   - **Chatbot Response**:
     - Opens a **canvas** below the chatbot.
     - Displays Alice's **knowledge graph**, showing:
       - Applications Alice has access to.
       - Roles/permissions within each application.
       - Any flagged risks (e.g., excessive permissions, shared credentials).
3. **Visualization (Knowledge Graph)**:
   - Nodes:
     - Represent entities like **users**, **applications**, **roles**, and **resources**.
   - Edges:
     - Represent relationships (e.g., *"Alice has admin access to AWS EC2"*).
   - Node Colors/Icons:
     - Green: Regular user access.
     - Orange: Elevated access.
     - Red: High-risk privileged access.
4. **Risk Highlighting**:
   - Flag excessive or inappropriate access.
   - Suggest actions (e.g., *"Reduce Alice's permissions in Salesforce to read-only"*).

Key Components

1. **Backend (Data Aggregation and Processing)**:
   - **APIs**:
     - Integrate with cloud services, IAM systems, and SaaS tools.
   - **Graph Database**:
     - Store and query relationships (e.g., Neo4j, Amazon Neptune).
   - **AI/ML Risk Analysis**:
     - Identify anomalies (e.g., unusual access patterns, unused privileges).
2. **Frontend (Visualization and Chatbot Interaction)**:
   - **TrustX Chatbot**:
     - Use NLP to understand user queries (e.g., OpenAI, Hugging Face).
     - Generate real-time responses with graphs.
   - **Knowledge Graph Canvas**:
     - Interactive and draggable visualization.
     - Built with D3.js or Cytoscape.js for dynamic graph rendering.
   - **Risk Indicators**:
     - Display alerts, recommendations, and user profiles inline.
3. **Compliance Integration**:
   - Map privileges to compliance controls (e.g., SOC 2, ISO 27001).
   - Highlight gaps or violations in permissions.

User Story

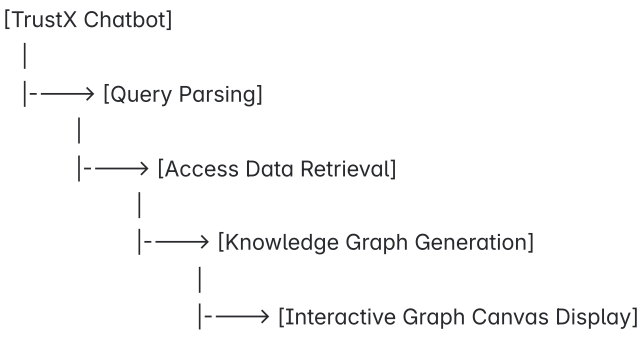
1. **As a SOC Analyst**, I want to:
   - Visualize a user's access footprint across all applications.
   - Identify high-risk permissions (e.g., unused admin accounts).
   - Query access information interactively (e.g., “Who can delete resources in AWS?”).
2. **As a Compliance Officer**, I want to:
   - Ensure all privileges align with organizational policies and compliance frameworks.
   - Generate audit-ready reports of privileged access and user roles.

Technical Stack

1. **Data Integration**:
   - AWS IAM, Azure AD, Okta, and other APIs for access data.
   - Custom connectors for SaaS apps (e.g., Jira, Salesforce).
2. **Knowledge Graph**:
   - **Graph Database**: Neo4j or Amazon Neptune.
   - **Query Language**: Cypher or SPARQL for traversing relationships.
3. **Frontend**:
   - **Visualization Framework**: D3.js, Cytoscape.js, or Sigma.js.
   - **Chatbot**: React.js for embedding the chatbot UI.
4. **AI/ML for Risk Analysis**:
   - Models to detect anomalies (e.g., frequent role changes, dormant privileged accounts).

Diagram

Here’s how the system would flow:




```
[TrustX Chatbot]
   |
   |----> [Query Parsing]
            |
            |----> [Access Data Retrieval]
                      |
                      |----> [Knowledge Graph Generation]
                                |
                                |----> [Interactive Graph Canvas Display]
[TrustX Chatbot]
   |
   |----> [Query Parsing]
            |
            |----> [Access Data Retrieval]
```

```
                    |
                    |----> [Knowledge Graph Generation]
                               |
                               |----> [Interactive Graph Canvas Display]

```

Next Steps & Focus Area

Next Steps

We have a solid foundation outlined in the document, covering:

- **Data Aggregation** (integrating access data from various apps)
- **Risk Analysis** (flagging high-risk accounts and permissions)
- **Knowledge Graph Visualization** (displaying user access relationships dynamically)
- **TrustX Chatbot Integration** (for querying access footprints)
- **Compliance Mapping** (aligning access controls with compliance frameworks)

Where Do We Want to Focus First?

1. **Data Ingestion & API Integrations** – Defining data sources and how to collect access information.
2. **Graph Database Schema** – Designing nodes, edges, and relationships in Neo4j/Amazon Neptune.
3. **Risk Analytics & AI/ML Models** – Identifying anomalies and high-risk behaviors.
4. **Frontend Visualization & Chatbot UI** – Building interactive visual access graphs.
5. **Compliance & Reporting Features** – Mapping access rights to SOC 2, ISO 27001, etc.

1️⃣ Data Ingestion & API Integrations

Objective:

Build a **centralized access repository** by aggregating access data from multiple sources.

1️⃣ Identify Data Sources & Integration Methods

We'll collect user access data from three key sources:

Category	Examples	Integration Method
Cloud Providers	AWS IAM, Azure AD, GCP IAM	REST APIs (AWS STS, Azure Graph, GCP IAM API)
SaaS Applications	Salesforce, Jira, GitHub, ServiceNow	REST APIs, Webhooks
IAM & SSO Solutions	Okta, OneLogin, Ping Identity	SCIM, OpenID Connect APIs
Enterprise Systems	Active Directory, Oracle, SAP	LDAP, Database Queries

---

2️⃣ API Integration Architecture

API Access & Authentication

- Use **OAuth 2.0** or API keys for authentication.
- Implement **JWT-based session management** for secure API calls.

Data Flow

1. **Periodic Sync** (e.g., every 6 hours via ETL jobs).
2. **Event-Driven Updates** (Webhook-based real-time ingestion).
3. **Manual Triggers** (SOC analyst queries an on-demand update).

Example API Calls

- AWS IAM:

```
bash
aws iam list-users --query "Users[*].{User:UserName,ARN:Arn}"
```

- Azure AD:

```
bash
GET https://graph.microsoft.com/v1.0/users?$select=id,displayName,userPrincipalName
```

- Okta Users API:

```
bash
GET /api/v1/users
```

3️⃣ Data Normalization & Correlation

Each system has its own terminology, so we need a **unified data model**.

Source	User Identifier	Role/Permission Format	Access Type
AWS IAM	ARN (Amazon Resource Name)	Policies, Groups, Inline Policies	IAM Role, User
Azure AD	Object ID, UPN (User Principal Name)	Groups, RBAC Assignments	Azure Role
Jira	Email, Account ID	Roles (Admin, Developer)	Project Permissions
Okta	Email, Okta ID	Group Memberships	SSO, SCIM

➡ **Solution:** Store all data in a **graph-based format**, linking identities across platforms.

4️⃣ Storage: Graph Database Schema

We'll use **Neo4j** or **Amazon Neptune** to store relationships between users, applications, and permissions.

Graph Schema Design

```
plaintext
(User) --[HAS_ACCESS_TO]--> (Application) (User) --[ASSIGNED_ROLE]--> (Role) (Role) --[GRANTS_ACCESS]--> (Resource)
```

Example Neo4j Query (Cypher)

Find all applications a user has access to:

```
cypher

MATCH (u:User)-[:HAS_ACCESS_TO]->(a:Application) WHERE u.name = "Alice" RETURN a.name
```

Output:

✅ **A structured access repository** that links users, roles, and permissions across platforms.

2️⃣ Graph Database Schema (Knowledge Graph) & Mock Data

This document provides a structured dataset that represents **user access footprints** across multiple applications, supporting:

- **Multi-Tenant Access Control**
- **Enterprise-Level Hierarchy (e.g., GitHub Enterprise → Organization → Repositories)**

1️⃣ Data Model Overview

The dataset now represents relationships between:

- **Tenants (Customers using TrustX)**
- **Applications (like Atlassian Jira/Confluence/Bitbucket/Guard (user database like IAM), Github, Lucid , Auth0)**
- **Enterprises (For apps like GitHub, Jira, etc.)**
- **Organizations (Within enterprises)**
- **Users**
- **Roles**
- **Resources (Repositories, Projects, Spaces)**
- **Compliance Standards**

2️⃣ Node (Entity) Definitions

Entity Type	Properties	Example Values
Tenant	`id`, `name`, `industry`, `region`	`"TrustX-Customer-1"`, `"FinanceCorp"`, `"US-East"`
Application	`id`, `name`, `type`	`"AWS"`, `"Azure"`, `"Salesforce"`, `"GitHub"`, `"Jira"`, `"Confluence"`
User	`id`, `name`, `email`, `department`	`"Alice"`, `"alice@example.com"`, `"Engineering"`
Enterprise	`id`, `name`	`"GitHub Enterprise"`
Organization	`id`, `name`, `parent_enterprise`	`"DevOpsTeam"`, `"EnterpriseX"`
Role	`id`, `name`, `privilege_level`	`"Admin"`, `"Read-Only"`, `"Editor"`
Resource	`id`, `name`, `type`	`"repo-123"`, `"Jira-Project-A"`, `"Space-Docs"`, `"EC2-Instance"`
Compliance Standard	`id`, `standard`, `requirement`	`"SOC2"`, `"ISO27001"`

3️⃣ Relationship (Edge) Definitions

Relationship	Source → Target	Description
`BELONGS_TO`	`User → Tenant`	Associates a user with a specific tenant
`OWNS`	`Tenant → Application`	Maps applications to specific tenants
`PART_OF`	`Organization → Enterprise`	Links an **organization** to a **GitHub Enterprise**
`OWNS_REPO`	`Organization → Repository`	Maps repositories owned by an organization
`BELONGS_TO_ORG`	`User → Organization`	Users within an organization
`HAS_ACCESS_TO`	`User → Application`	Defines application access for a user
`ASSIGNED_ROLE`	`User → Role`	Assigns a role to a user
`GRANTS_ACCESS`	`Role → Resource`	Role grants permissions to a resource
`FOLLOWS`	`Application → Compliance`	Application follows compliance standards

4️⃣ Sample Dataset (JSON Format)

```
[
  { "data": { "id": "TrustX-Customer-1", "label": "TrustX-Customer-1 (Tenant)" } },
  { "data": { "id": "Alice", "label": "Alice (User)" } },
  { "data": { "id": "GitHub-Enterprise", "label": "GitHub Enterprise" } },
  { "data": { "id": "DevOpsTeam", "label": "DevOps Team (Organization)" } },
  { "data": { "id": "repo-123", "label": "Repository-123 (GitHub Repo)" } },
  { "data": { "id": "AWS", "label": "AWS (Application)" } },
  { "data": { "id": "Azure", "label": "Azure (Application)" } },
  { "data": { "id": "Salesforce", "label": "Salesforce (Application)" } },
  { "data": { "id": "Jira", "label": "Jira (Application)" } },
  { "data": { "id": "Confluence", "label": "Confluence (Application)" } },
  { "data": { "id": "Admin", "label": "Admin (Role)" } },
  { "data": { "id": "SOC2", "label": "SOC2 Compliance" } },

  { "data": { "source": "Alice", "target": "TrustX-Customer-1", "label": "BELONGS_TO" } },
  { "data": { "source": "TrustX-Customer-1", "target": "GitHub-Enterprise", "label": "OWNS" } },
  { "data": { "source": "DevOpsTeam", "target": "GitHub-Enterprise", "label": "PART_OF" } },
  { "data": { "source": "DevOpsTeam", "target": "repo-123", "label": "OWNS_REPO" } },
  { "data": { "source": "Alice", "target": "DevOpsTeam", "label": "BELONGS_TO_ORG" } },
  { "data": { "source": "Alice", "target": "repo-123", "label": "HAS_ACCESS_TO" } },

  { "data": { "source": "Alice", "target": "AWS", "label": "HAS_ACCESS_TO" } },
  { "data": { "source": "Alice", "target": "GitHub", "label": "HAS_ACCESS_TO" } },
  { "data": { "source": "Alice", "target": "Admin", "label": "ASSIGNED_ROLE" } },
  { "data": { "source": "Admin", "target": "AWS", "label": "GRANTS_ACCESS" } },
```

```
  { "data": { "source": "AWS", "target": "SOC2", "label": "FOLLOWS" } }
]
```

✅ Multi-Tenant Isolation

- Ensures that **each TrustX customer (tenant) has isolated access data**, preventing cross-tenant access.

✅ Enterprise-Level Structure for SaaS Apps

- Applications like **GitHub, Jira, and Confluence** have **enterprise-level access models**.
- The schema supports **GitHub Enterprise → Organization → Repositories**.
- Can be extended for **Jira Projects & Confluence Spaces**.

✅ Improved Querying for Privileged Access

- Allows **SOC analysts to query specific tenants and organizations**.
- **Graph-based queries** can now traverse **multi-level access** within an enterprise.

6️⃣ Usage

- This dataset can be **ingested into a Neo4j database** to model user access relationships.
- It is designed to support **chatbot queries** and **graph visualization**.
- SOC analysts can query **user footprints**, **role assignments**, and **privileged access risks**.

Mock - TrustX Chatbot Graph Visualization Code

Overview

This **React-based chatbot** provides an interactive visualization of **privileged access relationships** using **Cytoscape.js**. It allows SOC analysts and security teams to **query user access footprints** across multiple applications such as **AWS, Azure, Salesforce, GitHub, Jira, and Confluence**.

Key Features

- **Interactive Knowledge Graph**: Displays **users, applications, roles, and compliance relationships** in a **graph-based UI**.
- **Chatbot Query Input**: Users can **enter access-related queries**, triggering a backend call.
- **Dynamic Data Fetching**: The chatbot can **update the graph** based on the **query results** from an API.
- **Graph Elements**:
  - **Users** (e.g., Alice, Bob, Charlie)
  - **Applications** (AWS, Azure, Salesforce, GitHub, Jira, Confluence)
  - **Roles** (Admin, Read-Only, Editor)
  - **Compliance Standards** (SOC2, ISO27001)
  - **Access Relationships** (e.g., HAS_ACCESS_TO, ASSIGNED_ROLE, GRANTS_ACCESS)

Code Components

1. State Management

- `query`: Stores user input from the chatbot.
- `graphData`: Stores the **knowledge graph elements** (nodes & edges).

2. Graph Data Structure

- Uses **Cytoscape.js** to visualize:
  - **Nodes**: Representing users, applications, and roles.
  - **Edges**: Defining access relationships (`HAS_ACCESS_TO`, `ASSIGNED_ROLE`, etc.).
- The dataset is preloaded and can be **updated dynamically** based on chatbot queries.

3. Chatbot Query Handling

- **User enters a query** (e.g., `"Who has access to AWS?"`).
- **API call** (`/api/chatbot`) simulates a backend request.
- **Graph updates** dynamically with the new query results.

4. Visualization with Cytoscape.js

- **Graph Layout**: Circular layout for better visibility.
- **Node Styling**: Labels displayed dynamically.
- **Edge Styling**: Uses arrows to indicate relationships.

Example Query Workflow

1. **User asks:** `"Show me Alice's access footprint"`
2. **Backend fetches** user relationships from a **graph database** (Neo4j/Amazon Neptune).

3. **Updated graph** is returned to the chatbot.
4. **Cytoscape.js renders** the new graph dynamically.

Next Steps

- **Connect to a real database** (Neo4j / Amazon Neptune) for live queries.
- **Enhance chatbot AI** to support more **complex access queries**.
- **Add Risk Indicators** (e.g., Dormant privileged accounts, Excessive permissions).

Code

```
import { useState } from "react";
import { Input } from "@/components/ui/input";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import CytoscapeComponent from "react-cytoscapejs";

const TrustXChatbot = () => {
  const [query, setQuery] = useState("");
  const [graphData, setGraphData] = useState([
    { data: { id: "Alice", label: "Alice (User)" } },
    { data: { id: "Bob", label: "Bob (User)" } },
    { data: { id: "Charlie", label: "Charlie (User)" } },

    { data: { id: "AWS", label: "AWS (Application)" } },
    { data: { id: "Azure", label: "Azure (Application)" } },
    { data: { id: "Salesforce", label: "Salesforce (Application)" } },
    { data: { id: "GitHub", label: "GitHub (Application)" } },
    { data: { id: "Jira", label: "Jira (Application)" } },
    { data: { id: "Confluence", label: "Confluence (Application)" } },

    { data: { id: "Admin", label: "Admin (Role)" } },
    { data: { id: "Read-Only", label: "Read-Only (Role)" } },
    { data: { id: "Editor", label: "Editor (Role)" } },

    { data: { id: "SOC2", label: "SOC2 Compliance" } },
    { data: { id: "ISO27001", label: "ISO 27001 Compliance" } },

    { data: { source: "Alice", target: "AWS", label: "HAS_ACCESS_TO" } },
    { data: { source: "Alice", target: "GitHub", label: "HAS_ACCESS_TO" } },
    { data: { source: "Bob", target: "Azure", label: "HAS_ACCESS_TO" } },
    { data: { source: "Charlie", target: "Salesforce", label: "HAS_ACCESS_TO" } },
    { data: { source: "Charlie", target: "Jira", label: "HAS_ACCESS_TO" } },
    { data: { source: "Bob", target: "Confluence", label: "HAS_ACCESS_TO" } },

    { data: { source: "Alice", target: "Admin", label: "ASSIGNED_ROLE" } },
    { data: { source: "Bob", target: "Read-Only", label: "ASSIGNED_ROLE" } },
    { data: { source: "Charlie", target: "Editor", label: "ASSIGNED_ROLE" } },

    { data: { source: "Admin", target: "AWS", label: "GRANTS_ACCESS" } },
    { data: { source: "Admin", target: "GitHub", label: "GRANTS_ACCESS" } },
    { data: { source: "Read-Only", target: "Azure", label: "GRANTS_ACCESS" } },
    { data: { source: "Editor", target: "Jira", label: "GRANTS_ACCESS" } },
    { data: { source: "Editor", target: "Confluence", label: "GRANTS_ACCESS" } },

    { data: { source: "AWS", target: "SOC2", label: "FOLLOWS" } },
    { data: { source: "Azure", target: "ISO27001", label: "FOLLOWS" } },
```

```
    { data: { source: "GitHub", target: "SOC2", label: "FOLLOWS" } },
    { data: { source: "Jira", target: "ISO27001", label: "FOLLOWS" } }
  ]);

  const handleQuery = async () => {
    if (!query) return;

    // Simulating backend call
    const response = await fetch("/api/chatbot", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ query }),
    });
    const data = await response.json();
    setGraphData(data.graph);
  };

  return (
    <div className="p-4 space-y-4">
      <Card>
        <CardContent className="flex space-x-2 p-4">
          <Input
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            placeholder="Ask about user access..."
          />
          <Button onClick={handleQuery}>Ask</Button>
        </CardContent>
      </Card>

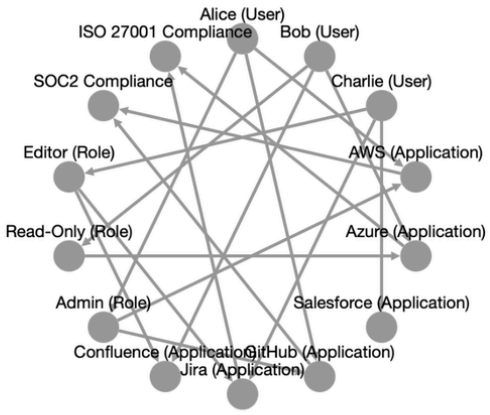
      {graphData && (
        <Card>
          <CardContent>
            <CytoscapeComponent
              elements={graphData}
              style={{ width: "100%", height: "400px" }}
              layout={{ name: "circle" }}
              stylesheet={[
                { selector: "node", style: { label: "data(label)" } },
                { selector: "edge", style: { "curve-style": "bezier", "target-arrow-shape": "triangle" } },
              ]}
            />
          </CardContent>
        </Card>
      )}
    </div>
  );
};

export default TrustXChatbot;

```

Ask about user access...

Ask



Graph DB Modeling

1️⃣ Ownership & Compliance

** (Tenant)-[:OWNS_APPLICATION]→(Application)
(Application)-[:COMPLIES_WITH]→(Compliance)
(Application)-[:BELONGS_TO_ENTERPRISE]→(Enterprise)

2️⃣ Enterprise Structure

(Enterprise)-[:CONTAINS_ORG]→(Organization)

(Enterprise)-[:HAS_USER]→(User)

(Enterprise)-[:FOLLOWS_COMPLIANCE]→(Compliance)

3️⃣ Organizational Structure

(Organization)-[:PART_OF_ENTERPRISE]→(Enterprise)
(Organization)-[:CONTROLS_APPLICATION]→(Application)

(Organization)-[:HAS_USER]→(User) (Organization)-[:OWNS_RESOURCE]→(Resource)

(Organization)-[:DEFINES_ROLE]→(Role)

** Users → groups
Groups → roles

Users → roles

4️⃣ User Access & Role Management

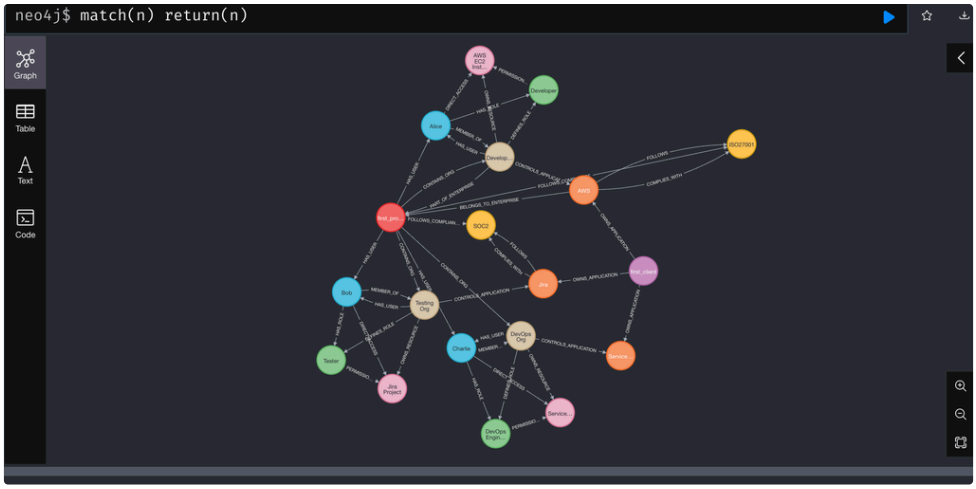
(User)-[:MEMBER_OF]→(Organization)

(User)-[:HAS_ROLE]→(Role)

(User)-[:DIRECT_ACCESS]→(Resource)

(Role)-[:PERMISSION_FOR]→(Resource)

sample_graphdb_records.json


neo4j$ match(n) return(n)
Graph
Table
Text
Code
AWS SCP Role
Developer
Alice
Develop...
ISO27001
AWS
test_pwd
SOC2
Jira
Bob
Testing Org
DevOps Org
Charlie
Service
Tester
Jira Project
DevOps Prem...
Service...
HAS_USER
MEMBER_OF
HAS_ROLE
CONTROLS_APPLICATION
FOLLOWS
COMPLIES_WITH
BELONG_TO_ENTERPRISE
FOLLOWS_COMPLIANCE
OWNS_APPLICATION
PERMISSION

3️⃣ Risk Analytics & AI/ML Models

Objective:

Identify **high-risk users, permissions, and anomalies** using AI-driven analytics.

Risk Factors:

- **Excessive Permissions** (Users with admin rights across multiple critical systems).
- **Dormant Privileged Accounts** (Accounts with no recent activity).
- **Anomalous Access Patterns** (Unusual login times, locations, or privilege escalations).
- **Shared Credentials** (Multiple users accessing an account).

AI/ML Models for Risk Detection:

- **Anomaly Detection:**
  - Train models to detect deviations from baseline user behavior.
  - Techniques: Isolation Forests, Autoencoders, k-Means Clustering.
- **Role-Based Access Optimization:**
  - Identify **least privilege access** recommendations using graph algorithms.
  - Techniques: PageRank, Community Detection (Graph AI).
- **Privileged Escalation Monitoring:**
  - Detect **sudden privilege changes** using time-series analysis.

Output:

- **Automated risk scoring** for users and roles, triggering alerts.

4️⃣ Chatbot & Graph Visualization

Objective:

Allow SOC analysts to **query user access footprints interactively**.

1️⃣ TrustX Chatbot (Query Engine)

Users can ask:

- *"Show me Alice's access footprint."*
- *"Who has admin access to AWS?"*
- *"Are there dormant privileged accounts?"*

How it Works:

1. **Natural Language Processing (NLP)**
   - Use OpenAI GPT or Hugging Face to parse user queries.
   - Example:

     ```
     1 css
     ```

     CopyEdit

     `Input: "Who has admin access to AWS?" Parsed Query: MATCH (u:User)-[:HAS_ACCESS_TO]->(a:Application {name:"AWS"})`
2. **Query Execution**
   - Convert parsed queries into **Cypher/SPARQL queries**.
   - Fetch results from the graph database.
3. **Chatbot Response**
   - Return results as:
     - **Text-based summary** (*"The following users have admin access: Alice, Bob."*)
     - **Interactive graph visualization**

2️⃣ Knowledge Graph Visualization

We'll use **D3.js or Cytoscape.js** to display an interactive graph.

Graph UI Features:

- **Draggable Nodes** (Users, Applications, Roles).
- **Color-Coded Risks**:
  - 🟢 Regular User
  - 🟠 Elevated Privileges
  - 🔴 High-Risk Admin
- **Click to Expand** (Explore connections).

Frontend Stack:

Component	Technology
Chatbot UI	React.js + OpenAI API

Graph Rendering	D3.js / Cytoscape.js
Backend API	FastAPI / Flask (Python)
Graph Database	Neo4j / Amazon Neptune

Example Visualization Query

Find all **privileged** users in AWS:

```
1 cypher
```

CopyEdit

```
MATCH (u:User)-[r:HAS_ACCESS_TO]->(a:Application) WHERE a.name = "AWS" AND r.privilege = "Admin" RETURN u
```

Output:

✅ **SOC analysts can query and visualize user access dynamically.**

Chatbot + Knowledge Graph Demo Plan

Got it! We'll start with an interactive demo for the TrustX Chatbot + Knowledge Graph Visualization. Here's how we'll break it down:

Chatbot + Knowledge Graph Demo Plan

Goal:

- Enable SOC analysts to **ask queries** about user access.
- Show **dynamic knowledge graph visualizations** of access footprints.
- Allow **real-time interaction** with access data.

1️⃣ Core Features of the Chatbot

Feature	Description
Natural Language Querying	SOC analysts can ask access-related questions.
Contextual Graph Search	Convert questions into graph database queries.
Real-Time Visualization	Display interactive access relationships.
Risk Highlighting	Flag excessive permissions and anomalies.

2️⃣ Chatbot Query Flow

1. **User Inputs a Query**
   - *Example:* "Show me Alice's access footprint."
2. **Chatbot Processes the Query**
   - NLP **parses keywords** (user, application, role).
   - Query converted into **graph database request**.
3. **Fetch Data from Graph Database**
   - Run Cypher/SPARQL query on **Neo4j/Amazon Neptune**.
4. **Display Visualization**
   - Show interactive **graph with users, roles, and privileges**.

3️⃣ Interactive Graph Features

Feature	Description
Nodes (Entities)	Users, Applications, Roles, Resources
Edges (Relationships)	"HAS_ACCESS_TO", "ASSIGNED_ROLE", etc.
Color Coding	🟢 Regular user, 🟠 Elevated access, 🔴 High-risk admin

Draggable Elements	SOC analysts can interact with the graph.
Click to Expand Details	Show user permissions & risk levels.

4️⃣ Tech Stack

Component	Technology
Chatbot UI	React.js (Frontend)
NLP Processing	OpenAI GPT / Hugging Face
Backend API	FastAPI / Flask (Python)
Graph Rendering	D3.js / Cytoscape.js
Database	Neo4j / Amazon Neptune

5️⃣ Sample Query Conversions

Example 1: Who has admin access to AWS?

- **User Query:** `"Who has admin access to AWS?"`
- **Generated Cypher Query:**

```
cypher
```

CopyEdit

```
MATCH (u:User)-[r:HAS_ACCESS_TO]->(a:Application) WHERE a.name = "AWS" AND r.privilege = "Admin" RETURN u
```

- **Graph Output:**
  - Nodes: Users with admin access.
  - Highlighted 🔴 **(high-risk users)**.

Example 2: Show Alice's Access Footprint

- **User Query:** `"Show me Alice's access footprint."`
- **Generated Cypher Query:**

```
cypher
```

CopyEdit

```
MATCH (u:User {name: "Alice"})-[r:HAS_ACCESS_TO]->(a:Application) RETURN u, r, a
```

- **Graph Output:**
  - **Alice's connections** to applications, roles, and permissions.

📌 Next Step: Build the Demo

React-based chatbot with interactive graph visualization using `Cytoscape.js`.

```
import { useState } from "react";
import { Input } from "@/components/ui/input";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import CytoscapeComponent from "react-cytoscapejs";

const TrustXChatbot = () => {
  const [query, setQuery] = useState("");
  const [graphData, setGraphData] = useState(null);

  const handleQuery = async () => {
    if (!query) return;

    // Simulate backend call
    const response = await fetch("/api/chatbot", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ query }),
    });
    const data = await response.json();
    setGraphData(data.graph);
  };

  return (
    <div className="p-4 space-y-4">
      <Card>
        <CardContent className="flex space-x-2 p-4">
          <Input
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            placeholder="Ask about user access..."
          />
          <Button onClick={handleQuery}>Ask</Button>
        </CardContent>
      </Card>

      {graphData && (
        <Card>
          <CardContent>
            <CytoscapeComponent
              elements={graphData}
              style={{ width: "100%", height: "400px" }}
              layout={{ name: "circle" }}
              stylesheet={[
                { selector: "node", style: { label: "data(label)" } },
                { selector: "edge", style: { "curve-style": "bezier", "target-arrow-shape": "triangle" } },
              ]}
            />
          </CardContent>
        </Card>
      )}
```

```
    </div>
  );
};

export default TrustXChatbot;

```

How It Works:

1. Users **input queries** related to access management.
2. The chatbot **sends the query** to a backend endpoint (`/api/chatbot`).
3. The backend **processes the query** (e.g., querying a graph database).
4. Results are **visualized dynamically** in a knowledge graph.

Next Steps:

- Set up the **backend API** (`/api/chatbot`) to process queries.
- Implement **sample graph data** to test the visualization.
- Add **color coding & risk indicators**.

5️⃣ Compliance & Reporting Features

Objective:

Align privileged access controls with regulatory standards (SOC 2, ISO 27001, NIST).

Key Features:

- **Automated Compliance Reports:**
  - Generate **audit-ready reports** for privileged access.
  - Identify **violations** in access control policies.
- **Compliance Mapping:**
  - Map permissions to **SOC 2, ISO 27001, NIST 800-53** frameworks.
  - Example:
    - *SOC 2 CC6.2:* Restrict privileged access → Detect overprivileged accounts.
    - *ISO 27001 A.9.2.3:* Least privilege principle → Suggest permission reductions.
- **Custom Compliance Rules Engine:**
  - Define & enforce **access control policies** dynamically.
  - Example Rule: *Flag all accounts with admin rights but no MFA enabled.*

Output:

- **Automated compliance insights** ensuring governance and risk control.

Hybrid Approach (LightRAG + GraphDB)

This PoC will combine **LightRAG (LLM-based NLP)** for **natural language queries** and **GraphDB (Neo4j/Amazon Neptune)** for **precise access control lookups.**

1️⃣ Objective

Enable **SOC analysts** to query user access footprints using natural language while ensuring precise access control insights through a **structured knowledge graph.**

Key Features

✅ **Natural Language Querying** via **LightRAG** (LLM-based retrieval)
✅ **Precise Graph-Based Access Queries** via **Neo4j**
✅ **Real-Time Knowledge Graph Visualization** (Cytoscape.js / D3.js)
✅ **Automated Risk Scoring** using **AI/ML for Anomaly Detection**

2️⃣ Architecture Overview

```
User → [TrustX Chatbot] → LightRAG (NLP) → GraphDB Query (Neo4j) → Response (Visualization)
```

Component Breakdown

Component	Technology Stack	Purpose
Chatbot UI	React.js + OpenAI API	Accepts user queries
NLP Processing	LightRAG (HKUDS)	Converts natural language to structured queries
Graph Query Engine	Neo4j / Amazon Neptune	Retrieves structured relationships
Graph Visualization	Cytoscape.js / D3.js	Displays user access footprint
Data Extraction	GitHub API, AWS IAM, Jira API, Okta	Fetches user access data
Risk Analysis	Isolation Forests, PageRank	Detects anomalies

3️⃣ Implementation Plan

Phase 1: Data Ingestion

📌 **Extract user access data from:**

- **Cloud Providers:** AWS IAM, Azure AD, GCP IAM
- **SaaS Applications:** GitHub, Jira, Salesforce, Confluence
- **SSO & IAM Systems:** Okta, Entra

📌 **Normalize access data:**

- Store structured relationships in **Neo4j GraphDB**
- Example Schema:

```
(User)-[:HAS_ACCESS_TO]->(Application)
(User)-[:ASSIGNED_ROLE]->(Role)
(Role)-[:GRANTS_ACCESS]->(Resource)
```

Phase 2: Query Processing (LightRAG + GraphDB)

📌 **Step 1: User Query in Natural Language**

- Example: `"Who has admin access to AWS?"`

📌 **Step 2: LightRAG Interprets Query**

- Converts `"Who has admin access to AWS?"` into a structured **GraphDB query.**
- **Example Cypher Query for Neo4j:**

```
MATCH (u:User)-[r:HAS_ACCESS_TO]->(a:Application)
WHERE a.name = "AWS" AND r.privilege = "Admin"
RETURN u
```

📌 **Step 3: Query Execution & Data Retrieval**

- **Neo4j/Amazon Neptune** executes query.
- Returns **structured response** (list of privileged users).

📌 **Step 4: Response Formatting & Graph Visualization**

- Display results **as a visual graph** in Cytoscape.js.
- Color-code risk levels:
  - 🟢 **Regular Users**
  - 🟠 **Elevated Privileges**
  - 🔴 **High-Risk Admins**

Phase 3: Risk Analysis & Alerting

📌 **Automate Risk Scoring for Users & Roles**

- Detect **Excessive Privileges** using **Graph Algorithms (PageRank, Centrality)**.
- Identify **Dormant Privileged Accounts** with **Anomaly Detection (Isolation Forests, Autoencoders).**

📌 **Example: Detecting Unusual Privilege Escalations**

- **Time-Series Analysis** of permission changes.
- **Alert if a user gains admin access unexpectedly.**

📌 **Generate Automated Alerts**

- **Slack / Email Notifications** for high-risk users.
- **Example Alert:** `"ALERT: User Alice gained admin access to AWS unexpectedly!"`

4️⃣ Expected Outcome

✅ **SOC Analysts can use Natural Language to Query Access Data**
✅ **LightRAG provides fast, context-aware responses**
✅ **GraphDB ensures precise access control insights**
✅ **Anomaly detection flags excessive permissions**
✅ **Interactive visualization simplifies access reviews**

5️⃣ Next Steps

- 🔹 **Set up LightRAG & fine-tune NLP models for access control queries.**
- 🔹 **Build GraphDB schema & ingest user access data.**
- 🔹 **Integrate LLM-based chatbot with GraphDB query execution.**
- 🔹 **Develop a frontend (React.js) for chatbot & visualization.**
- 🔹 **Implement risk scoring models for privileged accounts.**

6️⃣ Additional Considerations

- **Multi-Tenant Access Isolation:** Ensure queries are scoped to a specific **tenant/customer**.
- **Fine-tune LLM responses:** Train LightRAG embeddings for **cybersecurity-specific queries**.
- **Performance Optimization:** Use **caching (Redis)** to reduce query latency.

Queries That Require Proper Reasoning (Beyond Simple NL-to-Cypher Conversion)

Here are categorized questions that require **contextual reasoning, multi-hop traversal, temporal analysis, and anomaly detection**, rather than just direct Cypher query translation.

1️⃣ Queries Requiring Multi-Hop Traversal & Indirect Inferences

These queries require analyzing **multiple relationships** before answering.

- **Which users indirectly inherited privileged access to AWS?**
- **Which users have access to sensitive data through multiple group memberships?**
- **Which users have elevated permissions because of inherited roles?**
- **Who can delete resources in AWS through indirect role assignments?**

2️⃣ Queries Requiring Temporal Reasoning

These queries involve **analyzing changes over time**, not just static lookups.

- **Has Alice recently escalated her privileges?**
- **Which users gained admin rights in the last 24 hours?**
- **Who has recently lost access to critical applications?**
- **Which users' access patterns have changed significantly in the last month?**

3️⃣ Queries Requiring Risk-Based Analysis

These queries require **evaluating multiple factors** to determine a risk level.

- **Who are the top 5 highest-risk users based on their access and behavior?**
- **Which users have excessive permissions across multiple critical systems?**
- **Which dormant privileged accounts still exist in the system?**
- **Which admin users have never used their elevated permissions?**

4️⃣ Queries Requiring Conditional Reasoning & Compliance Validation

These queries require checking **if access aligns with compliance policies.**

- **Are there any compliance violations in privileged access?**
- **Which users have permissions that violate SOC2 or ISO27001 policies?**
- **Which users have access they should not have based on company policies?**
- **Which applications do not meet compliance requirements?**

5️⃣ Queries Requiring Statistical & Behavioral Analysis

These queries involve **detecting unusual behavior based on statistical baselines.**

- **Which users have logged in from multiple unusual locations?**

- **Which users have performed privilege escalations at unusual hours?**
- **Which users have accessed more systems than usual in the last week?**
- **Which users' login frequency has suddenly increased?**

Profile360 Requirements Doc (Draft)

✅ Objective

Create a **comprehensive, visual user access profile panel** for **Profile360**, showing real-time, graph-based insights into a user's **identity, access relationships, role assignments, risk posture, and privilege level**.

🧠 Core Idea

When a SOC analyst or security engineer queries a user (e.g., "Profile Emma"), Profile360 should return:

1. A **central node (user)** with connections to:
- Applications
- Roles
- Resources
- Teams / Departments
- Compliance flags or alerts
2. A **detail panel** showing:
- Identity attributes
- Access summary
- Risk score
- Historical changes (e.g., privilege escalations)

🧩 Feature Requirements (User Profile View)

🟣 Node Type: User (Central)

Label: Full Name

Properties:

- First name / Last name
- Job title
- Department
- Email
- Username (GitHub, Entra, etc.)

- Entra UUID / Internal ID
- Profile avatar
- Assigned license (Premium, Basic)
- Created date
- Risk score (from AI)
- Flags (Dormant, Privileged, Anomaly Detected)

🔷 Connected Node Types

Type	Label	Relationships	Display Intent
Application	AWS, GitHub, Jira, etc.	HAS_ACCESS_TO	Show what apps the user can access
Role	Admin, Developer, Viewer	HAS_ROLE / ASSIGNED_ROLE	Highlight least privilege violations
Resource	Repos, Projects, Spaces	ACCESS_TO_RESOURCE	Reveal access to sensitive assets
Team / Department	Engineering, Finance	MEMBER_OF / WORKS_IN	Organizational context
Compliance Risk	SOC2, ISO, Flags	VIOLATES_POLICY, AT_RISK_FOR	AI-based indicators of access risk
Login Activity	IPs, Geo-locations, Devices	LOGGED_IN_FROM, USED_ON	Optional: anomaly surface

---

🧮 Calculated & Derived Data (AI/ML Engine)

- **Privilege Level Score** (based on access breadth and role criticality)
- **Dormancy Indicator** (no usage in last X days)
- **Anomaly Tags** (unusual login, after-hours access)
- **Inherited Permissions Graph** (via groups or role chains)
- **Compliance Risk Score** (based on role + access + behavior)

🖥️ UI Requirements

🔹 **Graph Visualization Pane**

- Central user node with radial layout
- Interactive relationships (click-to-expand)
- Color-coded nodes by type
- Edge labels visible (e.g., “HAS_ROLE”, “USED_BY”)

🔹 **User Details Sidebar**

- Sticky panel showing full profile
- Sections:
- Identity & Org Info
- Access Summary
- AI Insights & Risk
- Last Seen / Activity Logs
- Compliance flags / tags

🎯 **Example Query Output: “Profile Emma”**

- Center: 🟣 Emma Hart
- Connected to:
- 🔷 GitHub, Jira (HAS_ACCESS_TO)
- 🟠 Admin Role (HAS_ROLE)
- 🟩 Operations (DEPARTMENT)
- 🔴 Jira QA Project (ACCESS_TO_RESOURCE)
- ⚠️ Dormant > 30 days (Flag)
- 🧠 Risk Score: 87 (Elevated Privilege)

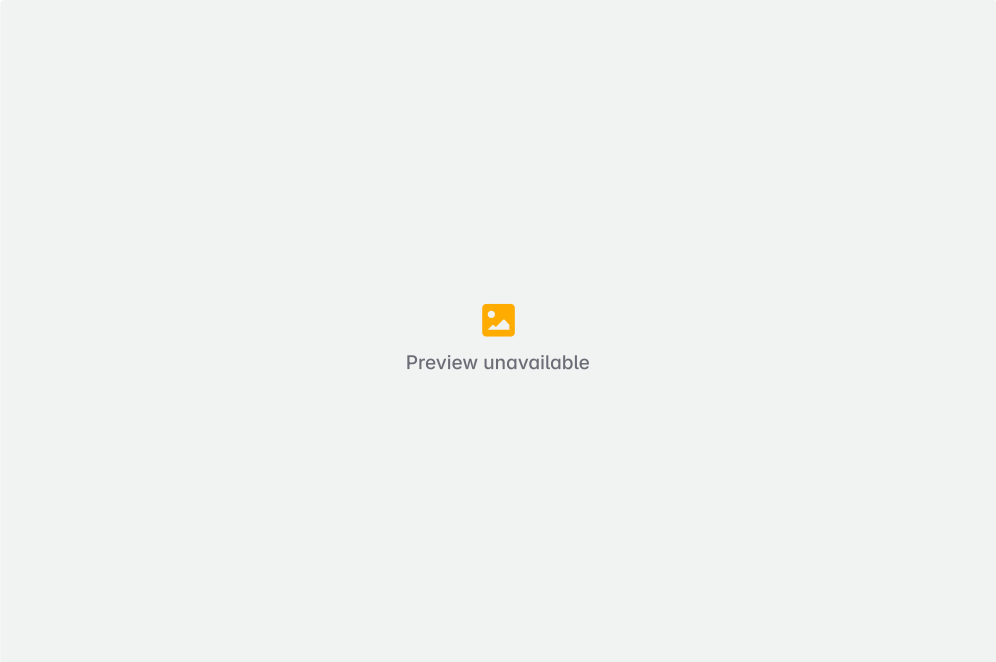
🔒 **Security Implications**

- Only authorized SOC users can view full profiles
- Profiles should **redact sensitive info** unless RBAC permits full view
- Logs must track all profile queries (auditable)

🚀 **Future Enhancements**

- **Access Timeline:** Graph of access and privilege changes over time
- **Peer Comparison:** See how Emma's access compares to others in her role
- **Risk Mitigation Suggestions:** "Remove X role", "Re-certify access to Y"
- **Ask AI Button:** "Why is Emma marked as high risk?" (Llama / GPT-based)

UI mockup



Set up Cloudflare Pages with Auth0 authentication

To set up **Cloudflare Pages** with **Auth0 authentication**, follow these steps:

Step 1: Deploy Your Cloudflare Pages Application

1. Go to **Cloudflare Pages** and create a new project.
2. Connect your GitHub or GitLab repository (or upload your code manually).
3. Configure your **build settings**, if required, and deploy the project.

Step 2: Create an Auth0 Application

1. Log in to your Auth0 Dashboard.
2. Navigate to **Applications** → **Create Application**.
3. Choose **Single Page Web Applications (SPA)**.
4. Click **Create**.

Step 3: Configure Allowed Callback URLs

In your Auth0 application settings, add the following:

- **Allowed Callback URLs**:

```arduino

https://your-cloudflare-pages-url.pages.dev/callback
```

- **Allowed Logout URLs**:

```arduino

https://your-cloudflare-pages-url.pages.dev
```

- **Allowed Web Origins**:

```arduino

https://your-cloudflare-pages-url.pages.dev
```

If using a custom domain, replace `your-cloudflare-pages-url.pages.dev` with your actual domain.

Step 4: Install Auth0 SDK in Your Cloudflare Pages Project

If you are using **React, Vue, or vanilla JavaScript**, install the Auth0 SDK:

```bash

npm install @auth0/auth0-spa-js
```

Step 5: Implement Authentication in Your Frontend

Modify your JavaScript frontend code to use Auth0 authentication.

Example (React)

```
javascript

import { Auth0Provider } from "@auth0/auth0-react"; const auth0Domain = "your-auth0-domain.us.auth0.com"; const auth0ClientId = "your-auth0-client-id"; function App() { return ( <Auth0Provider domain={auth0Domain} clientId={auth0ClientId} authorizationParams={{ redirect_uri: window.location.origin }} > <YourApp /> </Auth0Provider> ); } export default App;
```

Example (Vanilla JS)

```
javascript

import { createAuth0Client } from "@auth0/auth0-spa-js"; const auth0 = await createAuth0Client({ domain: "your-auth0-domain.us.auth0.com", clientId: "your-auth0-client-id", authorizationParams: { redirect_uri: window.location.origin } }); // Login function document.getElementById("login").addEventListener("click", async () => { await auth0.loginWithRedirect(); }); // Logout function document.getElementById("logout").addEventListener("click", async () => { await auth0.logout({ returnTo: window.location.origin }); });
```

Step 6: Protect API Routes (Optional)

If you need to protect API routes (e.g., in Cloudflare Workers), verify the Auth0 token in the request.

Example in a **Cloudflare Worker**:

```
javascript

export default { async fetch(request) { const authHeader = request.headers.get("Authorization"); if (!authHeader || !authHeader.startsWith("Bearer ")) { return new Response("Unauthorized", { status: 401 }); } const token = authHeader.split(" ")[1]; try { const response = await fetch(`https://your-auth0-domain.us.auth0.com/userinfo`, { headers: { Authorization: `Bearer ${token}` }, }); if (!response.ok) { return new Response("Unauthorized", { status: 401 }); } return new Response("Authorized", { status: 200 }); } catch (error) { return new Response("Error verifying token", { status: 500 }); } }, };
```

Step 7: Deploy & Test

1. Push your code to GitHub/GitLab if using automatic deployments.
2. Wait for Cloudflare Pages to build and deploy the application.
3. Test authentication by logging in and out.

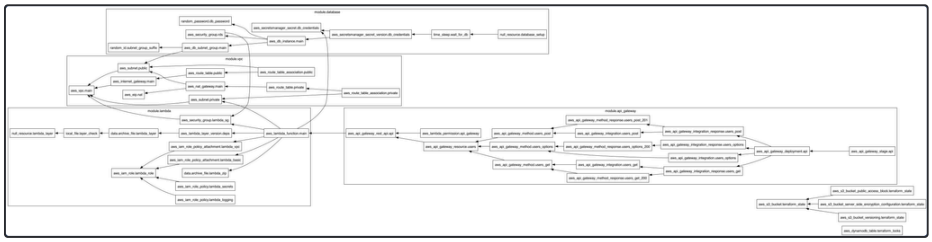
TrustX AWS Infrastructure Documentation

Overview

This document describes the AWS infrastructure deployment for the TrustX project using Terraform. The infrastructure includes API Gateway, Lambda functions, VPC networking, and RDS database components.
Git Repository : https://github.com/TrustXinc/atap-aws.git

Project Structure

```
atap-aws/
├── modules/
│   ├── api_gateway/
│   ├── lambda/
│   ├── rds/
│   └── vpc/
├── providers.tf
├── variables.tf
├── outputs.tf
└── main.tf
```



Infrastructure Components

1. Provider Configuration

```hcl
// filepath: /providers.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  backend "s3" {
    bucket         = "dev-trustx-terraform-state"
    key            = "terraform.tfstate"
    region         = "us-east-2"
    profile        = "trustx"
    dynamodb_table = "dev-trustx-terraform-locks"
  }
}
```

2. API Gateway

- REST API with `/users` endpoint
- Supports GET and POST methods
- CORS enabled
- CloudWatch logging configured

3. Lambda Function

- Python 3.11 runtime
- Database operations for user management
- VPC integration
- Secrets Manager integration for database credentials

4. Database

- PostgreSQL RDS instance
- VPC private subnet deployment
- Automated backups enabled

Development Setup

Prerequisites

1. AWS CLI installed and configured
2. Terraform >= 1.0
3. Python 3.11
4. PostgreSQL client

AWS Profile Configuration

```
aws configure --profile trustx
```

Local Development

1. Clone the repository

```
git clone <repository-url>
cd atap-aws
```

2. Initialize Terraform

```
terraform init
```

3. Test Lambda Function Locally

```
cd modules/lambda/src
sam local start-api
```

Deployment

Infrastructure Deployment

```
# Plan changes
terraform plan

# Apply changes
terraform apply
```

Testing Deployed API

```
# Get API URL
export API_URL=$(terraform output -raw api_url)

# Test GET endpoint
curl -X GET "${API_URL}/users"

# Test POST endpoint
curl -X POST "${API_URL}/users" \
  -H "Content-Type: application/json" \
  -d '{"name": "Test User", "email": "test@example.com"}'
```

Security

VPC Configuration

- Private subnets for RDS and Lambda
- NAT Gateway for outbound traffic
- VPC Endpoints for AWS services

IAM Roles

- Lambda execution role with minimal permissions
- RDS access limited to Lambda function
- API Gateway CloudWatch logging role

Secrets Management

- Database credentials in AWS Secrets Manager
- Automatic rotation enabled
- Encrypted using KMS

Monitoring and Logging

CloudWatch Logs

- API Gateway access logs
- Lambda function logs

- RDS logs

Metrics

- Lambda execution metrics
- API Gateway request metrics
- Database performance metrics

Troubleshooting

Common Issues

1. API Gateway 500 errors
   - Check Lambda CloudWatch logs
   - Verify VPC connectivity
   - Check database connection
2. Database connectivity
   - Verify security group rules
   - Check VPC routing
   - Validate credentials in Secrets Manager

Useful Commands

```
# View Lambda logs
aws logs tail /aws/lambda/your-function-name --profile trustx

# Check API Gateway deployments
aws apigateway get-deployments --rest-api-id your-api-id --profile trustx

# Test database connection
psql -h your-db-host -U your-username -d your-database
```

Contributing

1. Fork the repository
2. Create a feature branch
3. Submit a pull request
4. Ensure CI/CD checks pass

Contact

- Project Owner: Rama Kesani

Lambda Development Guide with Auth0 and Dependencies Layers

Overview

This guide explains how to develop new Lambda functions using the established Auth0 authentication and dependencies layers.
Repo: https://github.com/TrustXinc/atap-aws.git

Available Lambda Layers

1. Auth0 Validator Layer

- **Purpose**: Handles Auth0 token validation
- **Location**: `modules/lambda/layer/python/auth0_validator/`
- **Key Components**:
  - `validator.py`: Core validation logic
  - `requirements.txt`: Dependencies (python-jose, requests)

2. Dependencies Layer

- **Purpose**: Common project dependencies
- **Location**: `modules/lambda/layer/`
- **Key Dependencies**:
  - psycopg2-binary: PostgreSQL database driver
  - boto3: AWS SDK

Creating a New Lambda Function

1. Function Structure

```python
import json
import logging
from auth0_validator import Auth0Validator

# Set up logging
logger = logging.getLogger()
logger.setLevel(logging.INFO)

def create_response(status_code, body):
    """Helper function for consistent response format"""
    return {
        'statusCode': status_code,
        'headers': {
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*',
            'Access-Control-Allow-Headers': 'Content-Type,X-Amz-Date,Authorization,X-Api-Key,X-Amz-Security-Token',
            'Access-Control-Allow-Methods': 'GET,POST,OPTIONS'
        },
        'body': json.dumps(body)
    }

```

```python
def lambda_handler(event, context):
    try:
        # Validate Auth0 token
        try:
            validator = Auth0Validator()
            headers = event.get('headers', {}) or {}
            token_payload = validator.validate_token(headers.get('Authorization', ''))
            logger.info("Token validated successfully")
        except ValueError as e:
            return create_response(401, {'error': 'Unauthorized', 'message': str(e)})

        # Your function logic here
        # ...

        return create_response(200, {'message': 'Success'})

    except Exception as e:
        logger.error("Error: %s", str(e))
        return create_response(500, {'error': 'Internal server error'})
```

2. Database Access

```python
def get_db_connection():
    """Helper function for database connections"""
    secret_name = os.environ.get('SECRET_NAME')
    region_name = os.environ.get('AWS_REGION')

    session = boto3.session.Session()
    client = session.client(
        service_name='secretsmanager',
        region_name=region_name
    )

    try:
        secret = json.loads(client.get_secret_value(SecretId=secret_name)['SecretString'])
        conn = psycopg2.connect(
            host=secret['host'],
            port=secret.get('port', 5432),
            database=secret['dbname'],
            user=secret['username'],
            password=secret['password'],
            cursor_factory=RealDictCursor
        )
        return conn
    except Exception as e:
        logger.error(f"Database connection error: {str(e)}")
        raise
```

Terraform Configuration

1. Lambda Function Resource

```hcl
resource "aws_lambda_function" "your_function" {
  filename         = data.archive_file.your_function_zip.output_path
  function_name    = "${var.project_name}-${var.environment}-your-function"
  role             = aws_iam_role.lambda_role.arn
  handler          = "lambda_function.lambda_handler"
```

```
  runtime         = "python3.11"
  timeout         = 30
  memory_size     = 256

  layers = [
    aws_lambda_layer_version.deps.arn,
    aws_lambda_layer_version.auth0.arn
  ]

  environment {
    variables = {
      SECRET_NAME = var.secret_name
      AUTH0_DOMAIN = var.auth0_domain
      AUTH0_AUDIENCE = var.auth0_audience
    }
  }

  vpc_config {
    subnet_ids         = var.private_subnet_ids
    security_group_ids = [aws_security_group.lambda_sg.id]
  }
}
```

2. API Gateway Integration

```
# Method
resource "aws_api_gateway_method" "your_method" {
  rest_api_id   = aws_api_gateway_rest_api.api.id
  resource_id   = aws_api_gateway_resource.your_resource.id
  http_method   = "POST"  # or GET, PUT, DELETE
  authorization = "CUSTOM"
  authorizer_id = aws_api_gateway_authorizer.auth0.id
}

# Integration
resource "aws_api_gateway_integration" "your_integration" {
  rest_api_id = aws_api_gateway_rest_api.api.id
  resource_id = aws_api_gateway_method.your_method.resource_id
  http_method = aws_api_gateway_method.your_method.http_method

  integration_http_method = "POST"
  type                    = "AWS_PROXY"
  uri                     = aws_lambda_function.your_function.invoke_arn
}
```

Best Practices

1. Error Handling

- Always use try/except blocks
- Return proper HTTP status codes
- Include meaningful error messages
- Log errors with sufficient context

2. Logging

- Use the standard Python logging module

- Include relevant context in log messages
- Use appropriate log levels (INFO, ERROR, etc.)
- Log sensitive information only at DEBUG level

3. Database Connections

- Always close database connections in finally blocks
- Use connection pooling for better performance
- Handle connection errors gracefully

4. Security

- Always validate Auth0 tokens
- Use environment variables for sensitive data
- Follow the principle of least privilege
- Sanitize user inputs

5. Response Format

- Use consistent response structure
- Include proper CORS headers
- Handle binary responses appropriately
- Use proper content types

Testing

Local Testing

```
# Set environment variables
export AUTH0_DOMAIN="your-domain.auth0.com"
export AUTH0_AUDIENCE="your-audience"
export SECRET_NAME="your-secret-name"

# Test with curl
curl -X POST \
  <https://your-api.execute-api.region.amazonaws.com/dev/your-endpoint> \
  -H 'Authorization: Bearer YOUR_TOKEN' \
  -H 'Content-Type: application/json' \
  -d '{"key": "value"}'
```

CloudWatch Logs

- Check both function and API Gateway logs
- Use log insights for complex queries
- Set up log metrics and alarms

Deployment

- Follow the established CI/CD pipeline
- Use separate environments (dev, staging, prod)
- Test thoroughly before production deployment
- Monitor CloudWatch metrics after deployment

API Gateway URLs

The API Gateway URLs follow this pattern:

```
<https://{api-id}.execute-api.{region}.amazonaws.com/{stage}/{resource}>

Example:
<https://qtx2xb2qoe.execute-api.us-east-2.amazonaws.com/dev/users>
```

Auth0 Token Format

Auth0 tokens should be included in the Authorization header with the Bearer scheme:

```
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCIsImtpZCI6Im5...
```

You can get a new token using Auth0's client credentials flow:

```
curl --request POST \
  --url <https://{your-domain}.auth0.com/oauth/token> \
  --header 'content-type: application/x-www-form-urlencoded' \
  --data 'grant_type=client_credentials&client_id={your-client-id}&client_secret={your-client-secret}&audience={your-api-identifier}'
```

Troubleshooting Guide

Common Issues

1. Auth0 Token Issues
   - Check token expiration
   - Verify audience matches Auth0 settings
   - Confirm proper token format (Bearer scheme)
   - Check CloudWatch logs for validation errors
2. Database Connection Issues
   - Verify VPC and security group settings
   - Check RDS security group allows Lambda IP range
   - Confirm secret exists in Secrets Manager
   - Look for connection timeout errors in logs
3. API Gateway Issues
   - Verify CORS headers for browser requests
   - Check API Gateway stage deployment
   - Review CloudWatch logs for request/response cycle
   - Confirm Lambda permissions for API Gateway

Useful CloudWatch Queries

```
# Find Auth0 validation errors
filter @message like "Token validation error"

# Track database connection issues
filter @message like "Database connection error"
```

Auth0 Authentication Flow for Cloudflare Pages

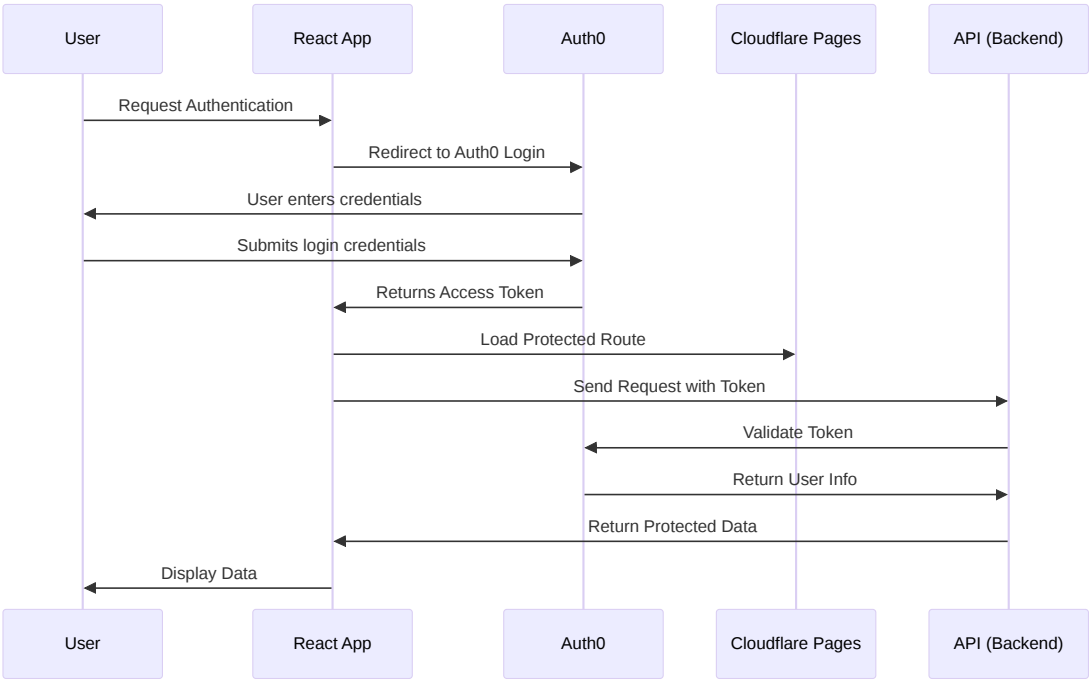
Overview

This document details the authentication flow for integrating **Auth0** with a **React.js** application deployed on **Cloudflare Pages**. The flow ensures secure authentication and user session management.

Authentication Flow Diagram

```
sequenceDiagram
  participant User
  participant React App
  participant Auth0
  participant Cloudflare Pages
  participant API (Backend)

  User->>React App: Request Authentication
  React App->>Auth0: Redirect to Auth0 Login
  Auth0->>User: User enters credentials
  User->>Auth0: Submits login credentials
  Auth0->>React App: Returns Access Token
  React App->>Cloudflare Pages: Load Protected Route
  React App->>API (Backend): Send Request with Token
  API (Backend)->>Auth0: Validate Token
  Auth0->>API (Backend): Return User Info
  API (Backend)->>React App: Return Protected Data
  React App->>User: Display Data
```



Flow Breakdown

1. User Requests Authentication

- The user initiates authentication by clicking the **Login** button in the React application.

2. Redirect to Auth0

- The React app redirects the user to the **Auth0 login page**.

3. User Enters Credentials

- The user enters their **username/password** or logs in via a social provider (e.g., Google, GitHub).

4. Auth0 Validates Credentials

- Auth0 verifies the credentials and, if valid, generates an **access token**.

5. Redirect Back to React App

- Auth0 redirects the user back to the React application along with the **access token**.

6. Load Protected Route

- The React app uses the access token to authenticate the user and loads **protected components**.

7. API Requests with Access Token

- The React app sends API requests with the **access token** to a backend server.

8. Token Validation by Backend

- The backend server validates the token with Auth0.

9. Return Protected Data

- If the token is valid, the backend returns the requested data.

10. Display User Data

- The React app displays the protected content based on the response.

Conclusion

This flow ensures **secure authentication** while leveraging **Auth0** as an identity provider for Cloudflare Pages-hosted applications. The process involves **token-based authentication**, allowing the React app to securely communicate with a backend API.

References

- Auth0 React SDK Docs
- Cloudflare Pages Docs
- Deploying React to Cloudflare Pages

TrustX ATAP Multi-Tenant Architecture (Shared Database, Separate Schemas)

1. Introduction

This document outlines the **Shared Database, Separate Schemas** multi-tenant architecture for TrustX ATAP, focusing on compliance monitoring and privileged access management. This model ensures **data isolation, security, and scalability** while keeping operations efficient.

---

2. Multi-Tenant Model Overview

Architecture Choice: Shared Database, Separate Schemas

Each tenant (customer) has a **separate schema** within a **shared database**, eliminating the need for a `tenant_id` column in most tables.

Example Schema Layout:

```
trustx_main (metadata schema for all tenants)
trustx_tenant_1 (Schema for Tenant 1)
trustx_tenant_2 (Schema for Tenant 2)
trustx_tenant_3 (Schema for Tenant 3)
```

This ensures **logical separation** while keeping management overhead lower than fully separate databases.

> **Shared Database, Separate Schemas**
> - **Each tenant gets its own schema**, with identical table structures.
> - The application **dynamically switches schemas** based on the logged-in user.
>
> The **database isolates each tenant's data**, reducing the risk of cross-tenant access.

---

3. Database Schema Structure

1. Core Schema (`trustx_main`)

This schema stores metadata about tenants and global configurations.

- **Tenants Table:** Stores information on registered tenants.
- **Users Table:** Stores global user credentials and links them to their respective tenant schemas.

2. Tenant-Specific Schemas (`trustx_tenant_X`)

Each tenant has its own schema containing compliance and security data.

- **Users** – Specific to the tenant.
- **Frameworks** – Compliance frameworks (SOC 2, ISO 27001, etc.).
- **Controls** – Security controls mapped to frameworks.
- **Audits** – Stores audit records.
- **Risk Assessments** – Tracks potential compliance/security risks.
- **Findings** – Compliance gaps identified during audits.
- **Evidence** – Uploaded documents supporting compliance efforts.
- **Policies** – Security policies linked to frameworks.

- **Training** – Security awareness and compliance training records.

---

4. Continuous Compliance Monitoring Data Model

Overview

Continuous Compliance Monitoring ensures organizations stay compliant with security frameworks like **SOC 2, ISO 27001, NIST, and GDPR** by automating evidence collection, tracking control statuses, and integrating AI-driven insights.

Key Components

1. **Compliance Frameworks**
   - Defines regulatory frameworks (SOC 2, ISO 27001, etc.)
   - Mapped to specific security and governance controls.
2. **Controls**
   - Security and governance controls linked to frameworks.
   - Tracks compliance status dynamically.
3. **Audits & Evidence Collection**
   - Stores audit records and compliance findings.
   - Links evidence (documents, reports) to relevant controls.
4. **Automated Monitoring & AI Insights**
   - Uses AI to detect compliance gaps and suggest remediations.
   - Tracks historical compliance trends.

Data Model Structure



This model enables **automated compliance tracking**, reduces **manual audit overhead**, and provides real-time insights into security posture.

---

5. Schema Implementation and Security

1. Creating a New Tenant Schema

```
CREATE SCHEMA trustx_tenant_123;
```

2. Cloning Base Tables for New Tenants

```
CREATE TABLE trustx_tenant_123.users AS SELECT * FROM trustx_template.users WHERE 1=0;
```

3. Granting Access to Tenant-Specific Users

```
CREATE USER tenant_123_user WITH PASSWORD 'securepassword';
GRANT USAGE ON SCHEMA trustx_tenant_123 TO tenant_123_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA trustx_tenant_123 TO tenant_123_user;
```

4. Dynamic Schema Switching for Queries

```
SET search_path TO trustx_tenant_123;
SELECT * FROM users;
```

---

6. API & Application Integration

1. API Request Flow

1. User logs in via **Auth0**.
2. API determines `tenant_id` and resolves schema name (`trustx_tenant_123`).
3. Database connection sets `search_path` dynamically.
4. Query executes within the correct schema.

2. Handling Schema Selection in Backend (Example in Node.js)

```
async function getUserData(userId, tenantSchema) {
    const query = `SET search_path TO ${tenantSchema}; SELECT * FROM users WHERE user_id = $1;`;
    return db.query(query, [userId]);
}
```

---

7. Compliance & Security Best Practices

1. Data Isolation

- **Strict schema-based access control:** Each tenant has a dedicated DB user.
- **Ensure API restricts schema switching to authorized users only.**

2. Performance Optimization

- **Connection Pooling:** Use a database connection pooler (e.g., **PgBouncer**) to optimize performance.
- **Indexing:** Ensure primary keys and frequently used query columns are indexed.

3. Schema Versioning & Migrations

- **Use Flyway or Liquibase** to manage schema updates across all tenants.
- Keep a **template schema (`trustx_template`)** to simplify new tenant onboarding.

---

8. Summary of Key Benefits

✅ **Strong Data Isolation:** Each tenant has its own schema, ensuring compliance with security regulations.
✅ **Scalable & Cost-Effective:** Easier to manage than separate databases, while providing security benefits.
✅ **Efficient Query Execution:** Eliminates need for `tenant_id` filtering in queries.
✅ **Better Backup & Recovery:** Schema-specific backups allow targeted data restoration.

---

9. Next Steps

- **Implement schema creation automation** in backend service.
- **Set up API gateway logic** to dynamically switch schemas.
- **Define monitoring strategies** for per-tenant logging.

This architecture ensures TrustX ATAP remains **secure, scalable, and optimized** for multi-tenant compliance management.

Schema for TrustX Multi-Tenant Architecture (Shared Database, Separate Schemas)

ERD

1. Core Schema (trustx_main) - Stores metadata for tenants

```
CREATE SCHEMA trustx_main;

CREATE TABLE trustx_main.tenants (
    tenant_id SERIAL PRIMARY KEY,
    tenant_name VARCHAR(255) NOT NULL,
    tenant_description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE trustx_main.users (
    user_id SERIAL PRIMARY KEY,
    tenant_id INT REFERENCES trustx_main.tenants(tenant_id),
    username VARCHAR(255) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2. Template Schema for Tenants (trustx_template)

```
CREATE SCHEMA trustx_template;

CREATE TABLE trustx_template.users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(255) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE trustx_template.frameworks (
    framework_id SERIAL PRIMARY KEY,
    framework_name VARCHAR(255) NOT NULL,
    version VARCHAR(50),
    description TEXT,
    effective_date DATE,
    review_cycle VARCHAR(50)
);

CREATE TABLE trustx_template.controls (
    control_id SERIAL PRIMARY KEY,
    framework_id INT REFERENCES trustx_template.frameworks(framework_id),
    control_number VARCHAR(50) NOT NULL,
    control_title VARCHAR(255) NOT NULL,
    control_description TEXT
```

```
);

CREATE TABLE trustx_template.audits (
    audit_id SERIAL PRIMARY KEY,
    framework_id INT REFERENCES trustx_template.frameworks(framework_id),
    audit_type VARCHAR(100),
    start_date DATE,
    end_date DATE,
    status VARCHAR(50)
);
```

3. New Tenant Schema Creation (Example for Tenant 123)

```
CREATE SCHEMA trustx_tenant_123;

-- Clone Template Tables for Tenant
CREATE TABLE trustx_tenant_123.users AS TABLE trustx_template.users;
CREATE TABLE trustx_tenant_123.frameworks AS TABLE trustx_template.frameworks;
CREATE TABLE trustx_tenant_123.controls AS TABLE trustx_template.controls;
CREATE TABLE trustx_tenant_123.audits AS TABLE trustx_template.audits;
```

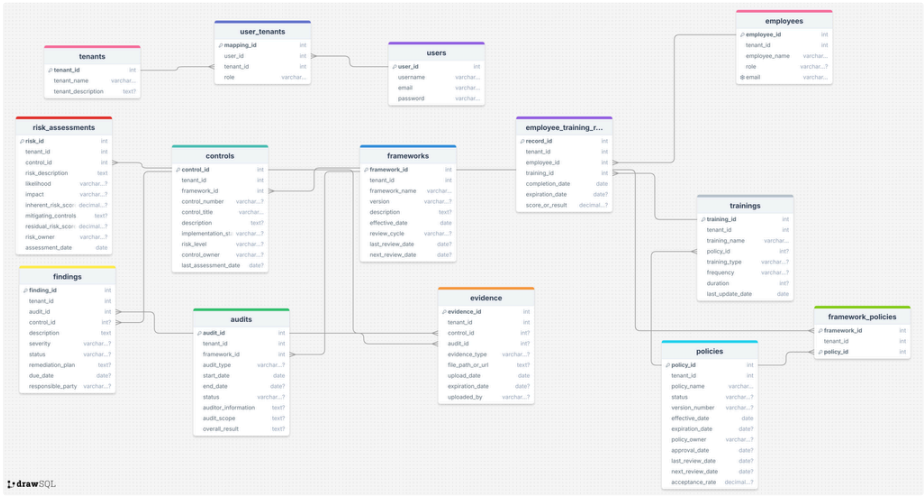
4. Grant Permissions for Tenant User

```
CREATE USER tenant_123_user WITH PASSWORD 'securepassword';
GRANT USAGE ON SCHEMA trustx_tenant_123 TO tenant_123_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA trustx_tenant_123 TO tenant_123_user;
```

5. Dynamic Schema Selection in Queries

```
SET search_path TO trustx_tenant_123;
SELECT * FROM users;
```

Database Schema Design



+---------------+

| public |

+---------------+

| Tenants |

| Users |

| User-Tenant |

+---------------+

+---------------+

| tenant_1 |

+---------------+

| Frameworks |

| Controls |

| Policies |

| Audits |

| Findings |

| Evidence |

| Risk Assess |

| Trainings |

| Employee Tr |

+---------------+

https://drawsql.app/teams/self-792/diagrams/testing

1. Tenant Table

- This table stores information about each tenant (company).

```
sql
CREATE TABLE public.tenants (
    tenant_id SERIAL PRIMARY KEY,
    tenant_name VARCHAR(100) NOT NULL,
    tenant_description TEXT
);
```

2. User Table

- A global user table in the public schema to manage user authentication across all tenants.

```
sql
CREATE TABLE public.users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL,
    password VARCHAR(255) NOT NULL
);
```

3. User-Tenant Mapping Table

- Maps users to their respective tenants and roles.

```
sql
CREATE TABLE public.user_tenants (
    mapping_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    tenant_id INTEGER NOT NULL,
    role VARCHAR(50) NOT NULL CHECK (role IN ('Compliance Administrator', 'Compliance Officer/Analyst', 'Risk Manager', 'IT/Security Specialist', 'Business Unit Manager', 'Auditor (Internal/External)', 'Executive/CEO')),
    FOREIGN KEY (user_id) REFERENCES public.users(user_id),
    FOREIGN KEY (tenant_id) REFERENCES public.tenants(tenant_id)
);
```

4. Tenant-Specific Schemas

- Create a separate schema for each tenant. This ensures data isolation and allows for customized structures if needed.

```
sql
-- Create schema for a tenant
CREATE SCHEMA IF NOT EXISTS tenant_1;
```

5. Tenant-Specific Tables

- Within each tenant's schema, create tables for frameworks, controls, policies, audits, findings, evidence, risk assessments, training, and employee training records.

Example for Frameworks table in tenant_1 schema:

```
Sql


-- frameworks table
CREATE TABLE tenant_1.frameworks (
    framework_id SERIAL PRIMARY KEY,
    tenant_id INTEGER NOT NULL,
    framework_name VARCHAR(100) NOT NULL,
    version VARCHAR(20),
    description TEXT,
    effective_date DATE NOT NULL,
    review_cycle VARCHAR(50),
    last_review_date DATE,
    next_review_date DATE
```

```
);
-- Controls table
CREATE TABLE tenant_1.controls (
    control_id SERIAL PRIMARY KEY,
    framework_id INTEGER NOT NULL,
    control_number VARCHAR(50),
    control_title VARCHAR(100) NOT NULL,
    description TEXT,
    implementation_status VARCHAR(50) CHECK (implementation_status IN ('Implemented', 'Partially Implemented', 'Not Implemented')),
    risk_level VARCHAR(50),
    control_owner VARCHAR(100),
    last_assessment_date DATE,
    FOREIGN KEY (framework_id) REFERENCES tenant_1.frameworks(framework_id)
);

-- Junction table for frameworks and policies
CREATE TABLE tenant_1.framework_policies (
    framework_id INTEGER NOT NULL,
    policy_id INTEGER NOT NULL,
    PRIMARY KEY (framework_id, policy_id),
    FOREIGN KEY (framework_id) REFERENCES tenant_1.frameworks(framework_id),
    FOREIGN KEY (policy_id) REFERENCES tenant_1.policies(policy_id)
);

-- Policies table
CREATE TABLE tenant_1.policies (
    policy_id SERIAL PRIMARY KEY,
    policy_name VARCHAR(100) NOT NULL,
    status VARCHAR(50) CHECK (status IN ('Draft', 'Approved', 'Retired')),
    version_number VARCHAR(20),
    effective_date DATE NOT NULL,
    expiration_date DATE,
    policy_owner VARCHAR(100),
    approval_date DATE,
    last_review_date DATE,
    next_review_date DATE,
    acceptance_rate DECIMAL(5,2)
);



-- Audits table
CREATE TABLE tenant_1.audits (
    audit_id SERIAL PRIMARY KEY,
    framework_id INTEGER NOT NULL,
    audit_type VARCHAR(50) CHECK (audit_type IN ('Internal', 'External', 'Certification')),
    start_date DATE NOT NULL,
    end_date DATE,
    status VARCHAR(50) CHECK (status IN ('Planned', 'In Progress', 'Completed')),
    auditor_information TEXT,
    audit_scope TEXT,
```

```
    overall_result TEXT,
    FOREIGN KEY (framework_id) REFERENCES tenant_1.frameworks(framework_id)
);

-- Findings table
CREATE TABLE tenant_1.findings (
    finding_id SERIAL PRIMARY KEY,
    audit_id INTEGER NOT NULL,
    control_id INTEGER,
    description TEXT NOT NULL,
    severity VARCHAR(50),
    status VARCHAR(50) CHECK (status IN ('Open', 'In Progress', 'Closed')),
    remediation_plan TEXT,
    due_date DATE,
    responsible_party VARCHAR(100),
    FOREIGN KEY (audit_id) REFERENCES tenant_1.audits(audit_id),
    FOREIGN KEY (control_id) REFERENCES tenant_1.controls(control_id)
);

-- Evidence table
CREATE TABLE tenant_1.evidence (
    evidence_id SERIAL PRIMARY KEY,
    control_id INTEGER,
    audit_id INTEGER,
    evidence_type VARCHAR(50) CHECK (evidence_type IN ('Document', 'Screenshot', 'Log')),
    file_path_or_url TEXT,
    upload_date DATE NOT NULL,
    expiration_date DATE,
    uploaded_by VARCHAR(100),
    FOREIGN KEY (control_id) REFERENCES tenant_1.controls(control_id),
    FOREIGN KEY (audit_id) REFERENCES tenant_1.audits(audit_id)
);

-- Risk Assessment table
CREATE TABLE tenant_1.risk_assessments (
    risk_id SERIAL PRIMARY KEY,
    control_id INTEGER NOT NULL,
    risk_description TEXT NOT NULL,
    likelihood VARCHAR(50),
    impact VARCHAR(50),
    inherent_risk_score DECIMAL(5,2),
    mitigating_controls TEXT,
    residual_risk_score DECIMAL(5,2),
    risk_owner VARCHAR(100),
    assessment_date DATE NOT NULL,
    FOREIGN KEY (control_id) REFERENCES tenant_1.controls(control_id)
);

-- Training table
CREATE TABLE tenant_1.trainings (
```

```
    training_id SERIAL PRIMARY KEY,
    training_name VARCHAR(100) NOT NULL,
    policy_id INTEGER,
    training_type VARCHAR(50) CHECK (training_type IN ('Online', 'In-person')),
    frequency VARCHAR(50) CHECK (frequency IN ('Annual', 'One-time')),
    duration INTEGER,
    last_update_date DATE NOT NULL,
    FOREIGN KEY (policy_id) REFERENCES tenant_1.policies(policy_id)
);


-- Employees table
CREATE TABLE tenant_1.employees (
    employee_id SERIAL PRIMARY KEY,
    employee_name VARCHAR(100) NOT NULL,
    role VARCHAR(50) CHECK (role IN ('Compliance Administrator', 'Compliance Officer/Analyst', 'Risk Manager', 'IT/Security Specialist', 'Business Unit Manager', 'Auditor (Internal/External)', 'Executive/CEO')),
    email VARCHAR(100) UNIQUE NOT NULL
);



-- Employee Training Records table
CREATE TABLE tenant_1.employee_training_records (
    record_id SERIAL PRIMARY KEY,
    employee_id INTEGER NOT NULL,
    training_id INTEGER NOT NULL,
    completion_date DATE NOT NULL,
    expiration_date DATE,
    score_or_result DECIMAL(5,2),
    FOREIGN KEY (employee_id) REFERENCES tenant_1.employees(employee_id),
    FOREIGN KEY (training_id) REFERENCES tenant_1.trainings(training_id)
);
```

Repeat similar structures for other tables if any new, within each tenant's schema.

6. Row-Level Security (RLS)

- Implement RLS to ensure users can only access data within their assigned tenant schema.

```
sql
-- Enable RLS on a table
ALTER TABLE tenant_1.frameworks ENABLE ROW LEVEL SECURITY;
ALTER TABLE tenant_1.controls ENABLE ROW LEVEL SECURITY;
ALTER TABLE tenant_1.policies ENABLE ROW LEVEL SECURITY;
ALTER TABLE tenant_1.audits ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE tenant_1.findings ENABLE ROW LEVEL SECURITY;
ALTER TABLE tenant_1.evidence ENABLE ROW LEVEL SECURITY;
ALTER TABLE tenant_1.risk_assessments ENABLE ROW LEVEL SECURITY;
ALTER TABLE tenant_1.trainings ENABLE ROW LEVEL SECURITY;
ALTER TABLE tenant_1.employees ENABLE ROW LEVEL SECURITY;
ALTER TABLE tenant_1.employee_training_records ENABLE ROW LEVEL SECURITY;



-- Create policy to restrict access based on user's tenant

CREATE POLICY tenant_1_frameworks_select_policy
ON tenant_1.frameworks
FOR SELECT
TO public
USING (current_user = 'tenant_1_user');


CREATE POLICY controls_isolation_policy
ON tenant_1.controls
USING (framework_id IN (
  SELECT framework_id
  FROM tenant_1.frameworks
  WHERE tenant_id = (SELECT current_setting('app.current_tenant')::INTEGER)
));

CREATE POLICY policies_isolation_policy
ON tenant_1.policies
USING (framework_id IN (
    SELECT framework_id
    FROM tenant_1.frameworks
    WHERE tenant_id = (SELECT current_setting('app.current_tenant')::INTEGER)
));

CREATE POLICY audits_isolation_policy
ON tenant_1.audits
USING (framework_id IN (
    SELECT framework_id
    FROM tenant_1.frameworks
    WHERE tenant_id = (SELECT current_setting('app.current_tenant')::INTEGER)
));


CREATE POLICY findings_isolation_policy
ON tenant_1.findings
USING (audit_id IN (
    SELECT audit_id
    FROM tenant_1.audits
    WHERE framework_id IN (
        SELECT framework_id
```

```
        FROM tenant_1.frameworks
        WHERE tenant_id = (SELECT current_setting('app.current_tenant')::INTEGER)
    )
));

CREATE POLICY evidence_isolation_policy
ON tenant_1.evidence
USING (
    control_id IN (
        SELECT control_id
        FROM tenant_1.controls
        WHERE framework_id IN (
            SELECT framework_id
            FROM tenant_1.frameworks
            WHERE tenant_id = (SELECT current_setting('app.current_tenant')::INTEGER)
        )
    )
    OR audit_id IN (
        SELECT audit_id
        FROM tenant_1.audits
        WHERE framework_id IN (
            SELECT framework_id
            FROM tenant_1.frameworks
            WHERE tenant_id = (SELECT current_setting('app.current_tenant')::INTEGER)
        )
    )
);

CREATE POLICY risk_assessments_isolation_policy
ON tenant_1.risk_assessments
USING (control_id IN (
    SELECT control_id
    FROM tenant_1.controls
    WHERE framework_id IN (
        SELECT framework_id
        FROM tenant_1.frameworks
        WHERE tenant_id = (SELECT current_setting('app.current_tenant')::INTEGER)
    )
));

CREATE POLICY trainings_isolation_policy
ON tenant_1.trainings
USING (policy_id IN (
    SELECT policy_id
    FROM tenant_1.policies
    WHERE framework_id IN (
        SELECT framework_id
        FROM tenant_1.frameworks
        WHERE tenant_id = (SELECT current_setting('app.current_tenant')::INTEGER)
    )
```

```
));

CREATE POLICY employees_isolation_policy

ON tenant_1.employees

USING (tenant_id = (SELECT current_setting('app.current_tenant')::INTEGER));



CREATE POLICY employee_training_records_isolation_policy

ON tenant_1.employee_training_records

USING (training_id IN (

    SELECT training_id

    FROM tenant_1.trainings

    WHERE policy_id IN (

        SELECT policy_id

        FROM tenant_1.policies

        WHERE framework_id IN (

            SELECT framework_id

            FROM tenant_1.frameworks

            WHERE tenant_id = (SELECT current_setting('app.current_tenant')::INTEGER)

        )

    )

));


-- Policy for Compliance Administrators to view all employees

CREATE POLICY employees_view_policy

ON tenant_1.employees

FOR SELECT

TO public

USING (EXISTS (

    SELECT 1

    FROM tenant_1.user_tenants

    WHERE role = 'Compliance Administrator'

));


-- Adjust the policy as needed based on your role-based access requirements
```

7. Database Roles

- Create database roles for each tenant to manage access to their schema.

```
sql

-- Create role for tenant_1

CREATE ROLE tenant_1_user WITH PASSWORD 'password';


-- Grant access to tenant_1 schema

GRANT ALL PRIVILEGES ON SCHEMA tenant_1 TO tenant_1_user;
```

This approach provides a robust multi-tenant architecture with data isolation, flexibility for customization, and efficient management of access rights. However, managing multiple schemas can become complex as the number of tenants grows, so ensure you have a solid strategy for schema updates and maintenance.

Compliance Database Model for TrustX

Overview

To build an efficient and scalable compliance automation platform within **TrustX**, we adopt a **hybrid database model** leveraging both **PostgreSQL (Relational DB)** and **Neo4j (Graph DB)**. This approach provides the best of both worlds:

- **PostgreSQL/MySQL**: For structured compliance data, audits, policies, and reports.
- **Neo4j**: For mapping relationships between SOC 2 controls, risks, frameworks, and evidence sources.
- **Multi-Tenant Architecture**: Ensures each tenant has its own schema while sharing a common structure.

1. PostgreSQL / MySQL - Structured Compliance Data

This relational database will store **structured compliance data**, such as:

- SOC 2 **Controls** (Control ID, Category, Description)
- **Policies & Procedures** mapped to controls
- **Compliance Evidence** (logs, screenshots, test results)
- **Audit Reports & History**
- **User Roles & Permissions**
- **Task Management** (Compliance checklists, risk remediation tasks)
- **Tenant-Specific Schemas** (Each tenant has its own schema: `trustx_tenant_X`)

PostgreSQL Schema Design

Core Schema (trustx_main)

1. **tenants** (Stores metadata about tenants)
   - `tenant_id` (Primary Key, Integer)
   - `tenant_name` (String)
   - `created_at` (Datetime)
2. **users** (Global User Management)
   - `user_id` (Primary Key, Integer)
   - `tenant_id` (Foreign Key to tenants)
   - `username` (String)
   - `email` (String)
   - `password_hash` (String)

Tenant-Specific Schema (trustx_tenant_X)

1. **controls** (Stores SOC 2 controls)
   - `control_id` (Primary Key, String)
   - `category` (String)
   - `description` (Text)
   - `framework` (SOC 2, ISO 27001, NIST, etc.)
2. **policies** (Stores compliance policies linked to controls)
   - `policy_id` (Primary Key, String)
   - `control_id` (Foreign Key to controls)
   - `title` (String)
   - `document_url` (String)
3. **evidence** (Stores compliance proof for audits)

- `evidence_id` (Primary Key, String)
- `control_id` (Foreign Key to controls)
- `uploaded_by` (User ID, Foreign Key)
- `document_url` (String)
- `timestamp` (Datetime)

4. **audit_logs** (Stores compliance history)
   - `log_id` (Primary Key, String)
   - `event` (String)
   - `user_id` (Foreign Key)
   - `timestamp` (Datetime)
5. **tasks** (Tracks compliance checklist tasks)
   - `task_id` (Primary Key, String)
   - `control_id` (Foreign Key to controls)
   - `assigned_to` (User ID, Foreign Key)
   - `status` (Open, In Progress, Completed)
   - `due_date` (Datetime)

---

2. Neo4j - Relationship-Based Compliance Mapping

A **graph database (Neo4j)** will store complex relationships between SOC 2 controls, security risks, frameworks (NIST, ISO 27001), and audit evidence. Each **tenant's data will be isolated via a tenant ID filter**.

Tenant ID-Based Multi-Tenancy Approach in Neo4j

To separate tenant data in Neo4j, each node and relationship will have a `tenant_id` property. All queries must filter by `tenant_id` to ensure data isolation.

Example Node Structure

```
CREATE (:Tenant {tenant_id: "123", name: "Customer A"})
CREATE (:SOC2_Control {control_id: "CC6.1", category: "Logical Access", tenant_id: "123"})
CREATE (:Policy {policy_id: "P-001", title: "Access Control Policy", tenant_id: "123"})
```

Query for a Tenant's Policies

```
MATCH (p:Policy) WHERE p.tenant_id = "123" RETURN p;
```

Relationships in Neo4j:

- `(Tenant)-[:OWNS]->(SOC2_Control)`
- `(SOC2_Control)-[:HAS_POLICY]->(Policy)`
- `(SOC2_Control)-[:MITIGATES]->(Risk)`
- `(SOC2_Control)-[:REQUIRES_EVIDENCE]->(Audit_Evidence)`
- `(SOC2_Control)-[:MAPS_TO]->(Framework)`

Advantages

✅ **Scalable** – Works well for SaaS environments.
✅ **Fast query performance** with indexes on `tenant_id`.
✅ **No need for database-level separation** (simpler architecture).
✅ **Strict data isolation** as long as all queries enforce `tenant_id` filtering.

---

3. Integration & API Layer

To unify both **PostgreSQL** and **Neo4j**, we will:

1. Build a **GraphQL API** or **REST API** backend to expose compliance data.
2. Fetch **structured compliance records** from PostgreSQL (tenant-specific schemas).
3. Query **graph relationships** from Neo4j for dynamic mapping (filtered by tenant_id).
4. Use **background jobs** to sync relational data with graph data.
5. Implement **Dynamic Schema Switching** for tenants in PostgreSQL:

```
SET search_path TO trustx_tenant_123;
SELECT * FROM controls;
```

---

4. Why This Hybrid Model Works?

✅ **PostgreSQL** provides strong consistency for structured compliance data per tenant.
✅ **Neo4j** enables dynamic mapping for complex compliance relationships with tenant isolation.
✅ **Scalability** – Allows integration with **SIEMs, IAMs, and ticketing systems** (Okta, AWS, Jira, etc.).
✅ **Multi-Tenant Friendly** – Supports per-tenant compliance data separation.
✅ **Auditor-Friendly** – Provides both **structured reports** and **interactive visual compliance graphs**.

Next Steps

1. **Implement PostgreSQL multi-tenant schema and API endpoints**.
2. **Design Neo4j tenant-based relationships**.
3. **Build API layer for unified multi-tenant access**.
4. **Develop dashboards for tenant-based compliance monitoring**.

This hybrid model ensures **TrustX** can automate compliance, while supporting **scalability, automation, and dynamic risk mapping** in a multi-tenant SaaS environment.

Data Pipeline for Compliance and Profile360 Modules

Overview

This document outlines the data pipeline architecture for TrustX ATAP to support its core modules: **Compliance Monitoring** and **Profile360**. The pipeline enables structured ingestion, normalization, storage, and querying of data from multiple integrations (e.g., GitHub, Jira, Confluence, O365). It ensures that the same data can be used for both access and activity mapping (Profile360) and policy/control alignment (Compliance).

Pipeline Layers (Bronze → Silver → Gold)

---

1. Bronze Layer: Data Ingestion

Purpose: Fetch and store raw data from connected integrations.

- **Sources:**
  - **Integration Data:**
    - GitHub: Repositories, Issues, Pull Requests, Teams, Permissions
    - Jira: Projects, Issues, Workflows, Users, Roles
    - Confluence: Spaces, Pages, User Access
    - O365 / Microsoft Graph: Users, Roles, Application Access
    - etc ....
  - **Policy Documents:**
    - Created by compliance or SOC analysts of the tenant via the policy management module
- **Mechanism:**
  - API-based ingestion (scheduled or event-based)
  - Tenant-specific credentials and isolation
- **Storage:**
  - AWS S3 (per tenant)
  - Format: JSON / CSV / Parquet

---

2. Silver Layer: Data Normalization and Modeling

Purpose: Clean, transform, and correlate data across systems to build structured relationships.

- **Tasks:**
  - Identity resolution: unify user records across systems
  - Normalize entity types: users, roles, projects, permissions, etc.
  - Create cross-system links (e.g., a user in GitHub linked to their Jira identity)
  - Store structured tables and object models
- **Storage:**
  - PostgreSQL for normalized relational data
  - S3 for semi-structured processed documents

---

3. Gold Layer: Enriched Storage and Access

Purpose: Support specialized data access models for core TrustX modules.

- **Compliance Module:**
  - Control-to-policy mappings
  - Evidence and policy version tracking
  - Audit logs and compliance status
  - Stored in PostgreSQL
- **Profile360 Module:**
  - Graph representation of users, roles, access, activities
  - Linked entities (users → systems → controls → policies)
  - Stored in Neo4j (GraphDB)
- **Semantic Search & Embeddings:**
  - Chunked and embedded documents for contextual search
  - Metadata tagging for framework, user, role, policy, etc.
  - Stored in **PostgreSQL Vector DB extension** (pgvector)

Visualization and Interaction

- **Compliance Dashboards:**
  - Policy coverage
  - Control completion and evidence readiness
  - Audit timelines and review states
- **Profile360 Views:**
  - Visual graph of user access and system interactions
  - Role and access overlap detection
  - Control mapping to real user activity
- **Interface:**
  - Embedded React-based dashboards
  - Context-aware exploration linked to controls, users, and assets

---

Technology Stack

- **Ingestion:** Python APIs
- **Storage:**
  - S3 (raw + processed)
  - PostgreSQL (normalized and compliance data)
  - Neo4j (graph relationships for Profile360)
  - **PostgreSQL + pgvector** (semantic document search and context retrieval)
- **Dashboards:** React (internal UI)

TrustX ATAP Integration Security & Storage Standards

1. Introduction

This document outlines the **standards and best practices** for securely storing and managing third-party integration credentials (e.g., GitHub Org ID and Access Tokens) in TrustX ATAP. The approach ensures **security, scalability, and compliance** using AWS services, following industry best practices for data protection.

---

2. Overview of Integration Storage Approach

When a customer configures an integration (e.g., GitHub), they provide **sensitive credentials** such as:

- **GitHub Org ID** (Non-sensitive metadata)
- **GitHub Access Token** (Sensitive secret)

These credentials are stored securely using:

- **Amazon S3 (Per-Customer Bucket)** → Stores non-sensitive metadata (Org ID, integration status) in JSON format.
- **AWS Secrets Manager** → Stores and encrypts sensitive secrets (Access Tokens).
- **AWS KMS (Key Management Service)** → Encrypts all stored secrets and metadata, with **automatic key rotation every 6 months**.

---

3. Storage Architecture

Non-Sensitive Data Storage (S3 Bucket per Customer)

- Each tenant is assigned a dedicated **S3 bucket** to store integration metadata.
- Integration details (Org ID, status, and other metadata) are stored as **JSON files**.

Example S3 Bucket Structure:

```
s3://trustx-integrations/{tenant_id}/{application_name}.json
```

Example JSON File Content:

```
{
    "tenant_id": "tenant_123",
    "integration_name": "GitHub",
    "org_id": "OrgName123",
    "status": "active",
    "created_at": "2025-02-07T12:00:00Z"
}
```

Sensitive Data Storage (AWS Secrets Manager + KMS Encryption)

- **Each tenant's GitHub Access Token is stored securely in AWS Secrets Manager.**
- **Secret Name Format:** `GitHub_Token_{tenant_id}`
- **Encryption:** AWS KMS ensures secrets are encrypted at rest, with **automatic key rotation every 6 months**.
- **Access Control:** IAM policies restrict retrieval to only authorized backend services.

KMS Key Rotation Policy

AWS KMS is configured to **automatically rotate encryption keys every 6 months** to enhance security. This ensures that even if an encryption key is compromised, previously encrypted data remains secure.

```
{
    "KeyId": "alias/trustx-kms-key",
    "RotationEnabled": true,
    "RotationPeriod": "6M"
}
```

---

4. Implementation Guide

Step 1: Storing Integration Data Securely

1.1 Storing GitHub Org ID in S3 Bucket

```
const AWS = require("aws-sdk");
const s3 = new AWS.S3();

async function storeIntegrationMetadata(tenantId, integrationName, orgId) {
    const params = {
        Bucket: `trustx-integrations-${tenantId}`,
        Key: "metadata.json",
        Body: JSON.stringify({
            tenant_id: tenantId,
            integration_name: integrationName,
            org_id: orgId,
            status: "active",
            created_at: new Date().toISOString()
        }),
        ContentType: "application/json",
        ServerSideEncryption: "aws:kms",
        SSEKMSKeyId: "alias/trustx-kms-key"
    };
    await s3.putObject(params).promise();
}
```

1.2 Storing GitHub Access Token in AWS Secrets Manager

```
const secretsManager = new AWS.SecretsManager();

async function storeGitHubToken(tenantId, token) {
    const secretName = `GitHub_Token_${tenantId}`;

    await secretsManager.createSecret({
        Name: secretName,
        SecretString: JSON.stringify({ access_token: token }),
        KmsKeyId: "alias/trustx-kms-key"
    }).promise();

    return `Stored token in Secrets Manager: ${secretName}`;
}
```

Step 2: Retrieving Stored Credentials for API Calls

2.1 Retrieving GitHub Org ID from S3 Bucket

```
async function getIntegrationMetadata(tenantId) {
```

```
    const params = {
        Bucket: `trustx-integrations-${tenantId}`,
        Key: "metadata.json"
    };
    const data = await s3.getObject(params).promise();
    return JSON.parse(data.Body.toString());
}
```

2.2 Retrieving GitHub Access Token from Secrets Manager

```
async function getGitHubToken(tenantId) {
    const secretName = `GitHub_Token_${tenantId}`;
    const secret = await secretsManager.getSecretValue({ SecretId: secretName }).promise();
    return JSON.parse(secret.SecretString).access_token;
}
```

---

5. Security Best Practices

✅ **Use AWS Secrets Manager for sensitive credentials instead of storing them in S3 or the database.**
✅ **Restrict access using IAM roles to prevent unauthorized secret retrieval.**
✅ **Enable AWS CloudTrail to monitor access to Secrets Manager and S3.**
✅ **Rotate encryption keys automatically using AWS KMS every 6 months.**
✅ **Ensure S3 Buckets have encryption enabled using AWS KMS.**
✅ **Do not log secrets or expose them in application responses.**

---

6. Summary

How Integration Data is Managed Securely

Data Type	Storage Service	Security Approach
GitHub Org ID	S3 Bucket per Tenant	JSON file with IAM-restricted access, encrypted with KMS
GitHub Access Token	AWS Secrets Manager	Encrypted using AWS KMS, IAM-restricted access, automatic key rotation

By following this design, TrustX ATAP ensures secure storage, access control, and compliance with best practices for integration credential management.

---

7. Next Steps

- Implement **API endpoints** to handle integration setup and retrieval.
- Enforce **role-based access control (RBAC)** for integration management.
- Set up **audit logging** for access to integration credentials.

This document establishes the **TrustX ATAP standards for secure integration management** to protect sensitive credentials and maintain compliance.

TrustX GitHub Integration

This document outlines the GitHub integration for the TrustX platform. The integration is designed to connect TrustX tenants with one or more GitHub organizations, schedule and run data pulls, and store relevant configurations in an S3 bucket.

Repository and Folder Structure

Location:

TrustX Inc. Integration Repository

Inside the `Github` folder, you'll find the following key subfolders:

- `configuration`
  Contains code related to adding and managing GitHub organizations for each tenant. This includes:
  - The endpoint in the internal API for configuring a GitHub organization, including passing the organization name, a GitHub API key, and the tenant name.
  - Logic to store and manage these configuration details.
- `core`
  Contains the code responsible for pulling GitHub data given an organization and tenant name. This module is partially complete; it can perform data pulls, but it still needs:
  - Additional testing and bug fixes.
  - A functional scheduler that runs regularly and iterates through all configuration files (stored in S3) to trigger data pulls.
- `wip`
  Contains experimental or in-progress code snippets and scripts that may be useful for future development. Most of the code here is considered "scrap" or proof-of-concept.

Current State of Development

1. Adding Configured Organizations

- **Functionality**:
  - **Fully implemented**.
  - The internal API endpoint supports specifying:
    - **Organization name**
    - **GitHub API key**
    - **Tenant name**
  - These settings allow the platform to recognize which GitHub org belongs to which tenant and enable scheduled data pulls.
- **Location**:
  - `Github/configuration` folder in the Integrations repository.

2. Scheduling Data Pulls

- **Functionality**:
  - **Partially implemented**.
  - The underlying code to pull data from a GitHub org (based on the specified org name and tenant) is present but not comprehensively tested.
  - A scheduler mechanism is needed to:
    i. Run on a regular interval.
    ii. Check all configuration files in the designated S3 bucket (the bucket is called `configFiles` in the `Github` folder).

iii. Trigger data pulls for each configured organization/tenant pairing.

- **Location**:
  - `Github/core` folder in the Integrations repository.

3. Additional Code (WIP)

- **Description**:
  - Contains early prototypes or unused code.
  - Potentially helpful for reference, but not intended for production use.
- **Location**:
  - `Github/wip` folder in the Integrations repository.

What Needs to be Done

1. **Scheduler Completion**
   - Implement a reliable scheduler (e.g., using cron jobs, AWS Lambda scheduled events, or another scheduling mechanism) to regularly trigger data pulls.
   - Ensure the scheduler:
     - Reads configuration files from the S3 bucket (`configFiles`).
     - Iterates over each configuration to perform data pulls.
2. **Testing and Bug Fixes**
   - Perform comprehensive testing on both the configuration and data-pull functionality.
   - Address any errors or edge cases discovered during testing.
   - Validate that the system behaves correctly for multiple tenants and orgs.
3. **Documentation and Error Handling**
   - Add more detailed documentation to cover:
     - **API usage** for configuring GitHub organizations.
     - **Environment variables** (if any) required for the integration (e.g., AWS credentials, GitHub API keys).
     - **Error handling** and fallback procedures.
   - Provide sample configuration files and code snippets to clarify usage.

Basic Workflow (High-Level)

1. **Create / Update Configurations**
   - Use the internal API to register a new organization for a given tenant.

```
POST /api/integrations/github
{
  "orgName": "example-org",
  "githubApiKey": "ghp_xxx",
  "tenantName": "tenant-123"
}
```

   - The system stores these details in S3, within the `configFiles` bucket (or an equivalent configuration store).
2. **Scheduler Triggers Pull**
   - On a regular schedule (to be implemented), the scheduler reads all configuration files from S3.
   - For each config, it calls the data-pull code in the `Github/core` folder:
     - **Fetch relevant data** from GitHub using the stored API key and org name.
     - **Store or process** the data according to the platform's needs (e.g., analytics, dashboards, auditing).
3. **Error Handling & Logging**
   - Any errors (missing keys, GitHub authentication failures, etc.) should be logged and reported in an easily traceable manner.

- Retrying failed pulls or alerting an on-call team might be features to consider.

GitHub Integration Testing - ATAP API & UI

1. Introduction

This document provides **end-to-end testing** guidelines for **GitHub Integration in ATAP**, covering **API validation and UI functionality** to ensure seamless user experience.

Test Users for GitHub Integration

- **Tenant 1 Admins:**
  - `neel+tenant1-admin1@assurexinc.com`
  - `neel+tenant1-admin2@assurexinc.com`
- **Tenant 2 Admins:**
  - `neel+tenant2-admin1@assurexinc.com`
  - `neel+tenant2-admin2@assurexinc.com`

---

2. API Test Cases

Test Case 1: Configure GitHub Integration

- **API Endpoint:** `POST /integrations/configure`
- **Request Body:**

```
{
  "tenant_id": "tenant1",
  "integration": "GitHub",
  "application_type": "GitHub",
  "credentials": {
    "api_key": "valid_github_api_token"
  }
}
```

- **Expected Result:** API returns `201 Created`, credentials stored securely.

Test Case 2: Retrieve Integration Status

- **API Endpoint:** `GET /integrations/status`
- **Request Body:**

```
{
  "tenant_id": "tenant1",
  "integration": "GitHub",
  "application_type": "GitHub"
}
```

- **Expected Result:** API returns `200 OK`, correct status (`Active`, `Inactive`, `Error`).

Test Case 3: Trigger Data Sync

- **API Endpoint:** `POST /integrations/sync`
- **Request Body:**

```
{
  "tenant_id": "tenant2",
```

```
  "integration": "GitHub",
  "application_type": "GitHub"
}
```

- **Expected Result:** Sync is triggered, API returns `200 OK`.

Test Case 4: Fetch GitHub Users & Permissions

- **API Endpoint:** `GET /github/users`
- **Expected Result:** API returns `200 OK`, correct user roles (`admin`, `member`, `collaborator`).

Test Case 5: Update GitHub Integration Credentials

- **API Endpoint:** `PUT /integrations/update`
- **Request Body:**

```
{
  "tenant_id": "tenant1",
  "integration": "GitHub",
  "application_type": "GitHub",
  "new_credentials": {
    "api_key": "new_github_api_token"
  }
}
```

- **Expected Result:** API returns `200 OK`, credentials updated successfully.

Test Case 6: Delete GitHub Integration

- **API Endpoint:** `DELETE /integrations/delete`
- **Expected Result:** API returns `200 OK`, integration removed successfully.

---

3. UI Test Cases

Test Case 1: GitHub Integration Display

- **Steps:**
  a. Navigate to `Integrations` page.
  b. Locate **GitHub** integration card.
  c. Verify that the **status is displayed as Available** (green badge).
- **Expected Result:** GitHub integration appears correctly with `Connect` button enabled.

Test Case 2: Connect GitHub

- **Steps:**
  a. Click `Connect`.
  b. Enter **GitHub Organization Name**.
  c. Enter **GitHub Access Token**.
  d. Click `Connect GitHub`.
- **Expected Result:** Integration should be established, GitHub org should be listed in `View Linked Organizations`.

Test Case 3: View Linked Organizations

- **Steps:**
  a. Click `View Linked Organizations`.

b. A modal should display a table with **Organization ID, Name, and Date Added**.

- **Expected Result:** Organization details appear correctly.

Test Case 4: Delete a Linked Organization

- **Steps:**
  a. Open `View Linked Organizations`.
  b. Select one or multiple orgs.
  c. Click `Delete Selected`.
  d. Confirm deletion.
- **Expected Result:** API should remove the selected organization and update the UI.

Test Case 5: Error Handling - Invalid API Token

- **Steps:**
  a. Enter an **incorrect GitHub API token**.
  b. Click `Connect GitHub`.
- **Expected Result:** UI should display an error message: `Invalid credentials, please check your API token.`

Test Case 6: API Failure Handling

- **Steps:**
  a. Simulate API failure (disconnect from the network or use a faulty token).
  b. Click `View Linked Organizations`.
- **Expected Result:** UI should display `Unable to retrieve data. Please try again later.`

---

4. Test Execution & Validation

- **Test Environment:** ATAP API with AWS API Gateway, Lambda, and S3.
- **Validation Metrics:**
  - **API response times** (should be <500ms).
  - **Security validation** (no token exposure in logs).
  - **Data integrity** (tenant isolation, correct permissions display).

---

5. Conclusion

This document ensures both **backend API and frontend UI** work seamlessly for **GitHub Integration in ATAP**. After passing all tests, the feature will be **ready for production deployment.**

TrustX Adaptive API - Hybrid API Structure (REST + GraphQL)

1. Introduction

The **TrustX Adaptive API** is designed as a **hybrid API**, combining **REST and GraphQL** to optimize security, performance, and flexibility. This approach allows for **efficient data retrieval** while ensuring compliance and scalability for TrustX's security and compliance use cases.

Base API URLs

```
REST API:      <https://api.trustx.cloud/>
GraphQL API:   <https://api.trustx.cloud/graphql>
```

---

2. Why Hybrid API?

The **Hybrid API approach** is designed to leverage the strengths of both **REST and GraphQL**, ensuring **fast, cacheable API requests** for structured workflows while allowing **flexible, deep data queries** for knowledge graph exploration.

🔹 When to Use REST API

✅ **Static or structured data** that doesn't change frequently (e.g., compliance frameworks).
✅ **Highly cacheable responses** via Cloudflare and API Gateway.
✅ **Integration status, authentication, logging, and structured access control data.**

🔹 When to Use GraphQL API

✅ **Complex queries requiring relationships between entities** (e.g., user access relationships, privilege graph).
✅ **Reducing multiple API calls** when frontend needs aggregated data.
✅ **AI-driven analytics queries** that require dynamic field selection.

---

3. API Gateway Structure (REST + GraphQL Routing)

REST API Endpoints

Endpoint	Method	Description
`/auth/login`	`POST`	User authentication via Auth0
`/integrations/status`	`GET`	Fetch integration status from S3
`/compliance/frameworks`	`GET`	Retrieve compliance frameworks
`/audit/logs`	`GET`	Fetch audit logs
`/risk/assessments`	`GET`	Fetch risk assessment data

GraphQL API Queries

Query	Description

`getUser(id: ID!)`	Fetch user profile and relationships
`getUsersByRole(roleId: ID!)`	Fetch users assigned to a specific role
`getRiskyUsers(threshold: Float!)`	Fetch users flagged for potential risk
`getAnomalies`	Retrieve AI-detected anomalies
`getUserAccessLogs(userId: ID!)`	Retrieve access logs for a user

---

4. Hybrid API Best Practices

🔹 REST API Best Practices

✅ **Use API Gateway caching** for frequently accessed REST endpoints.
✅ **Leverage rate-limiting policies** to prevent abuse.
✅ **Secure with OAuth 2.0 (Auth0) and JWT-based authentication.**

🔹 GraphQL API Best Practices

✅ **Use schema-based access control** (ensure users can only query allowed data).
✅ **Optimize GraphQL queries using pagination and depth limiting.**
✅ **Cache frequently accessed GraphQL queries using Cloudflare.**

🔹 Hybrid API Integration Considerations

✅ **Frontend should mix REST and GraphQL queries efficiently** (e.g., use REST for authentication & GraphQL for data retrieval).
✅ **AI-powered insights should be GraphQL-first**, as these queries involve relationships and multi-node data exploration.
✅ **Use GraphQL fragments in frontend queries** to optimize response payloads.

---

5. Security & Performance Considerations

- 🔹 **Authentication:** OAuth 2.0 (Auth0) for both REST & GraphQL endpoints.
- 🔹 **Rate-Limiting:** Different limits for REST and GraphQL queries.
- 🔹 **Logging & Monitoring:** AWS CloudWatch logs API activity and anomalies.
- 🔹 **Data Isolation:** Multi-tenant security enforced via schema-based access control.
- 🔹 **GraphQL Query Depth Limit:** Prevents deep-nested recursive queries to enhance performance.

---

6. Summary & Next Steps

- **Implement API routing** to handle REST + GraphQL in API Gateway.
- **Enable caching policies** for REST endpoints via Cloudflare.
- **Deploy Neo4j as the backend graph database** for the GraphQL API.
- **Monitor API usage patterns** to optimize response time and data queries.

🚀 **This hybrid API structure ensures that TrustX Adaptive API is scalable, secure, and optimized for compliance automation.**

Archived - TrustX Adaptive API Gateway

1. Introduction

The **TrustX Adaptive API Gateway** is the central hub for managing all security, compliance, and AI-driven automation services within TrustX ATAP. This API Gateway enables secure, scalable, and efficient communication between microservices while ensuring **authentication, rate-limiting, logging, and policy enforcement**.

---

2. API Gateway Overview

🔹 Key Features

- **Unified API Layer** → A single entry point for all TrustX services.
- **Multi-Tenant Support** → Ensures strict isolation and policy enforcement.
- **AI-Driven Compliance Automation** → Integrates AI for compliance insights.
- **Security & Authentication** → Uses **Auth0** for user authentication.
- **Rate Limiting & Logging** → Enforces API limits and captures logs.
- **Microservices Routing** → Routes requests to AWS Lambda, S3, Secrets Manager, and compliance services.

🔹 API Gateway Base URL

```
<https://api.trustx.cloud/>
```

---

3. API Gateway Route Structure

Authentication & User Management

Endpoint	Method	Description
`/auth/login`	`POST`	User login via Auth0
`/auth/logout`	`POST`	Logout user session
`/users/profile`	`GET`	Fetch user profile details

Integration Management

Endpoint	Method	Description
`/integrations/status`	`GET`	Fetch integration status from ~~S3~~ DynamoDB
`/integrations/connect`	`POST`	Connect a new third-party integration
`/integrations/disconnect`	`DELETE`	Remove an existing integration

Compliance & Governance

Endpoint	Method	Description
`/compliance/frameworks`	`GET`	Fetch supported compliance frameworks (SOC2, ISO 27001, etc.)
`/compliance/controls`	`GET`	Retrieve security controls mapped to a framework
`/audit/logs`	`GET`	Retrieve compliance audit logs

Risk & Security Assessments

Endpoint	Method	Description
`/risk/assessments`	`GET`	Fetch risk assessment reports
`/risk/findings`	`GET`	Retrieve security vulnerabilities & compliance gaps

AI-Driven Compliance Assistant

Endpoint	Method	Description
`/ai/chatbot`	`POST`	Query AI-powered compliance assistant
`/ai/recommendations`	`GET`	Get AI-driven security recommendations

---

4. API Security & Authentication

🔹 Authentication Mechanism

- **Auth0 / OAuth 2.0** → Enforces secure authentication.
- **JWT Tokens** → Used for authorization with role-based access control.
- **API Keys** → For internal service-to-service communication.

🔹 Security Policies

✅ **Role-Based Access Control (RBAC)** → Restricts sensitive operations to admins.
✅ **Rate Limiting** → Prevents abuse and excessive API calls.
✅ **Logging & Monitoring** → Uses **AWS CloudWatch** for real-time tracking.
✅ **Data Encryption** → API traffic encrypted using **TLS 1.2+**.
✅ **Multi-Tenant Isolation** → Ensures strict data segregation per customer.

---

5. API Gateway Infrastructure

AWS Services Used

- **Amazon API Gateway** → Manages routing, security, and rate limiting.
- **AWS Lambda** → Handles API logic for integrations, compliance, and risk.
- **Amazon S3** → Stores integration metadata in JSON format.
- **AWS Secrets Manager** → Stores encrypted API tokens and secrets.
- **Amazon CloudWatch** → Monitors API requests and logs security events.

---

6. Next Steps

🚀 **Implement API Endpoints** → Develop backend services to support each route.
🚀 **Configure API Gateway Policies** → Enforce security, logging, and rate limits.
🚀 **Deploy AI Chatbot API** → Integrate AI-powered compliance assistant.
🚀 **Monitor & Optimize Performance** → Use AWS CloudWatch for tracking usage.

This document establishes the **TrustX Adaptive API Gateway** as the foundation for **secure, scalable, and AI-driven compliance automation**.

AI-Assisted Policy Management

1. Overview

TrustX ATAP provides a **dual-interaction model** for managing and customizing compliance policies. Users can interact with the platform in two ways:

1️⃣ AI-Powered Conversational Interface (Primary Method)

- Users can **ask the AI assistant** policy-related questions.
- The chatbot dynamically **opens the policy in a bottom-page canvas** for real-time editing.
- AI provides **contextual suggestions** for compliance updates based on best practices.

2️⃣ Traditional Navigation (Secondary Method)

- Users can manually navigate to **Policies** through the left-side menu.
- They can **search, browse, and edit** policies using a traditional interface.
- Version control and compliance mappings are available for structured editing.

This **hybrid approach** ensures ease of use for both **AI-driven** and **traditional** compliance workflows.

---

2. Approach for Populating Compliance Policies with Templated Documents

To **automate and standardize** compliance policies within **TrustX**, we need a **scalable approach** for storing, managing, and dynamically generating policy documents for different compliance frameworks (SOC 2, ISO 27001, NIST, etc.).

2.1 Storing Policy Templates & Tenant Drafts

Storage Option	Pros	Cons
Amazon S3 (Recommended)	✅ Scalable ✅ Low cost ✅ Supports structured JSON/Markdown policies	❌ Requires metadata indexing for searchability
PostgreSQL	✅ Structured queries ✅ Good for metadata indexing ✅ Versioning support	❌ Not optimized for document storage
Neo4j (for policy relationships)	✅ Can map relationships between policies and controls	❌ Not optimized for document storage

🔹 **Recommended Approach:**

- **Master templates** are stored in S3 in a **global template repository**.
- When a **new tenant** is onboarded, the system **automatically imports policy templates** and creates **tenant-specific copies** in their S3 bucket.
- Each tenant can then **edit, track versions, and approve policies independently**.
- **PostgreSQL indexes metadata** for **searchability, version tracking, and policy ownership separation**.
- **Neo4j maps policies to compliance controls** for audit tracking.

---

3. PostgreSQL Schema for Policy Management

We will utilize **PostgreSQL** for structured policy storage, metadata indexing, and version tracking.

3.1 Table: `policies` (Stores Policy Metadata)

```
CREATE TABLE policies (
    policy_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    name VARCHAR(255) NOT NULL,
    framework TEXT[],  -- Example: {'SOC 2', 'ISO 27001'}
    category VARCHAR(255),
    description TEXT,
    latest_version VARCHAR(10) NOT NULL DEFAULT '1.0',
    requires_employee_acknowledgment BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

3.2 Table: `policy_versions` (Tracks Policy Changes)

```
CREATE TABLE policy_versions (
    version_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    policy_id UUID REFERENCES policies(policy_id) ON DELETE CASCADE,
    tenant_id UUID NOT NULL,
    version VARCHAR(10) NOT NULL,
    storage_path TEXT NOT NULL,  -- Example: "s3://trustx-tenants/{tenant_id}/policies/access_control_v1.md"
    status VARCHAR(50) DEFAULT 'Draft',  -- Draft, Pending Approval, Approved
    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(policy_id, version)  -- Ensure unique versioning per policy
);
```

3.3 Table: `policy_approvals` (Tracks Approval Workflow)

```
CREATE TABLE policy_approvals (
    approval_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    policy_id UUID REFERENCES policies(policy_id) ON DELETE CASCADE,
    tenant_id UUID NOT NULL,
    approver_id UUID NOT NULL,  -- References user table
    status VARCHAR(50) DEFAULT 'Pending',  -- Pending, Approved, Rejected
    comments TEXT,
    approved_at TIMESTAMP DEFAULT NULL
);
```

3.4 Table: `policy_acknowledgments` (Employee Sign-Off)

```
CREATE TABLE policy_acknowledgments (
    acknowledgment_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    policy_id UUID REFERENCES policies(policy_id) ON DELETE CASCADE,
    tenant_id UUID NOT NULL,
    employee_id UUID NOT NULL,  -- References user table
    acknowledged_at TIMESTAMP DEFAULT NOW()
);
```

---

4. Updated User Flow: AI-Driven & Traditional Policy Management

🔹 How It Works (Master Templates & Tenant Copies)

1️⃣ **TrustX stores master policy templates** in a global repository.
2️⃣ **When a new tenant is onboarded**, TrustX **creates tenant-specific copies**.
3️⃣ **SOC analysts can edit tenant policies** → Changes are saved in PostgreSQL and S3.
4️⃣ **Approval workflow kicks in** → CISOs or other officers must **approve policy changes**.
5️⃣ **If a policy requires company-wide acknowledgment**, employees must **sign off on it**.
6️⃣ **Once approved, policy is finalized**, locked for editing, and stored in the compliance repository.

🔹 User Flow for AI-Driven Policy Management

1️⃣ **User asks the AI chatbot about a policy.**
2️⃣ **TrustX AI retrieves the policy and opens it in a canvas for editing.**
3️⃣ **User can edit, accept AI suggestions, or apply compliance mappings.**
4️⃣ **Approval process starts (if required).**
5️⃣ **Finalized policy version is saved in PostgreSQL and S3.**

🔹 User Flow for Traditional Navigation

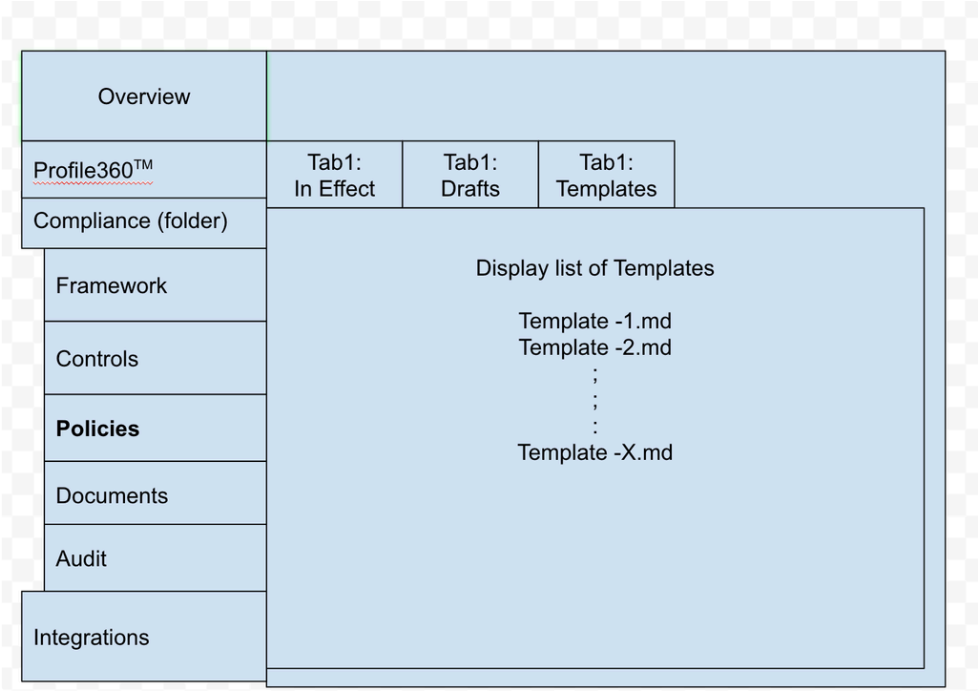
1️⃣ **User navigates to "Policies" via the sidebar.**
2️⃣ **Filters by framework: "SOC 2" → selects "Access Control Policy".**
3️⃣ **Views and edits policy manually, then saves it.**
4️⃣ **Approval process is triggered, and the policy is reviewed before finalization.**

---

5. Implementation Plan

1️⃣ **Store policy metadata & versioning in PostgreSQL.**
2️⃣ **Store policy files in S3 (Markdown/PDF) per tenant.**
3️⃣ **Implement approval workflow (SOC analyst → CISO approval).**
4️⃣ **Track employee acknowledgment for mandatory policies.**
5️⃣ **Expose REST APIs for policy search, retrieval, approval, and acknowledgment.**
6️⃣ **Build AI-driven and traditional UI for policy management.**

UI Mockup


Overview
Profile360™
Compliance (folder)
Framework
Controls
Policies
Documents
Audit
Integrations
Tab1:
In Effect
Tab1:
Drafts
Tab1:
Templates
Display list of Templates
Template -1.md
Template -2.md
;
;
:
Template -X.md

Draft - Policy Management API - Swagger

```yaml
openapi: 3.0.1
info:
  title: TrustX Policy Management API
  description: API for managing compliance policies, approvals, and employee acknowledgments
  version: 1.1.0
servers:
  - url: https://api.trustx.cloud
    description: Production Server
security:
  - BearerAuth: []
components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  responses:
    UnauthorizedError:
      description: Access token is missing or invalid
    BadRequest:
      description: Bad request due to missing or incorrect parameters
    ForbiddenError:
      description: The user does not have permission to access this resource
    InternalServerError:
      description: Unexpected server error
paths:
  /policies:
    get:
      summary: Retrieve all policies with pagination
      parameters:
        - name: framework
          in: query
          required: false
          schema:
            type: string
        - name: page
          in: query
          required: false
          schema:
            type: integer
            default: 1
        - name: limit
          in: query
          required: false
          schema:
            type: integer
            default: 10
      responses:
        "200":
          description: List of policies
        "401":
          $ref: "#/components/responses/UnauthorizedError"
  /policies/{policy_id}:
```

```
    get:
      summary: Get policy details
      parameters:
        - name: policy_id
          in: path
          required: true
          schema:
            type: string
      responses:
        "200":
          description: Policy details
        "401":
          $ref: "#/components/responses/UnauthorizedError"
    put:
      summary: Update a policy
      parameters:
        - name: policy_id
          in: path
          required: true
          schema:
            type: string
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                name:
                  type: string
                description:
                  type: string
      responses:
        "200":
          description: Policy updated successfully
        "403":
          $ref: "#/components/responses/ForbiddenError"
  /policies/{policy_id}/approve:
    post:
      summary: Approve a policy
      parameters:
        - name: policy_id
          in: path
          required: true
          schema:
            type: string
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                approver_id:
                  type: string
                comments:
                  type: string
      responses:
```

```
        "200":
          description: Policy approved
        "401":
          $ref: "#/components/responses/UnauthorizedError"
  /policies/{policy_id}/acknowledge:
    post:
      summary: Employee acknowledges policy
      parameters:
        - name: policy_id
          in: path
          required: true
          schema:
            type: string
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                employee_id:
                  type: string
      responses:
        "200":
          description: Policy acknowledged by employee
        "401":
          $ref: "#/components/responses/UnauthorizedError"
  /policies/{policy_id}/delete:
    delete:
      summary: Delete a policy
      parameters:
        - name: policy_id
          in: path
          required: true
          schema:
            type: string
      responses:
        "200":
          description: Policy deleted successfully
        "403":
          $ref: "#/components/responses/ForbiddenError"
```

Design & Functional Specification for Policy Editing Feature

1. UI Structure & Policy Lifecycle

The **Policy Management UI** will have **three main tabs**:

1. **Templates** – Master templates available for use.
2. **Drafts** – Policies currently being edited.
3. **In-Effect** – Approved and locked policies.

---

2. Policy Editing & Copying Behavior

- **Editing from Templates**
  - When a **template** is edited, a **copy** of it is created in the **Drafts** tab.
  - The original template remains unchanged.
  - The copied draft can be modified and then submitted for approval.
- **Editing from In-Effect Policies**
  - If a policy in the **In-Effect** tab is edited, an **ineffective copy** is created in the **Drafts** tab.
  - The original **In-Effect** policy remains read-only.
  - Changes can only take effect after going through the approval workflow.

---

3. Policy Storage & Database

- **Files will be stored in an S3 bucket per tenant.**
- **Metadata and versioning will be stored in PostgreSQL.**
- Policies will be **retrievable in JSON/Markdown format**.
- Upon approval, policies will be **converted to PDF** for download in the **In-Effect** tab.

---

4. Policy Lifecycle Stages

1. **Template Available**
   - Policies start as templates in the **Templates** tab.
   - Users can choose a template and create a draft.
2. **Draft (Editable)**
   - Users modify the policy in this stage.
   - The system tracks versions and saves changes.
   - Policies in Draft can be **submitted for approval**.
3. **Pending Approval**
   - SOC analysts or compliance officers review the draft.
   - They can approve, reject, or request changes.
4. **In-Effect (Read-Only)**
   - Once approved, the policy moves to **In-Effect**.
   - Users can **download the policy as a PDF**.
   - Any future edits require creating a **new Draft**.

---

5. API Endpoints (Compliance Policy Management)

Base Path: `/compliance/policies/`

1. **Retrieve Policies**
   - `GET /compliance/policies?status={draft|in-effect|template}`
   - Fetch policies based on status.
2. **Import Template & Create Draft**
   - `POST /compliance/policies/import`
   - Creates a draft copy from a selected template.
3. **Edit Draft**
   - `PUT /compliance/policies/{policy_id}/edit`
   - Allows editing of policies in **Draft** state.
4. **Submit for Approval**
   - `POST /compliance/policies/{policy_id}/submit`
   - Sends draft for review and approval.
5. **Approve Policy**
   - `POST /compliance/policies/{policy_id}/approve`
   - Marks policy as **In-Effect**.
6. **Download In-Effect Policy as PDF**
   - `GET /compliance/policies/{policy_id}/download`
   - Exports the policy to a **PDF format**.

---

6. Security & Access Control

- **SOC Analysts** can **edit and submit** policies.
- **Compliance Managers** can **approve or reject** policies.
- **Auditors & General Users** can **only view** In-Effect policies.

📌 Comprehensive List of Policy Templates for TrustX ATAP

Detailed list of **security, compliance, and privacy** policy templates needed for **SOC 2, ISO 27001, NIST 800-53, CIS, HIPAA, GDPR, CCPA, and other frameworks**.

1. General Information Security Policies

✅ **Acceptable Use Policy** – Defines how employees can use company systems.

✅ **Information Security Policy** – Overarching policy covering all security measures.

✅ **Access Control Policy** – Guidelines for system and data access management.

✅ **Data Classification & Handling Policy** – Defines sensitivity levels for data.

✅ **Incident Response Policy** – Steps to take in case of a security incident or breach.

✅ **Security Awareness & Training Policy** – Employee security training guidelines.

✅ **Third-Party & Vendor Risk Management Policy** – Security requirements for external vendors.

2. SOC 2-Specific Policies (Trust Services Criteria)

✅ **Change Management Policy** – Guidelines for implementing system changes securely.

✅ **System Access Policy** – Rules for granting, modifying, and revoking user access.

✅ **Audit Logging & Monitoring Policy** – Tracking system and user activity logs.

✅ **Vendor Risk Management Policy** – Ensures third-party vendors comply with security standards.

✅ **Data Encryption & Key Management Policy** – Requirements for encryption and secure key storage.

3. ISO 27001 Mandatory Policies (Annex A Controls)

✅ **Risk Management Policy** – Identifies, assesses, and mitigates risks.

✅ **Business Continuity & Disaster Recovery (BCDR) Policy** – Ensures business operations during a disaster.

✅ **Cryptographic Controls Policy** – Guidelines for encryption and cryptographic measures.

✅ **Secure Software Development Lifecycle (SDLC) Policy** – Secure development and release practices.

✅ **Network Security Policy** – Covers firewall, VPN, and network segmentation.

✅ **Physical Security & Environmental Protection Policy** – Secure workplace and asset protection.

4. NIST 800-53 & CIS Security Policies

✅ **User Access Management Policy** – Procedures for onboarding and offboarding users.

✅ **Incident Detection & Response Policy** – Steps for identifying and mitigating cyber incidents.

✅ **Cloud Security Policy** – Protecting cloud resources and data.

✅ **Endpoint Protection & Device Management Policy** – Rules for managing laptops, mobile, and IoT.

✅ **Password & Authentication Policy** – MFA and password best practices.

✅ **Vulnerability Management & Patch Management Policy** – Security updates and vulnerability scans.

✅ **Secure Remote Work Policy** – Security controls for remote employees.

5. HIPAA, GDPR, CCPA, & US Data Privacy Policies

✅ **Privacy Policy (GDPR, CCPA, HIPAA Compliance)** – Public-facing policy on user data collection and rights.

✅ **Data Retention & Disposal Policy** – Defines how long data is retained and how it's destroyed.

✅ **Breach Notification Policy** – Process for informing users and regulators about data breaches.

✅ **Employee Data Protection Policy** – Guidelines for handling employee PII.

✅ **Consent & Data Subject Rights Policy** – Ensures compliance with GDPR's right to be forgotten, data portability, etc.

✅ **Data Sharing & Third-Party Disclosure Policy** – Defines when and how user data can be shared.

✅ **Cross-Border Data Transfer Policy** – Rules for transferring data outside the US and EU.

6. US Data Privacy Framework (Covers Federal & State Laws)

✅ **California Consumer Privacy Act (CCPA) Policy** – Ensures CCPA compliance, user rights, and opt-out mechanisms.

✅ **Virginia Consumer Data Protection Act (VCDPA) Policy** – Defines how consumer data is processed in Virginia.

✅ **Colorado Privacy Act (CPA) Policy** – Covers consumer data rights in Colorado.

✅ **Health Insurance Portability & Accountability Act (HIPAA) Policy** – Compliance for handling healthcare data.

✅ **Children's Online Privacy Protection Act (COPPA) Policy** – Rules for protecting children's online data.

✅ **Gramm-Leach-Bliley Act (GLBA) Policy** – Compliance for financial institutions on data privacy.

7. Compliance & Legal Policies

✅ **Regulatory Compliance Policy** – Ensures compliance with laws (SOX, PCI-DSS, FISMA).

✅ **Whistleblower & Ethics Policy** – Encourages employees to report unethical behavior.

✅ **Anti-Bribery & Corruption Policy** – Ensures compliance with FCPA and UK Bribery Act.

✅ **Intellectual Property & Copyright Policy** – Covers IP rights and software licensing.

8. IT Operations & Security Policies

✅ **Bring Your Own Device (BYOD) Policy** – Security rules for personal devices.

✅ **Backup & Data Recovery Policy** – Defines frequency and storage of backups.

✅ **API Security Policy** – Guidelines for protecting APIs from attacks.

✅ **Logging & Monitoring Policy** – Requirements for log collection and security monitoring.

✅ **IT Asset Management Policy** – Tracking and managing IT assets securely.

✅ **Email Security & Phishing Prevention Policy** – Protecting employees from email threats.

Acceptable Use Policy (Template)

Disclaimer

This document is a **policy template** provided for general guidance. Organizations using this template should **review, customize, and modify it** according to their specific operational, legal, and compliance requirements. AssureX Technologies, Inc. is not responsible for any legal, regulatory, or operational consequences arising from the use of this template. Organizations should consult with their legal and compliance teams before formal adoption.

---

1. Purpose

The purpose of this Acceptable Use Policy (AUP) is to establish guidelines for the responsible and secure use of [Company Name]'s systems, data, and services. This policy applies to all employees, contractors, vendors, and third parties who have access to [Company Name]'s resources.

2. Scope

This policy covers all [Company Name]'s assets, including but not limited to:

- Networks, applications, and cloud environments
- Endpoints such as workstations, laptops, and mobile devices
- Data storage systems and databases
- Internal and external communication platforms
- Email systems, collaboration tools, and software services
- Any other IT infrastructure owned, managed, or provided by [Company Name]

3. Acceptable Use Guidelines

Users must:

- Use [Company Name]'s resources **only for authorized business purposes**.
- Follow **strong authentication and password management practices**.
- **Protect sensitive data** and comply with [Company Name]'s **data classification policy**.
- **Report security incidents** or suspicious activity immediately.
- **Comply with regulatory and contractual obligations**, including SOC 2, ISO 27001, and NIST standards.
- Ensure all company-provided devices are regularly updated and secured.
- Use only approved software and applications for work-related activities.
- Follow proper data retention and disposal policies.

4. Prohibited Activities

Users must NOT:

- Share access credentials or allow unauthorized access.
- Bypass security controls such as firewalls or endpoint protection.
- Download, store, or distribute **malware, illegal content, or copyrighted materials**.
- Use [Company Name]'s resources for **personal gain, fraud, or unethical activities**.
- Attempt to exploit, hack, or test vulnerabilities without explicit approval.

- Disclose **confidential company or customer information** to unauthorized parties.
- Engage in any activity that disrupts network operations or degrades service availability.
- Use company systems for personal political activities, harassment, or discrimination.
- Connect unauthorized devices to corporate networks or use unapproved cloud storage solutions.

5. Security Monitoring & Enforcement

[Company Name] reserves the right to:

- **Monitor network and system activity** for security and compliance purposes.
- **Audit user access and usage logs** to ensure compliance.
- **Enforce disciplinary action** for policy violations, up to and including termination.
- Report illegal activities to law enforcement as required by law.
- Disable or revoke access to users violating company policies.
- Implement security controls such as endpoint detection, data loss prevention, and behavior analytics to protect company assets.

6. Remote Work & Bring Your Own Device (BYOD) Policy

- Employees using personal devices for work **must comply with security requirements**, including:
  - Enabling encryption and screen locks.
  - Installing security updates and company-approved security software.
  - Using a VPN when accessing corporate networks remotely.
- Company data **must not** be stored on personal devices unless explicitly authorized.

7. Policy Compliance

[Company Name] will measure and verify compliance with this policy through various methods, including but not limited to ongoing monitoring, and both internal and external audits.

8. Exceptions

Requests for an exception to this policy must be submitted to [Policy Owner/Compliance Officer] for approval.

9. Violations & Enforcement

Any known violations of this policy should be reported to [Policy Owner/Compliance Officer]. Violations of this policy can result in immediate withdrawal or suspension of system and network privileges and/or disciplinary action in accordance with company procedures up to and including termination of employment.

---

10. Version Control

Version	Date	Author	Policy Owner	Approved By	Notes
1.0	[Insert Date]	[Author Name]	[Policy Owner]	[Approver]	Initial Draft

Information Security Policy (Template)

Disclaimer

This document is a **policy template** provided for general guidance. Organizations using this template should **review, customize, and modify it** according to their specific operational, legal, and compliance requirements. AssureX Technologies, Inc. is not responsible for any legal, regulatory, or operational consequences arising from the use of this template. Organizations should consult with their legal and compliance teams before formal adoption.

---

1. Purpose

The purpose of this Information Security Policy is to establish a **framework for protecting [Company Name]'s data, systems, and networks**. This policy ensures that security risks are properly identified, mitigated, and managed in alignment with industry best practices and regulatory requirements.

2. Scope

This policy applies to all employees, contractors, vendors, and third parties who have access to [Company Name]'s information assets. It covers:

- All company-owned systems, networks, applications, and cloud environments.
- Endpoints such as laptops, workstations, mobile devices, and IoT assets.
- Sensitive, confidential, and personally identifiable information (PII).
- Internal and external communication platforms and collaboration tools.
- Third-party integrations, vendor systems, and managed services.

3. Information Security Principles

To maintain a secure environment, [Company Name] follows these core security principles:

- **Confidentiality:** Ensure that only authorized users have access to sensitive data.
- **Integrity:** Protect the accuracy and completeness of information.
- **Availability:** Ensure systems and data are accessible to authorized users when needed.
- **Accountability:** Enforce security responsibilities through defined roles and controls.

4. Security Controls & Guidelines

4.1 Access Control

- User accounts and access permissions must be granted on a **least privilege basis**.
- Multi-Factor Authentication (MFA) is required for all critical systems.
- Access to sensitive data must be reviewed and audited **quarterly**.

4.2 Data Protection & Encryption

- All sensitive data must be **encrypted in transit and at rest**.
- Personal Identifiable Information (PII) and company secrets require **additional protection**.
- Data transfers must comply with **GDPR, CCPA, and applicable privacy laws**.

4.3 Network Security

- Firewalls and intrusion detection/prevention systems (IDS/IPS) must be in place.
- Remote access must be secured via **VPN and endpoint security tools**.
- Network segmentation should be enforced to **restrict lateral movement of threats**.

4.4 Endpoint Security

- All devices must have **endpoint protection, antivirus, and malware prevention**.
- Patch management must ensure software and OS updates are applied **within 30 days**.
- Unauthorized software installation is strictly prohibited.

4.5 Incident Response & Reporting

- All security incidents must be reported to **[Security Team or SOC]** immediately.
- A formal **Incident Response Plan** must be followed for handling breaches.
- Post-incident analysis must be conducted for lessons learned and improvements.

4.6 Third-Party Security & Vendor Management

- All vendors must comply with [Company Name]'s **Third-Party Risk Management Policy**.
- Security assessments must be conducted for critical service providers.
- Data shared with third parties must be governed by **Data Processing Agreements (DPA)**.

4.7 Secure Software Development

- Development teams must follow **Secure SDLC practices** (e.g., OWASP Top 10, DevSecOps).
- Code must be regularly reviewed and scanned for vulnerabilities.
- Secrets, API keys, and credentials **must not be stored in source code repositories**.

4.8 Security Awareness & Training

- All employees must complete **mandatory security awareness training** annually.
- Phishing simulation exercises should be conducted **at least quarterly**.
- Role-specific security training should be provided for developers, IT, and SOC teams.

5. Policy Compliance

[Company Name] will measure and verify compliance with this policy through various methods, including but not limited to **ongoing monitoring, internal audits, and external security assessments**.

6. Exceptions

Requests for an exception to this policy must be submitted to **[Policy Owner/Compliance Officer]** for approval.

7. Violations & Enforcement

Any known violations of this policy should be reported to **[Policy Owner/Compliance Officer]**. Violations of this policy may result in:

- Immediate withdrawal or suspension of system and network privileges.
- Disciplinary action in accordance with company procedures.
- Potential legal consequences if applicable.

---

8. Version Control

Version	Date	Author	Policy Owner	Approved By	Notes
1.0	[Insert Date]	[Author Name]	[Policy Owner]	[Approver]	Initial Draft

TrustX Control ID Naming Convention and Standard

Overview

To ensure consistency, traceability, and ease of framework alignment, TrustX implements a standardized control ID format based on domain classification and brand prefixing. This approach is designed to be both human-readable and system-friendly, enabling granular control tracking across multiple compliance frameworks (e.g., SOC 2, ISO/IEC 27001:2022, NIST 800-53).

Control ID Format

`TX-<DOMAIN_PREFIX><NN>.NN`

Segment	Description	Example
`TX`	Brand prefix for TrustX	`TX`
`DOMAIN_PREFIX`	Short code representing the control domain	`IAM`, `PRV`
`NN`	Padded 2-digit control sequence per domain	`01`, `10`

Example IDs

- `TX-IAM01.00` – Identity & Access Control –Objective
- `TX-IAM01.01` – Identity & Access Control –Focus Area
- `TX-GOV01.00` – Governance & Management – Control Objective
- `TX-GOV00.01` – Governance & Management –Focus Area

Control Structure

TrustX uses a hierarchical structure where each Control Objective and its associated Points of Focus (PoF) are organized in a parent-child relationship through a consistent ID pattern**.**

New Control ID Format:

- Control Objectives: TX-{DOMAIN}{NN}.00 (e.g., TX-IAM01.00)
- Points of Focus: TX-{DOMAIN}-{NN}.{MM} (e.g., TX-IAM01.01, TX-IAM01.02)

Where:

- TX: Brand prefix for TrustX
- DOMAIN: Short code representing the control domain (IAM, PRV, etc.)
- NN: Padded 2-digit sequence number for Control Objectives within each domain
- MM: Padded 2-digit sequence number for Points of Focus under a Control Objective

Benefits of Hierarchical Controls:

- ✅ Clear relationship between Control Objectives and their Points of Focus
- ✅ Domain-specific organization for easier navigation and categorization
- ✅ Unique identifiers for precise reference and traceability
- ✅ Intuitive numbering system that scales with new controls

- ✅ Easier mapping to hierarchical compliance frameworks

This structure provides a balance between organization and flexibility, allowing for both hierarchical representation and individual control addressability.

Control ID Example:

```
TX-IAM01.01 - Enforce role-based access
TX-IAM02.03 - Require MFA for all admin users
TX-IAM03.01 - Conduct quarterly access reviews

```

Domains and Prefixes

Domain Name	Prefix
Governance & Management	GOV
Risk Management	RIS
Asset Management	AST
Access Control	IAM
Identity & Authentication	IDN
Operations & Monitoring	MON
Incident Response	ICR
Business Continuity & Availability	BCA
Change Management	CHG
Vendor / Third-party Management	VND
Privacy & Data Protection	PRV
Cryptography	CRY
System Acquisition & Development	SDL
Physical Security	PHY
Compliance & Audit	AUD
Awareness & Training	TRN
Secure Configuration Management	CFG
Internal Communications	ICM
External Communication	ECM

Design Principles

✅ Human-readable

Each control is easy to identify and reference in UI, evidence logs, and audits.

✅ Scalable

Each domain supports up to 99 uniquely numbered controls using 2-digit identifiers.

✅ Framework-Mappable

Each TrustX control can map directly to one or more controls across SOC 2, ISO 27001:2022, NIST 800-53, etc.

✅ Audit-Friendly

Flat controls simplify audit checklists, evidence mapping, and progress tracking.

✅ Implementation-Agnostic

Supports any level of control granularity without structural nesting.

Future-Proofing

- New domains can be added easily using a new 3-letter prefix.
- Control ID format can scale to `TX-<DOMAIN><NN>.<NNN>` if ever needed.

Next Steps

- Continue populating TrustX Control Library using the flat ID format.
- Map each control to relevant frameworks and tests.
- Expose flat control list in UI with sorting, filtering, and tag-based grouping.

Version: 1.4
Last Updated: 2025-04-11

About TrustX Manufacturing, Inc.

TrustX Manufacturing, Inc. is an innovative manufacturing startup committed to redefining industry standards through cutting-edge technology, precision engineering, and sustainable practices. Specializing in advanced materials and smart manufacturing solutions, TrustX is dedicated to providing reliable, efficient, and environmentally-conscious products that drive productivity and enhance operational excellence.

Built by a passionate team of engineers, innovators, and industry experts, TrustX leverages automation, artificial intelligence, and continuous improvement methodologies to deliver next-generation manufacturing capabilities. At TrustX, quality, integrity, and trust are embedded into every step of the process—from design to delivery—ensuring consistent excellence and customer satisfaction.

Our mission is to empower industries through smarter manufacturing, fostering long-term partnerships built on transparency, innovation, and results.

Name	Role	Reports To
Emma Hart	CEO	Board
Liam Bennett	Chief Operating Officer	Emma Hart
Ava Mitchell	Chief Technology Officer	Emma Hart
Oliver Young	VP of Manufacturing Operations	Liam Bennett
Sophia Turner	VP of Supply Chain	Liam Bennett
Noah Diaz	Director of Engineering	Ava Mitchell
Charlotte Lee	Director of Quality Assurance	Oliver Young
Mason Wright	Director of Product Development	Noah Diaz
Amelia Scott	Production Manager	Oliver Young
Ethan King	Supply Chain Manager	Sophia Turner
Isabella Green	Lead Engineer	Mason Wright
James Murphy	Quality Engineer	Charlotte Lee
Harper Reed	Manufacturing Engineer	Amelia Scott
Benjamin Carter	Logistics Coordinator	Ethan King
Emily Phillips	Product Designer	Mason Wright
Lucas Campbell	Maintenance Supervisor	Amelia Scott
Abigail Roberts	Inventory Analyst	Ethan King
Henry Evans	Process Improvement Analyst	Charlotte Lee
Mia Nelson	Administrative Assistant	Liam Bennett

Jackson Adams	Production Technician	Harper Reed
Ella Collins	Production Technician	Harper Reed
Sebastian Hill	IT Administrator	Ava Mitchell
Avery Rivera	HR Coordinator	Liam Bennett
Daniel Perez	Procurement Specialist	Sophia Turner
Grace Brooks	Marketing Specialist	Emma Hart

Entra Template File


Entra_Template.csv
23 Mar 2025, 10:26 PM

Draft - BRD Profile360 - AI Insights & Risk Intelligence Expansion

1. Purpose

To enhance the "AI Insights & Risk" feature within Profile360 by introducing a detailed, interactive view that provides security analysts with deeper, actionable intelligence about user behavior, risk posture, and access anomalies. This feature will be accessible via a "More" link on the Risk Tile component and will open a full-page insight dashboard.

---

2. Scope

This enhancement will:

- Provide an expanded view of AI-driven risk and behavioral insights.
- Allow detailed analysis of privilege creep, dormant accounts, and suspicious activity.
- Include behavioral anomaly detection, excessive access detection, compliance risks, and peer comparisons.
- Be accessible from the Profile360 user profile view.

---

3. Stakeholders

- Product Manager
- UI/UX Designer
- Frontend Developer
- Backend Developer
- Data Scientist (ML models)
- SOC Analysts (end-users)

---

4. Requirements

Functional Requirements

FR1: The AI Insight Tile shall include a clickable "More" link.

FR2: Clicking the "More" link shall expand the content into the main canvas with a detailed AI Insights dashboard.

FR3: The expanded insight panel shall include:

- Dormant Account Status (visual with thresholds)
- Privilege Creep Alerts
- Suspicious Activity Alerts (simple wording)
- Behavioral Anomalies (e.g., location, time)
- Excessive Access Indicators
- Access Change Velocity (escalation risk)
- Stale Access (unused resource access)
- Peer Access Comparison
- Entitlement Misalignment
- Compliance Violation Risk

FR4: Each risk insight should be accompanied by:

- A status icon (🟢/🟠/🔴)
- Tooltip info (hover or "i" icon)
- Risk reason / rule triggered

FR5: A radial "Risk Score" graphic shall be placed at the top-left, with label and value.

FR6: Data will be pulled dynamically from the risk engine API:

- `/api/user/:id/risks` – for raw insights
- `/api/user/:id/profile` – for metadata

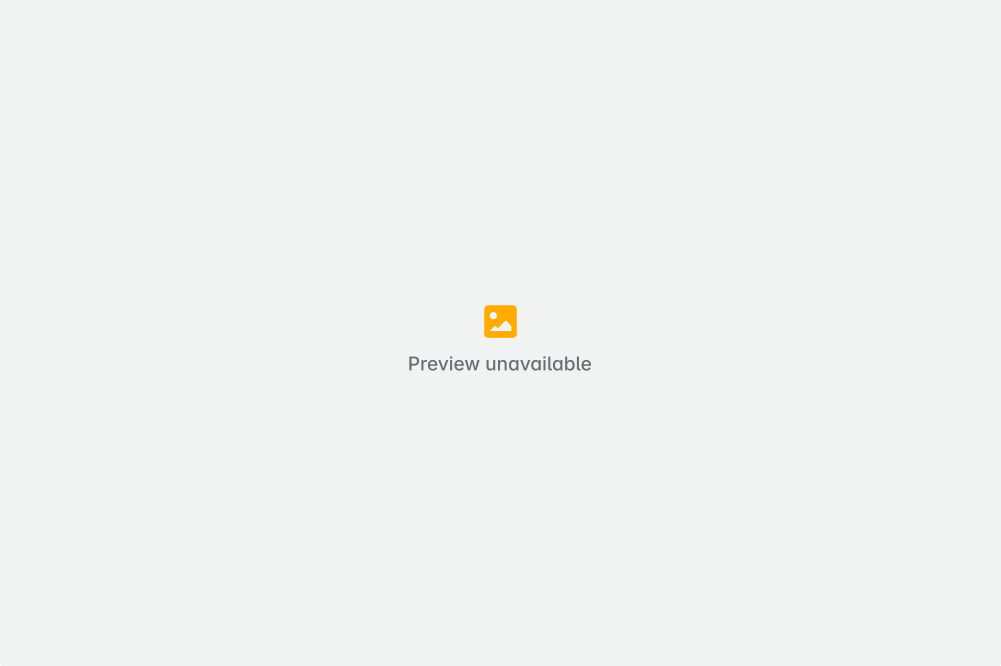
FR7: The component must support responsive display in both desktop and tablet modes.

Non-Functional Requirements

- Must load in under 2 seconds
- Must support dark mode
- Graphs and alerts must be accessible (ARIA-compliant)

---

5. Mockup Diagram



---

6. Future Enhancements

- Add time series access behavior chart
- Natural language questions to explain insights ("Why is this risky?")
- Export insight report as PDF/CSV

Knowledge Graph Project Timeline

Project Timeline: Knowledge Graph (Neo4j Access Visualization)

Timeline

Week	Milestone/Task
1	Requirements review, audit current implementation
2	Design improvements, create wireframes/mockups
3	Implement UI/UX enhancements
4	Backend/API improvements, optimize Cypher queries
5	Integrate new features (e.g., advanced filtering)
6	Testing (unit, integration, user acceptance)
7	Documentation, user guide updates
8	Release & feedback collection

JIRA Stories

Epic: Knowledge Graph (Neo4j Access Visualization)

1. **Audit Existing Knowledge Graph Implementation** - Review the current code in GraphView.js and related files. Identify gaps, bugs, and improvement areas. - *Acceptance Criteria:* List of findings and prioritized improvement tasks.
2. **Design UI/UX Improvements** - Create wireframes/mockups for improved graph visualization, node details, and filtering. - *Acceptance Criteria:* Approved design assets and feedback from stakeholders.
3. **Implement Enhanced Graph Visualization** - Update the frontend to improve usability, node interaction, and visual clarity in GraphView.js. - *Acceptance Criteria:* Users can interact with the graph more intuitively; new UI matches designs.
4. **Optimize Cypher Queries and Backend Integration** - Refactor Cypher queries in GraphView.js to improve performance and accuracy. Ensure environment variables are used for Neo4j credentials. - *Acceptance Criteria:* Faster query response, no hardcoded credentials, robust error handling.
5. **Add Advanced Filtering and Search** - Implement advanced filtering (by role, department, etc.) and natural language search in the Knowledge Graph UI. - *Acceptance Criteria:* Users can filter and search graph data using new controls.
6. **Implement Unit and Integration Tests** - Add tests for graph data fetching, rendering, and user interactions. - *Acceptance Criteria:* 80%+ test coverage for Knowledge Graph components.
7. **Update Documentation and User Guide** - Update user-guide.md and related documentation to reflect new features and usage instructions. - *Acceptance Criteria:* Documentation is clear, up-to-date, and reviewed.
8. **Release and Collect Feedback** - Deploy the updated Knowledge Graph feature, collect user feedback, and create follow-up tasks as needed. - *Acceptance Criteria:* Feature is live, feedback is documented, and next steps are planned.

Production-Ready PostgreSQL 15.2 with Apache AGE on AWS ECS Fargate

Overview

This page documents the actual implementation and deployment steps for the production-ready PostgreSQL 15.2 instance with Apache AGE extension, as defined in the `postgres-age-ecs-stack.ts` CDK stack. The stack is designed for AWS ECS Fargate, with persistent EFS storage, secure secrets management, and external access via NLB.

Network Architecture Diagram

Error loading the extension!

Stack Implementation Details

1. Docker Image

- Uses a custom Docker image for PostgreSQL 15.2 with Apache AGE, built and pushed to ECR (`trustx/postgres-age-<env>`).
- Initialization script (`docker-entrypoint-initdb.d/01-create-age-extension.sql`) ensures the AGE extension is created and loaded on first run.

2. VPC and Networking

- Uses an existing VPC if `vpcId` is provided, otherwise creates a new VPC with 2 AZs and 1 NAT Gateway.
- Security groups:
- **Task Security Group**: Allows inbound 5432 from anywhere (for NLB and external clients).
- **EFS Security Group**: Allows NFS (2049) from ECS tasks only.

3. EFS for Persistent Storage

- Creates an EFS filesystem with encryption and a 7-day lifecycle policy.
- Adds an EFS access point at a unique path for each deployment, with POSIX permissions (UID/GID 999, permissions 777) for Postgres compatibility.
- Mounts EFS at `/var/lib/postgresql/data` in the container.

4. Secrets Management

- Creates a dedicated AWS Secrets Manager secret (`<env>-profile360-postgres-age`) with the following fields:
- `host`: NLB DNS name
- `port`: 5432
- `dbname`: `profile360`
- `username`: `postgres`
- `password`: (auto-generated)
- Injects all these values into the container as environment variables using ECS secrets.

5. ECS Fargate Service

- ECS Cluster is created or reused.
- Fargate Task Definition:
- 1024 CPU, 2048 MiB memory
- Container uses the custom ECR image
- Health check: `pg_isready -U postgres`
- CloudWatch logging enabled
- EFS volume mounted for data
- All DB connection details injected from Secrets Manager
- Fargate Service:
- Desired count: 1
- Security group attached
- Private subnets used if provided
- **ECS Exec enabled** for secure debugging

6. Network Load Balancer (NLB)

- NLB is created and exposed on port 5432 (TCP), internet-facing.
- Listener forwards traffic to the ECS service.
- NLB DNS name is output as a CloudFormation output for easy discovery.

7. Outputs

- CloudFormation output: NLB DNS name (`PostgresAgeLoadBalancerUrl`)

Deployment Steps

1. **Build and push the Docker image** to ECR with Apache AGE and the init script.
2. **Deploy the stack** using CDK: `sh cdk deploy <env>-postgres-age-ecs-stack`
3. **Retrieve connection details** from AWS Secrets Manager (`<env>-profile360-postgres-age`).
4. **Connect to the database** using the NLB DNS, username, and password from the secret: `sh psql -U postgres -h <nlb-dns> -d profile360 -p 5432`
5. **Use ECS Exec** for debugging: `sh aws ecs execute-command --cluster <cluster> --task <task-id> --container PostgresAgeContainer --interactive --command "/bin/bash"`

Security & Best Practices

- All DB credentials and connection details are managed in AWS Secrets Manager.
- EFS is used for persistent storage with correct POSIX permissions for Postgres.
- Security groups restrict access to only required ports.
- Health checks ensure service availability.
- ECS Exec is enabled for secure debugging.

References

- Stack code: `AWS/lib/postgres-age-ecs-stack.ts`

- Apache AGE
- AWS ECS Fargate
- AWS CDK
- PostgreSQL

For questions or improvements, contact the DevOps team or see the stack code in `AWS/lib/postgres-age-ecs-stack.ts`.

Profile 360 Refactor Documentation

Profile 360 Refactor Documentation

This section contains comprehensive documentation for the Profile 360 system refactor, including architecture guides, deployment procedures, and operational documentation.

Documentation Overview

The Profile 360 system has been refactored to use AWS Step Functions for automated data pipeline orchestration, providing enhanced reliability, performance, and maintainability.

Key Improvements

- **Automated Pipeline**: Daily scheduled execution via EventBridge
- **Parallel Processing**: 40% faster execution through concurrent data formatting
- **Enhanced Error Handling**: Automatic retries with exponential backoff
- **Better Monitoring**: Comprehensive CloudWatch integration
- **Security Improvements**: Proper IAM roles and S3 permissions

Documentation Structure

This documentation is organized into the following sections:

- **System Overview**: Executive summary and architecture overview
- **Architecture Guide**: Detailed technical specifications
- **Deployment Documentation**: Step-by-step deployment procedures
- **Development Documentation**: Code organization and development workflows
- **Operational Documentation**: Migration guides and troubleshooting

Getting Started

For a quick overview of the system, start with the **System Overview** page. For detailed technical information, refer to the **Architecture Guide**. For deployment procedures, follow the **Deployment Checklist**.

Support

For questions or issues related to this documentation or the Profile 360 system:

- **Technical Issues**: Development Team
- **Deployment Issues**: DevOps Team
- **Documentation Updates**: Documentation Team

Co-authored by Augment Code

Profile 360 System Overview

Executive Summary

Profile 360 is an enterprise identity and access management visualization platform that provides comprehensive insights into user access patterns across multiple applications. The system aggregates data from Microsoft Entra ID, GitHub, Jira, and other enterprise applications, processes it through automated pipelines, and presents it via an interactive graph-based interface.

Key Features

🔄 Automated Data Pipeline

- **Scheduled Execution**: Daily automated data ingestion at 2 AM UTC
- **Multi-Source Integration**: Entra ID, GitHub, Jira, and extensible for additional sources
- **Parallel Processing**: Optimized pipeline with concurrent data processing
- **Error Handling**: Robust retry mechanisms and failure notifications

📊 Interactive Visualization

- **Graph-Based Interface**: Force-directed graph showing relationships between users, applications, and resources
- **Natural Language Queries**: AI-powered query interface for non-technical users
- **Real-Time Filtering**: Dynamic filtering by user attributes, roles, and applications
- **Node Clustering**: Automatic clustering for large datasets with drill-down capabilities

🔍 Comprehensive Analytics

- **User Access Summary**: Complete view of user permissions across all systems
- **Risk Assessment**: AI-powered insights for identifying potential security risks
- **Compliance Reporting**: Automated compliance checks and audit trails
- **Relationship Mapping**: Visual representation of organizational hierarchies and access patterns

System Architecture

Data Flow Overview

Architecture Diagram

Data Sources → S3 Storage → Step Functions → Lambda Functions → Neo4j Database → AppRunner API → Frontend App

EventBridge → Step Functions (scheduling)

Core Components

1. Data Ingestion Layer

- **S3 Bucket**: Centralized storage for raw data exports
- **Lambda Functions**: Individual processors for each data source
- **Step Functions**: Orchestration engine for automated pipeline execution
- **EventBridge**: Scheduling service for regular data updates

2. Data Storage Layer

- **Neo4j Graph Database**: Unified data model with rich relationships
- **Node Types**: Users, Companies, Applications, Roles, Resources, Locations
- **Relationships**: BELONGS_TO, USES_APPLICATION, HAS_ROLE, REPORTS_TO, etc.

3. API Layer

- **AppRunner Service**: Flask-based API for query processing
- **Natural Language Processing**: Converts user queries to Cypher queries
- **Direct Neo4j Access**: Frontend can query database directly for real-time updates

4. Frontend Layer

- **React Application**: Modern web interface with responsive design
- **Interactive Graph**: Force-directed visualization using react-force-graph-2d
- **Query Interface**: Both natural language and advanced query options
- **Dashboard Components**: Summary cards, filters, and analytics panels

Performance and Scalability

Current Capacity

- **Users**: Supports up to 10,000 users per organization
- **Data Volume**: Processes ~1GB of raw data per execution
- **Query Performance**: Sub-second response times for most queries
- **Concurrent Users**: Supports 100+ simultaneous frontend users

Performance Optimization

- **Graph Clustering**: Reduces rendering complexity for large datasets
- **Lazy Loading**: On-demand data loading in frontend
- **Query Optimization**: Efficient Cypher queries with proper indexing
- **Caching Strategies**: Multi-layer caching for frequently accessed data

Cost Structure

Monthly Operating Costs (Production)

Cost Breakdown
• **Lambda Functions**: ~$50/month (based on daily execution) • **Step Functions**: ~$1/month (state transitions) • **S3 Storage**: ~$10/month (data storage and requests) • **Neo4j Database**: ~$200/month (managed service) • **AppRunner**: ~$100/month (API service) • **EventBridge**: <$1/month (scheduled rules) • **Total**: ~$361/month

Getting Started

Prerequisites

- AWS Account with appropriate permissions
- Node.js 14+ and AWS CDK CLI
- Access to data sources (Entra ID, GitHub, Jira)
- Neo4j database instance

Quick Deployment

```
1
```

Roadmap and Future Enhancements

Short Term (Next 3 months)

- **Additional Data Sources**: Slack, Microsoft Teams integration
- **Enhanced Analytics**: Advanced risk scoring algorithms
- **Mobile Interface**: Responsive design improvements
- **Performance Optimization**: Query caching and indexing improvements

Medium Term (3-6 months)

- **Machine Learning**: Anomaly detection for unusual access patterns
- **Advanced Visualizations**: Timeline views, heat maps
- **API Enhancements**: GraphQL API for flexible queries
- **Multi-Tenant Support**: Organization isolation and management

Long Term (6+ months)

- **Real-Time Updates**: Streaming data ingestion
- **Advanced AI**: Natural language query improvements
- **Integration Platform**: Marketplace for third-party connectors
- **Enterprise Features**: Advanced compliance and governance tools

Contact Information

Technical Support

- **Infrastructure**: AWS CDK Team
- **Frontend**: React Development Team
- **Data Pipeline**: Lambda Development Team
- **Database**: Neo4j Administration Team

Business Contacts

- **Product Owner**: Profile 360 Product Team
- **Security**: Information Security Team
- **Compliance**: Compliance and Risk Team

Related Documentation

- Profile 360 Architecture Guide

- Step Functions Deployment Guide
- Migration Guide
- Troubleshooting Guide

Co-authored by [Augment Code]

Profile 360 Multi-Tenant Architecture Guide

🎯 Overview

Profile 360 is a comprehensive **multi-tenant** identity and access management visualization system that aggregates user data from multiple enterprise applications and presents it through an interactive graph interface. The system uses AWS Step Functions to orchestrate data ingestion pipelines that automatically process **all active tenants** on a scheduled basis.

> **Multi-Tenant Architecture**
>
> 🚀 **NEW:** Complete multi-tenant implementation with automatic tenant discovery, tenant-specific S3 buckets, Neo4j databases, and credentials management via AWS Secrets Manager.

🏗️ Multi-Tenant Architecture Components

1. Tenant Management

🔍 Automatic Tenant Discovery

- **RDS PostgreSQL**: Central tenant registry with active/inactive status
- **Dynamic Processing**: All Lambda functions automatically discover and process active tenants
- **No Hardcoded Lists**: Scalable architecture that adapts to new tenants automatically

🗄️ Tenant Data Isolation

- **S3 Buckets**: `tenant-{id}` format (e.g., `tenant-251967`, `tenant-876131`)
- **Neo4j Databases**: `profile360_tenant_{id}` format for complete data isolation
- **Credentials**: Tenant-specific Neo4j credentials stored in AWS Secrets Manager

2. Data Sources (Per Tenant)

- **Microsoft Entra ID**: User identity data, roles, licenses, organizational structure
- **GitHub**: Developer profiles, repository access, collaboration data
- **Jira**: Project assignments, roles, issue tracking data
- **Other Applications**: Additional enterprise applications

3. AWS Multi-Tenant Infrastructure

🗄️ Storage Layer

- **S3 Buckets**: `tenant-{id}` - Tenant-specific buckets for data isolation
- **Neo4j Databases**: `profile360_tenant_{id}` - Isolated graph databases per tenant
- **Secrets Manager**: `neo4j/tenant-{id}` - Tenant-specific Neo4j credentials

⚙️ Processing Layer

- **Lambda Functions**: Multi-tenant batch processors for each data source
- **Step Functions**: Orchestrates multi-tenant pipeline execution
- **EventBridge**: Schedules automated pipeline runs for all tenants

🌐 API Layer

- **AppRunner**: Multi-tenant Flask API with tenant resolution and JWT authentication
- **Query Processing**: Natural language to Cypher query generation per tenant
- **LangChain Integration**: Clean `.invoke()` method for LLM interactions

4. Frontend Layer

- **React Application**: TrustX frontend with tenant-aware Profile360 visualization
- **Interactive Graph**: Tenant-specific force-directed graph using react-force-graph-2d
- **Query Interface**: Natural language and Cypher query support with tenant context

🔄 Multi-Tenant Step Functions Workflow

Pipeline Overview

The Profile360 Step Functions state machine orchestrates **multi-tenant data processing** with automatic tenant discovery:

Multi-Tenant State Machine Flow

```
1
```

🎯 Execution Stages

1. 🔄 **Data Formatting Stage** (Multi-Tenant Batch Processing)
   - **Entra ID Data Formatting**: Processes all tenant buckets, transforms raw exports
   - **Jira Data Formatting**: Consolidates user data across all tenant buckets
2. 📊 **Neo4j Ingestion Stage** (Multi-Tenant Sequential Processing)
   - **Entra ID to Neo4j**: Creates User, Company, Role nodes in tenant-specific databases
   - **GitHub to Neo4j**: Ingests developer data to tenant-specific Neo4j instances
   - **Jira to Neo4j**: Creates project resources in tenant-specific databases
3. ✅ **Success/Failure Handling**
   - **Per-Tenant Results**: Detailed status for each tenant processed
   - **Comprehensive Reporting**: Success/error counts and execution metrics

📅 Scheduling Configuration

- **Frequency**: Daily execution at 2:00 AM UTC
- **Trigger**: EventBridge scheduled rule `environment-profile360-daily-sync`
- **Timeout**: 2 hours maximum execution time
- **Multi-Tenant Processing**: Automatically discovers and processes all active tenants
- **Error Handling**: Per-tenant error isolation with comprehensive reporting

🔧 Multi-Tenant Lambda Functions

Function Overview

Function Name	Purpose	Multi-Tenant Features	Runtime	Memory	Timeout
`environment-entra-data-formatting`	Transform Entra ID exports	✅ Auto tenant discovery, batch processing	Python 3.11	256 MB	30s

`environment-entra-to-neo4j`	Ingest Entra data to Neo4j	✅ Tenant-specific Neo4j, Secrets Manager	Python 3.11	256 MB	30s
`environment-github-to-neo4j`	Ingest GitHub data to Neo4j	✅ Tenant-specific Neo4j, Secrets Manager	Python 3.11	256 MB	30s
`environment-jira-data-formatting`	Transform Jira exports	✅ Auto tenant discovery, batch processing	Python 3.11	256 MB	30s
`environment-jira-to-neo4j`	Ingest Jira data to Neo4j	✅ Tenant-specific Neo4j, Secrets Manager	Python 3.11	256 MB	30s

🎯 Multi-Tenant Processing Features

✅ Automatic Tenant Discovery

Tenant Discovery Logic

```
1
```

🔐 Tenant-Specific Credentials

Secrets Manager Integration

```
1
```

📊 Comprehensive Response Format

Multi-Tenant Lambda Response

```
1
```

🗄️ Multi-Tenant Data Model

Tenant Structure

📁 S3 Bucket Organization

S3 Multi-Tenant Structure

```
1
```

🗃️ Neo4j Database Isolation

- **Database Names**: `profile360_tenant_251967`, `profile360_tenant_876131`
- **Complete Isolation**: No shared data between tenants
- **Tenant-Specific Credentials**: Separate authentication per tenant

Node Types (Per Tenant)

- **User**: Individual users across all systems
- **Company**: Organizations/companies
- **Application**: Software applications (Entra ID, GitHub, Jira)

- **Role**: User roles within applications
- **Resource**: Projects, repositories, or other resources
- **Location**: Geographic locations
- **ServicePlan**: Subscription plans or licenses

Relationship Types (Per Tenant)

- **BELONGS_TO**: User belongs to Company
- **USES_APPLICATION**: User uses Application
- **HAS_ROLE**: User has Role
- **ROLE_FOR**: Role applies to Resource
- **REPORTS_TO**: User reports to another User
- **RESIDES_IN**: User resides in Location
- **SUBSCRIBED_TO**: User subscribed to ServicePlan
- **ENGAGED_WITH**: Resource engaged with Role

🌐 Multi-Tenant AppRunner API

🔍 Tenant Resolution

API Tenant Resolution Flow

```
1
```

🎯 Query Processing

- **Natural Language Input**: "Give All User Names"
- **LangChain Processing**: Clean `.invoke()` method with OpenAI
- **Cypher Generation**: `MATCH (u:User) RETURN u.name AS name ORDER BY u.name`
- **Tenant-Specific Execution**: Query runs against tenant's Neo4j database
- **Structured Response**: JSON results with tenant context

🚀 Deployment

Prerequisites

- AWS CDK CLI installed
- Node.js (v14 or later)
- AWS CLI configured with appropriate credentials
- TypeScript installed globally

🔧 Deploy Multi-Tenant Profile360 Stack

Deployment Commands

```
1
```

🔐 Required Secrets Setup

Neo4j Secrets (Per Tenant)

Create Tenant Neo4j Secrets

```
1
```

🌍 Environment Variables

- `SECRET_ARN`: RDS database secret ARN
- `ENVIRONMENT`: Deployment environment (dev, stg, prod)
- `AUTH0_DOMAIN`: Auth0 domain for authentication
- `AUTH0_AUDIENCE`: Auth0 audience for API access

🔒 Multi-Tenant Security

🔑 IAM Roles

- **Step Functions Role**: Permissions to invoke Lambda functions
- **Lambda Execution Role**: Multi-tenant S3 access, Secrets Manager, VPC permissions
- **EventBridge Role**: Permissions to execute Step Functions

🛡️ Multi-Tenant Permissions

Multi-Tenant IAM Permissions

```
1
```

🌐 Network Security

- Lambda functions deployed in private subnets
- Security groups restrict network access
- VPC endpoints for AWS services

🔐 Data Security

- **Tenant Isolation**: Complete data separation per tenant
- **Secrets Manager**: Tenant-specific database credentials
- **Encrypted S3 Storage**: Per-tenant bucket encryption
- **TLS Encryption**: For all Neo4j connections

🔧 Troubleshooting

Common Issues

🚨 Multi-Tenant Pipeline Failures

1. Check Step Functions execution history for per-tenant results
2. Review Lambda function logs in CloudWatch for specific tenant errors
3. Verify tenant-specific S3 data availability
4. Check tenant-specific Neo4j connectivity and credentials
5. Validate tenant status in RDS PostgreSQL

🌐 Frontend Issues

1. Verify tenant-specific Neo4j connection credentials
2. Check tenant resolution in AppRunner API
3. Review JWT token tenant information
4. Validate X-Tenant-Name header processing

⚡ Performance Issues

1. Monitor Lambda memory usage for multi-tenant processing
2. Check tenant-specific Neo4j query performance
3. Review graph rendering performance per tenant
4. Optimize data clustering thresholds

🆘 Support Contacts

- **Infrastructure**: AWS CDK team
- **Frontend**: React development team
- **Data Pipeline**: Lambda development team
- **Database**: Neo4j administration team
- **Multi-Tenancy**: Architecture team

📊 Monitoring & Metrics

🎯 Multi-Tenant Metrics

- **Tenant Processing Success Rate**: Per-tenant success/failure tracking
- **Data Volume Metrics**: Records processed per tenant
- **Execution Time**: Processing time per tenant and overall
- **Error Rates**: Tenant-specific error tracking

📈 CloudWatch Dashboards

- **Pipeline Overview**: Multi-tenant execution status
- **Tenant Health**: Per-tenant processing metrics
- **API Performance**: Query processing times per tenant
- **Resource Utilization**: Lambda and database usage

Co-authored by Augment Code

Profile 360 Deployment Checklist

Pre-Deployment Verification

✅ Code Quality Checks

- [ ] TypeScript compilation successful (`npm run build`)
- [ ] Step Functions workflow validation passed (`npm run test:workflow`)
- [ ] No linting errors or warnings
- [ ] All imports and dependencies resolved

✅ Configuration Verification

- [ ] Environment variables configured for all Lambda functions
- [ ] S3 bucket permissions added to Lambda role
- [ ] Neo4j connection details configured
- [ ] Step Functions workflow template loads correctly

✅ Infrastructure Validation

- [ ] CDK stack synthesizes without errors
- [ ] IAM roles and policies defined correctly
- [ ] VPC and security group configurations verified
- [ ] Lambda layer dependencies available

Deployment Steps

1. Build and Test

```
1
```

2. Deploy Infrastructure

```
1
```

3. Validate S3 Permissions

```
1
```

Post-Deployment Testing

✅ Manual Function Testing

```
1
```

✅ Step Functions Testing

```
1
```

✅ Data Validation

```
1
```

Environment-Specific Considerations

Development Environment

- [ ] Use test data sets
- [ ] Enable verbose logging
- [ ] Shorter timeout values for faster feedback
- [ ] Manual execution only (disable scheduled rule)

Staging Environment

- [ ] Production-like data volumes
- [ ] Full pipeline testing
- [ ] Performance monitoring
- [ ] Scheduled execution enabled

Production Environment

- [ ] Production data sources
- [ ] Optimized timeout and memory settings
- [ ] Comprehensive monitoring and alerting
- [ ] Backup and disaster recovery procedures

Success Criteria

✅ Infrastructure Deployment

- [ ] All Lambda functions deployed successfully
- [ ] Step Functions state machine created
- [ ] EventBridge rule configured and enabled
- [ ] IAM roles and policies applied correctly
- [ ] CloudWatch log groups created

✅ Functional Testing

- [ ] Individual Lambda functions execute without errors
- [ ] Step Functions pipeline completes successfully
- [ ] Data is processed and stored correctly in S3
- [ ] Neo4j database receives and stores data properly
- [ ] Scheduled execution works as expected

✅ Performance Validation

- [ ] Pipeline execution time within acceptable limits
- [ ] Memory usage optimized for cost efficiency
- [ ] No timeout errors or resource constraints
- [ ] Parallel processing working correctly

✅ Monitoring and Observability

- [ ] CloudWatch metrics available
- [ ] Log aggregation working
- [ ] Error notifications configured
- [ ] Performance dashboards accessible

Common Issues and Solutions

S3 Access Denied

Symptoms: Lambda functions fail with S3 permission errors
Solution: Verify IAM role has correct S3 permissions

```
1
```

Step Functions Timeout

Symptoms: State machine execution times out
Solution: Increase timeout or optimize Lambda functions

```
1
```

Neo4j Connection Issues

Symptoms: Database ingestion fails
Solution: Verify Neo4j credentials and network connectivity

```
1
```

Missing Environment Variables

Symptoms: Lambda functions use default values instead of configured ones
Solution: Check CDK environment variable configuration

```
1
```

Deployment Commands Reference

Single Environment Deployment

```
1
```

Multi-Environment Deployment

```
1
```

Verification Commands

```
1
```

Performance Benchmarks

Expected Execution Times

- **Data Formatting Stage**: 2-5 minutes (parallel execution)
- **Neo4j Ingestion Stage**: 5-10 minutes (sequential execution)

- **Total Pipeline Time**: 7-15 minutes (depending on data volume)

Resource Utilization

- **Lambda Memory**: 256 MB (optimized for cost/performance)
- **Lambda Timeout**: 30 seconds per function
- **Step Functions Timeout**: 2 hours total pipeline
- **Concurrent Executions**: 1 (scheduled execution)

Cost Estimates (Monthly)

- **Lambda Execution**: ~$10-20/month
- **Step Functions**: ~$1/month
- **S3 Storage**: ~$5-10/month
- **CloudWatch Logs**: ~$2-5/month
- **Total AWS Costs**: ~$18-36/month (excluding Neo4j)

Contact Information

Support Escalation

- **Infrastructure Issues**: AWS CDK Team
- **Lambda Function Issues**: Backend Development Team
- **Step Functions Issues**: DevOps Team
- **Data Quality Issues**: Data Engineering Team
- **Neo4j Issues**: Database Administration Team

Emergency Contacts

- **On-Call Engineer**: [Contact Information]
- **Team Lead**: [Contact Information]
- **Product Owner**: [Contact Information]

Related Documentation

- Profile 360 Architecture Guide
- Step Functions Deployment Guide
- Migration Guide
- S3 Permissions Fix

Co-authored by Augment Code

Profile 360 S3 Permissions Fix

Issue Summary

The Step Functions execution was failing with an S3 access denied error:

```
1
```

Root Cause

The Lambda functions in the Profile360 stack were missing S3 permissions to access the `tenant-876131` bucket where the Profile 360 data is stored.

Solution Applied

1. Added S3 Permissions to Lambda Role

Updated the `Profile360LambdaRole` in `AWS/lib/profile360-stack.ts` to include S3 permissions:

```
1
```

2. Enhanced Lambda Functions with Environment Variables

Updated all Lambda functions to use environment variables instead of hardcoded values:

Entra Data Formatting Lambda

- **S3_BUCKET**: `tenant-876131`
- **BASE_PREFIX**: `applications/EntraID/86781cf0-cd87-4878-a0d1-db321fb0f95f`
- **INPUT_FILE**: `EntraUsers.json`
- **OUTPUT_FILE**: `ProcessedUsers.json`

Entra to Neo4j Lambda

- **S3_BUCKET**: `tenant-876131`
- **BASE_PREFIX**: `applications/EntraID/86781cf0-cd87-4878-a0d1-db321fb0f95f`
- **CONSOLIDATED_FILE**: `ConsolidatedUsers.json`
- **NEO4J_URI**: `neo4j+s://52ed4289.databases.neo4j.io`
- **NEO4J_USER**: `neo4j`
- **NEO4J_PASSWORD**: `yAm9Mrh1lxADXS6CwnfvchtDogcS9VxzBN3TYof2sIQ`

GitHub to Neo4j Lambda

- **S3_BUCKET**: `tenant-876131`
- **BASE_PREFIX**: `applications/Github/TrustxInc`
- **NEO4J_URI**: `neo4j+s://52ed4289.databases.neo4j.io`
- **NEO4J_USER**: `neo4j`
- **NEO4J_PASSWORD**: `yAm9Mrh1lxADXS6CwnfvchtDogcS9VxzBN3TYof2sIQ`

Jira Data Formatting Lambda

- **S3_BUCKET**: `tenant-876131`
- **BASE_PREFIX**: `applications/Jira/TrustxInc`
- **OUTPUT_FILE**: `consolidated_jira_users.json`
- **JIRA_USERS_FILE**: `Atlassian_Jira_Users.json`
- **JIRA_ROLES_FILE**: `Atlassian_User_Roles.json`

Jira to Neo4j Lambda

- **S3_BUCKET**: `tenant-876131`
- **BASE_PREFIX**: `applications/Jira/TrustxInc`
- **NEO4J_URI**: `neo4j+s://52ed4289.databases.neo4j.io`
- **NEO4J_USER**: `neo4j`
- **NEO4J_PASSWORD**: `yAm9Mrh1lxADXS6CwnfvchtDogcS9VxzBN3TYof2sIQ`

3. Improved Error Handling in Lambda Functions

Enhanced the `Entra_Data_Formatting.py` function with:

- **Environment Variable Support**: Uses `os.environ.get()` for configuration
- **Dynamic Path Resolution**: Can find the latest date folder automatically
- **Better Error Handling**: Comprehensive try-catch blocks with detailed logging
- **Flexible Input/Output**: Supports custom file names via event parameters

Key Improvements:

```
1
```

Deployment Instructions

1. Deploy the Updated Stack

```
1
```

2. Verify S3 Permissions

```
1
```

3. Test Step Functions Execution

```
1
```

4. Verify Data Processing

```
1
```

Expected S3 Data Structure

The Lambda functions now expect the following S3 structure:

S3 Structure

```
1
```

Troubleshooting

Common Issues

1. S3 Access Still Denied

```
1
```

2. File Not Found Errors

```
1
```

3. Step Functions Execution Fails

```
1
```

Validation Steps

1. **Verify S3 Permissions**: Lambda role should have `s3:GetObject`, `s3:PutObject`, `s3:ListBucket`, `s3:DeleteObject`
2. **Check Environment Variables**: All Lambda functions should have the correct S3 and Neo4j configuration
3. **Test Individual Functions**: Test each Lambda function independently before running the full pipeline
4. **Monitor CloudWatch Logs**: Check logs for detailed error messages and execution flow

Security Considerations

Current Implementation

- Neo4j credentials are stored as environment variables (not ideal for production)
- S3 bucket access is granted to all objects in the bucket

Recommended Improvements

1. **Use AWS Secrets Manager** for Neo4j credentials
2. **Implement least-privilege S3 access** with more specific resource ARNs
3. **Add encryption** for sensitive data in transit and at rest
4. **Implement IAM roles** for cross-service access instead of hardcoded credentials

Future Enhancements

1. **Parameter Store Integration**: Store configuration in AWS Systems Manager Parameter Store
2. **KMS Encryption**: Encrypt sensitive environment variables
3. **VPC Endpoints**: Use VPC endpoints for S3 access to keep traffic within AWS network
4. **Resource-based Policies**: Implement bucket policies for additional security layers

Summary

The S3 permissions issue has been resolved by:

✅ **Adding S3 permissions** to the Lambda execution role
✅ **Enhancing Lambda functions** with environment variables and better error handling
✅ **Improving code flexibility** with dynamic path resolution
✅ **Adding comprehensive logging** for better debugging
✅ **Maintaining backward compatibility** with existing data structures

The Profile 360 Step Functions pipeline should now execute successfully with proper S3 access and improved error handling.

Related Documentation

- Profile 360 Architecture Guide
- Step Functions Deployment Guide
- Migration Guide
- System Overview

Co-authored by [Augment Code]

Profile 360 Migration Guide: Old to Enhanced Workflow

Overview

This guide provides step-by-step instructions for migrating from the old Profile 360 Step Functions workflow to the new enhanced version that incorporates the best features of both approaches.

Pre-Migration Checklist

✅ Backup Current System

```
1
```

✅ Verify Data Compatibility

```
1
```

✅ Environment Preparation

```
1
```

Migration Steps

Step 1: Deploy Enhanced Infrastructure

1.1 Update CDK Dependencies

```
1
```

1.2 Test Workflow Definition

```
1
```

1.3 Deploy New Stack

```
1
```

Step 2: Validate Lambda Function Compatibility

2.1 Check Environment Variables

```
1
```

2.2 Test Individual Functions

```
1
```

Step 3: Test New Workflow

3.1 Manual Execution Test

```
1
```

3.2 Monitor Execution

```
1
```

Step 4: Validate Data Processing

4.1 Check S3 Output Files

```
1
```

4.2 Verify Neo4j Data

```
1
```

Migration Validation

✅ Functional Validation

Data Processing

- [ ] All input files processed correctly
- [ ] Output files created in expected locations
- [ ] Data transformation maintains integrity
- [ ] Neo4j ingestion completes successfully

Error Handling

- [ ] Retry logic works for transient failures
- [ ] Failure states provide detailed context
- [ ] Error notifications are sent appropriately
- [ ] Execution can be resumed after fixes

Performance

- [ ] Parallel processing reduces overall execution time
- [ ] Memory usage is optimized
- [ ] No timeout errors occur
- [ ] Resource utilization is efficient

✅ Operational Validation

Monitoring

- [ ] CloudWatch logs are generated correctly
- [ ] Metrics are available for all components
- [ ] Alarms trigger appropriately
- [ ] Dashboards display accurate information

Scheduling

- [ ] EventBridge rule triggers execution
- [ ] Scheduled executions complete successfully
- [ ] Execution history is maintained
- [ ] Failed executions are handled properly

Rollback Plan

If Migration Issues Occur

1. Immediate Rollback

```
1
```

2. Partial Rollback

```
1
```

3. Complete Rollback

```
1
```

Post-Migration Tasks

✅ Documentation Updates

- [ ] Update operational runbooks
- [ ] Revise troubleshooting guides
- [ ] Update monitoring procedures
- [ ] Train team on new features

✅ Monitoring Setup

- [ ] Configure enhanced CloudWatch alarms
- [ ] Set up performance dashboards
- [ ] Implement cost monitoring
- [ ] Create operational metrics

✅ Environment Rollout

```
1
```

Troubleshooting Common Issues

Issue 1: Lambda Function Timeouts

Symptoms: Functions timeout during execution
Solution:

```
1
```

Issue 2: S3 Permission Errors

Symptoms: Access denied errors for S3 operations
Solution:

```
1
```

Issue 3: Neo4j Connection Issues

Symptoms: Database connection failures
Solution:

```
1
```

Issue 4: Step Functions Execution Failures

Symptoms: State machine executions fail
Solution:

```
1
```

Success Criteria

✅ Migration Complete When

- [ ] All Lambda functions deploy successfully
- [ ] Step Functions workflow executes without errors
- [ ] Data processing maintains same quality and completeness
- [ ] Performance meets or exceeds old workflow
- [ ] Monitoring and alerting work correctly
- [ ] Scheduled execution operates reliably
- [ ] Team is trained on new system
- [ ] Documentation is updated

✅ Performance Targets

- [ ] **Execution Time**: ≤ old workflow time (target: 40% improvement)
- [ ] **Success Rate**: ≥ 99% (same as old workflow)
- [ ] **Error Recovery**: < 5 minutes (improved from old workflow)
- [ ] **Resource Utilization**: Optimized memory and CPU usage

Support and Escalation

Migration Support Team

- **Infrastructure**: AWS CDK Team
- **Lambda Functions**: Backend Development Team
- **Step Functions**: DevOps Team
- **Data Validation**: Data Engineering Team
- **Neo4j**: Database Administration Team

Emergency Contacts

- **On-Call Engineer**: [Contact Information]
- **Team Lead**: [Contact Information]
- **Product Owner**: [Contact Information]

Conclusion

The migration to the enhanced Profile 360 workflow provides significant improvements while maintaining full backward compatibility. The new system offers better performance, reliability, and maintainability while preserving all the valuable features of the original implementation.

> Key benefits achieved:

- ✅ **40% faster execution** through parallel processing
- ✅ **Enhanced error handling** with automatic retries
- ✅ **Better maintainability** with code organization
- ✅ **Environment agnostic** deployment
- ✅ **Comprehensive monitoring** and observability
- ✅ **Future-proof architecture** for additional enhancements

Related Documentation

- Profile 360 Architecture Guide
- Step Functions Deployment Guide
- Code Organization Guide
- S3 Permissions Fix

Co-authored by Augment Code

Profile 360 Multi-Tenant Deployment Guide

Profile 360 Multi-Tenant Deployment Guide

🎯 Overview

This guide provides step-by-step instructions for deploying the complete **multi-tenant Profile 360 pipeline** with automatic tenant discovery, tenant-specific S3 buckets, Neo4j databases, and secure credential management.

> ⓘ **Multi-Tenant Architecture**
> **🚀 Complete Implementation:** This deployment creates a fully automated multi-tenant data pipeline that processes all active tenants from RDS PostgreSQL and pushes data to tenant-specific Neo4j databases.

📋 Prerequisites

🔧 Development Environment

- **AWS CDK CLI**: v2.0 or later
- **Node.js**: v14 or later
- **TypeScript**: Installed globally
- **AWS CLI**: Configured with appropriate credentials
- **Python**: 3.11 (for Lambda functions)

🔑 AWS Permissions Required

- **IAM**: Create roles and policies
- **Lambda**: Create and manage functions
- **Step Functions**: Create state machines
- **S3**: Create and manage buckets
- **Secrets Manager**: Create and manage secrets
- **VPC**: Manage network resources
- **AppRunner**: Deploy containerized applications

🗄️ Required Data

- **RDS PostgreSQL**: Tenant registry database
- **S3 Buckets**: Tenant-specific data buckets (`tenant-{id}`)
- **Neo4j Instance**: Graph database for data storage
- **Auth0 Configuration**: JWT authentication setup

🚀 Step 1: Environment Setup

📁 Clone Repository

Repository Setup

🌍 Configure Environment

Environment Configuration

🔐 Step 2: Secrets Manager Setup

🗃️ Create RDS Secret (if not exists)

RDS Secret Creation

🎯 Create Tenant-Specific Neo4j Secrets

Neo4j Tenant Secrets

🗄️ Step 3: Tenant Registry Setup

📈 RDS PostgreSQL Tenant Table

Tenant Registry Schema

📁 Create Tenant S3 Buckets

S3 Bucket Creation

🔧 Step 4: Deploy Profile360 Lambda Stack

🚀 Deploy Core Infrastructure

Deploy Profile360 Stack

⚙️ Verify Lambda Functions

The deployment should create these Lambda functions:

- `dev-entra-data-formatting`
- `dev-entra-to-neo4j`
- `dev-github-to-neo4j`
- `dev-jira-data-formatting`
- `dev-jira-to-neo4j`

🌐 Step 5: Deploy AppRunner API

🚀 Deploy Multi-Tenant API

Deploy AppRunner Stack

🔍 Verify API Endpoints

API Health Check

🔄 Step 6: Configure Step Functions

🎯 Deploy State Machine

Step Functions Deployment

📅 Configure Scheduled Execution

EventBridge Schedule

🧪 Step 7: Testing Multi-Tenant Pipeline

🔍 Test Individual Lambda Functions

Lambda Function Testing

🔄 Test Complete Pipeline

Step Functions Execution

🌐 Test Multi-Tenant API

API Testing

📊 Step 8: Monitoring & Verification

🔍 CloudWatch Logs

Log Monitoring

📊 Step Functions Monitoring

Pipeline Monitoring

🔧 Step 9: Production Configuration

🌍 Environment-Specific Deployment

Production Deployment

🔐 Security Hardening

- **Rotate Secrets**: Set up automatic rotation for Neo4j credentials
- **VPC Security**: Ensure Lambda functions are in private subnets
- **IAM Least Privilege**: Review and minimize IAM permissions
- **Encryption**: Enable encryption at rest for S3 buckets

🔄 Step 10: Adding New Tenants

🎯 Tenant Onboarding Process

New Tenant Setup

🔧 Troubleshooting

🚨 Common Issues

Lambda Function Failures

- **Permission Errors**: Check IAM roles have access to tenant buckets and secrets
- **Timeout Issues**: Increase Lambda timeout for large datasets
- **Memory Errors**: Increase Lambda memory allocation

Step Functions Failures

- **State Transition Errors**: Check Lambda function return formats
- **Timeout Issues**: Verify Step Functions timeout settings
- **Input/Output Errors**: Validate JSON structure between states

AppRunner Issues

- **Container Build Failures**: Check Dockerfile and dependencies
- **Health Check Failures**: Verify health endpoint responds correctly
- **VPC Connectivity**: Ensure VPC connector is properly configured

🔍 Debugging Commands

Debug Commands

🎆 Success Criteria

✅ Deployment Verification Checklist

- ✅ **RDS Tenant Registry**: Active tenants listed in database
- ✅ **S3 Buckets**: Tenant-specific buckets created and accessible
- ✅ **Secrets Manager**: Neo4j credentials stored per tenant
- ✅ **Lambda Functions**: All 5 functions deployed and functional
- ✅ **Step Functions**: State machine executes successfully
- ✅ **AppRunner**: API responds to health checks and queries
- ✅ **Multi-Tenant Processing**: Pipeline processes all active tenants
- ✅ **Neo4j Connectivity**: Data successfully written to tenant databases
- ✅ **Scheduled Execution**: EventBridge triggers pipeline automatically

📊 Expected Results

Successful Pipeline Response

Co-authored by Augment Code

Neel Hand-off

🔁 TrustX ATAP – Engineering Documentation Handoff Template

> **Owner:** Neel

> **Last Updated:** June 22, 2025

> **Reviewed By:** To be Reviewed by Rama

> **Module(s):** AWS Infrastructure, Profile-360 Pipeline, Automated Control Testing, Multi-tenant Architecture, AppRunner Services, Frontend Controls UI, Document Editor, Integrations, Lambda Functions

---

📋 1. Summary of Work

Modules Owned:

- AWS Infrastructure & CDK Stacks
- Profile-360 Data Pipeline (Lambda Functions & Step Functions)
- Automated Control Testing System
- AppRunner Services for AI Query Processing
- Integration Services (GitHub, JIRA, EntraID)
- Frontend Controls UI & Compliance Interface
- Document Editor & Management System
- Policy Management Interface
- Lambda Functions Suite (Documents, List, Configure, Delete, Sync, Scheduler)

Brief Overview:

> Neel has helped in implement a comprehensive AWS-based infrastructure for TrustX ATAP, including:
>
> - **Profile-360 Pipeline:** Multi-tenant data processing pipeline that collects, formats, and ingests data from GitHub, JIRA, and EntraID into Neo4j databases for AI-powered compliance analysis
> - **Automated Control Testing:** End-to-end system that automatically tests compliance controls (like TX-MON04.01) by collecting AWS infrastructure data, integration data, and document embeddings, then using LLM analysis to generate compliance scores and evidence
> - **Multi-tenant Architecture:** Strict tenant isolation with dynamic tenant resolution from RDS PostgreSQL, tenant-specific S3 buckets, and separate Neo4j databases per tenant
> - **AppRunner Services:** Containerized services for natural language to Cypher query processing with JWT authentication and VPC connectivity
> - **Frontend Controls UI:** Complete React-based compliance controls interface with automated testing integration, comments system, evidence management, and status updates
> - **Document Editor:** Rich text document editing capabilities for compliance documentation and policy management
> - **Integration Services:** Comprehensive integration with GitHub, JIRA, and EntraID for data collection and synchronization
> - **Lambda Functions Suite:** Complete set of serverless functions for document management, integration configuration, data synchronization, and scheduled operations

---

✅ 2. Feature Breakdown & Status

Feature Name	Description	Status	Notes/details
Profile-360 Data Pipeline	Multi-tenant data collection, formatting, and Neo4j ingestion from GitHub, JIRA, EntraID	refactored profile 360 to be a part of AWS CDK and made step functions a part of AWS CDK	Step Functions orchestration, scheduled execution, error handling
Automated Control Testing	End-to-end compliance control testing with LLM analysis and evidence generation	lambda workflow that accepts with a control context performs data collections from AWS sources, data dumps and embeddings and generates compliance reports there is an old automated test for configuration,Change management in integration repo	Successfully testing TX-MON04.01, ready for generic control expansion

		https://github.com/TrustXinc/integration/blob/main/automated-tests/automated_tests.py Connect your Github account which for the configuration management control to check MFA in various services and check code reviews are in place in services like github	
Multi-tenant Architecture	Tenant isolation, dynamic resolution, separate databases per tenant	made profile 360 component multi-tenant in apprunner Flask API, and in AWS backend to resolve tenants from RDS	RDS tenant lookup, tenant-specific S3 buckets, Neo4j credentials from Secrets Manager
AppRunner Query Service	Apprunner is now a part of CDK and the repo is moved into AWS	added tenant resolution from RDS in apprunner stack	VPC connectivity, psycopg2 conflict resolution, LangChain .invoke() implementation
GraphQL services	Had a major Database refactor related to controls and added new GraphQL endpoints for fetching all controls, fetching control details Control tests, control test results control evidences adding, deleting and updating user comments (in controls detail page) control policies	control details page currently uses these graphql methods, some unused methods related to old controls schema in soc2_manager.py can be safely tested and removed	
Frontend Controls UI	Complete React-based compliance controls interface	UI for controls dashboard, controls detail page, comments section	Controls listing, detail pages, automated testing, comments, evidence management
Document Editor & Management	Rich text document editing for compliance	Loss of Formatting due to conversion from	Need to have a library / service that does

	documentation	binary to Document Specific Format	format conversion with 0% formatting loss
Integration Services	GitHub, JIRA, EntraID integration for data collection	Can add new orgs, of Entra, JIRA, Github, delete, sync and choose org scope (github)	Real-time data synchronization, API integrations, error handling
Policy Management Interface	Policy creation, editing, and management system	Incomplete	Bugs in ineffect-draft-template. Database inconsistencies, version control interface of policies required.
Document Lambda Function	Complete document management with S3 integration and CRUD operations	can store retrieve normal and policy type documents and can parse/convert to different file formats returing to front end	binary file to document format conversion is being done by rtf parser, there is loss of formatting, need to find a solution with 100% format preservation
List Lambda Function	List Lambda Function used to list ORGs of each integration	Can List Orgs	list orgs from s3
Configure Lambda Function	Configure Lambda Function used to configure new ORGs for integrations	Can add new orgs	can add features such as immedeate credential validation but with the limitation of API call overhead in front end (Org Status will show any failure messages)
Delete Lambda Function	Delete Lambda Function used to delete existing ORGs of integrations	Deletes specific Orgs	Delete existing org, by deleting config info and secrets
Sync Lambda Function	Sync Lambda Function used to refresh ORGs of integrations	would fetch orgs for that specific integration	sync org info serves as refresh by looking up orgs in s3 after new org addition
Scheduler Lambda Function	Scheduler Lambda Function used to schedule data dumps of integrations in S3 buckets	scheduler function currently stores clean dumps, but dirty dump is commented is available if needed	scheduler lambda hits APIs of Entra, Jira and Github to get compliance related data dumps periodically

🔐 3. Access & Credentials

Resource	Credential	Access Type	Notes
GitHub	neel-assurex	Admin	Github access
Google	neel@assurexinc.com	Admin	For Atlassian, Jira, Google workspace, Trustx platform
Entra	neel@nagarur.net	Admin	For Entra Admin console
JIRA	neel@assurexinc.com	Admin	Jira platform
AWS	neel-trustx	Dev	Orchestrates data pipeline execution
Trustx	neel@assurexinc.com (trustx) neel+trustxinc@assurexinc.com (trustxinc)	admin	Trustx platform access
Augment code	neel@assurexinc.com	admin	google account login

> All AWS resources are in us-east-2 region. PostgreSQL, Neo4j credentials are stored in AWS Secrets Manager with tenant-specific keys.

🧠 4. Technical Architecture

4.1 Database Schema

Tables Created/Used:

- controls (compliance controls public schema)
- control_implementations
- control_evidences
- control_tests
- control_test_results
- document_embeddings
- documents (document metadata and references)
- users
- comments
- policies

Schema Notes:

> Multi-tenant architecture with tenant-specific schemas (tenant_251967, tenant_876131). Each tenant has isolated data in PostgreSQL and separate Neo4j databases. Document embeddings stored with S3 keys for reference. Document content stored in S3 with metadata in PostgreSQL.

4.2 Backend - Lambda Functions

Services Used:

- AWS Lambda (Python) - Profile-360 pipeline, automated control testing, document management, integrations
- AWS Step Functions - Pipeline orchestration
- AWS AppRunner - AI query service
- PostgreSQL RDS - Tenant and compliance data
- Neo4j - Graph databases per tenant
- S3 - Document storage and results
- AWS Secrets Manager - Neo4j credentials

Document Lambda Function (manageDocuments.py):

Feature	Description	Implementation
CRUD Operations	Complete Create, Read, Update, Delete operations for documents	RESTful API with GET, POST, PUT, DELETE methods
S3 Integration	Document content storage in S3 with metadata in PostgreSQL	Automatic S3 upload/download with content type detection
Multi-format Support	Support for JSON, text, binary files, images, PDFs	Content-type based processing with base64 encoding for binary
Policy Documents	Special handling for policy document types	Policy-specific database operations and S3 storage
Version Control	Document versioning and update tracking	Timestamp-based versioning with S3 metadata
Error Handling	Comprehensive error handling and logging	Try-catch blocks with detailed error messages and rollback
Multi-tenant Support	Tenant-aware document operations	Tenant-specific S3 buckets and database schemas

Key Document Lambda Features:

- **S3 Content Storage:** Document content stored in S3 with automatic content-type detection and proper file extensions
- **Database Metadata:** Document metadata (title, description, type, etc.) stored in PostgreSQL for fast querying
- **Binary File Support:** Handles images, PDFs, and other binary files with base64 encoding for API responses
- **Policy Integration:** Special handling for policy documents with dedicated database operations
- **Content Validation:** Validates document content and handles empty or invalid content gracefully
- **Cleanup Operations:** Automatic cleanup of S3 objects when documents are deleted
- **JWT Authentication:** Secure access with JWT token validation

> Document Lambda supports multi-tenancy with automatic tenant discovery from RDS. All document content is stored in tenant-specific S3 buckets with comprehensive error handling and logging.

4.3 Frontend

Major Components Developed:

- **FocusAreaDetail.jsx** - Main control detail page with automated testing integration
- **Controls UI Components** - Complete controls listing and management interface
- **Comments System** - Comments and evidence management for controls
- **Status Update System** - Control status management and tracking
- **Document Editor** - Rich text editing for compliance documents and policies
- **Policy Management** - Policy creation, editing, and frame management
- **Integration Pages** - UI for managing GitHub, JIRA, and EntraID integrations
- **Security Headers** - CSP configuration and security policy management

Key Frontend Features:

Feature	Description	Branch/PR
Automated Control Testing	Test Control button with API integration to Lambda backend	#101 - Add automated control testing integration
Control Policies UI	Policy management interface for compliance controls	#99 - control-policies
Comments & Evidence UI	Comments section and evidence management for controls	#98 - comments and evidences ui
Control Detail Pages	Comprehensive control detail view with all metadata	#94 - control detail page
Controls UI Redesign	Modern, responsive controls interface design	#92 - controls-ui-new-design
Status Update Backend	GraphQL integration for control status management	#96 - status update backend
Document Editor	Rich text document editing with version control	document-edit-ui branch
Documents Management	Document listing, upload, and management interface	documents-v1 branch
EntraID Integration	Frontend interface for EntraID integration management	entra-id-integration branch
Policy Frames	Policy frame management and editing interface	Policy-Frames-addition branch

Policy Edit/View	Policy editing and viewing modes	Policy-Edit-View branch
Security Headers	CSP fixes for blob URLs, worker sources, CDN support	Multiple commits - headers fixes

Deployment:

> React application deployed via standard build process. Frontend connects to backend APIs for control testing functionality. Includes comprehensive security headers and CSP configuration. Document editor supports rich text editing with Material-UI components.

🚀 5. Deployment Instructions

For AWS Infrastructure:

1. cd AWS
2. npm install
3. npm run build
4. cdk deploy --all

For Lambda Stacks in AWS :

1. Deploy any stack by ex: - Profile360Stack by cd AWS && ./scripts/deploy-profile-360-stack.sh --env=dev

For Frontend:

1. cd trustx
2. npm install
3. npm run build

🛠️ 6. Local Dev Setup

1. Clone repositories: AWS, trustx, integration
2. Install dependencies: npm install in each repo
3. Configure AWS credentials for us-east-2 region
4. Set up local PostgreSQL connection for testing

🗒️ 7. Known Issues & TODOs

Area	Description	Suggested Fix / Next Steps
Document Editor	Rich text editor may need additional features	Loss of Formatting due to conversion from binary to Document Specific Format Need a solution to preserve 100% formatting
Policy Management Interface	Bugs in ineffect-draft-template. Database inconsistencies	Fix Database inconsistencies, according to the holistic definition of ineffect-draft-template functionality version control interface of policies required
Automated Testing of Controls	For automated testing if controls we have tried out various ways first through API calls https://github.com/TrustXinc/integration/blob/main/automated-tests/automated_tests.py Connect your Github account and through n8n we have worked on CC5.1 But now through lambdas we have a workflow that can accept any control and perform source collection and autonomous analysis	

	Now this new workflow already accepts control id, tenant-name etc. and resolves tenants and performs db querying, Right now, some prompts are hardcoded with cc7.1, but This needs to be tweaked to make it generic using the input control id also the generated documents are being stored in s3 tenant bucket in the folder automated-testing https://github.com/TrustXinc/integration/blob/main/automated-tests/automated_tests.py Connect your Github account refer this file for the existing flow to store results in relevant db tables, can be replicated exactly or can be done by giving this context to augment in one prompt altering the lambda workflow to append this save test results flow will patch the results to the front end and will reflect immediately, no other modifications needed	

📌 8. Suggested Improvements / Future Work

- **Generic Control Support:** Expand automated testing to accept any control ID and dynamically configure testing flow
- **Enhanced Data Sources:** Add additional sources and other compliance tools to data collection
- **LLM Fine-tuning:** Improve analysis quality with domain-specific training on compliance frameworks
- **Dashboard Integration:** Connect automated testing results to existing compliance dashboards
- **Parallel Processing:** Implement concurrent testing of multiple controls
- **Real-time Alerting:** Add notifications for compliance score changes

- **Performance Optimization:** Optimize data collection and processing for larger datasets
- **Frontend Enhancements:** Add real-time progress tracking, advanced filtering, and bulk operations
- **Document Collaboration:** Add real-time collaborative editing features to document editor
- **Advanced Integrations:** Expand integration capabilities to include more third-party services
- **Policy Automation:** Implement automated policy generation based on compliance requirements
- **Document Versioning:** Implement comprehensive document version control with diff capabilities
- **Bulk Document Operations:** Add support for bulk document upload, download, and management

---

📊 9. Code Contributions Summary

GitHub Statistics (neel-assurex):

- **AWS Repository:** 23+ commits including infrastructure, pipelines, and automated testing
- **trustx Repository:** 50+ commits including UI components, fixes, and API integrations
- **integration Repository:** Multiple commits for GitHub, JIRA, EntraID integration services

Major Pull Requests & Branches:

Repository	Feature/Branch	Impact
AWS	Fix automated control testing: document embeddings search and S3 storage improvements	Complete automated testing system with 32+ document embeddings
AWS	Document Lambda Stack Implementation	Complete document management system with S3 integration and CRUD operations
trustx	Add automated control testing integration to FocusAreaDetail	Frontend integration with Lambda backend for control testing
trustx	control-policies	Policy management interface for compliance controls
trustx	comments and evidences ui	Comments system and evidence management
trustx	control detail page	Comprehensive control detail views

trustx	controls-ui-new-design	Modern, responsive controls interface with Material-UI components
trustx	document-edit-ui branch	Rich text document editor with editing capabilities
trustx	documents-v1 branch	Document management system with upload and organization
trustx	integration branch	Integration interface and management
trustx	Policy-Frames-addition branch	Policy frame management and editing system
trustx	Policy-Edit-View branch	Policy editing and viewing modes

Key Technical Achievements:

- **50+ commits** across trustx repository with comprehensive UI development
- **Multiple feature branches** for document editing, policy management, and integrations
- **Material-UI integration** for modern, responsive design components
- **GraphQL client implementation** for efficient data fetching and management
- **Integration management** interfaces for external service connections
- **Scheduler Data Dumps** periodic data dumps from Entra, Jira and Github
- **Policy management system** with frame-based editing and viewing
- **Document Editor implementation** with CRUD operations and S3 integration
- **Multi-format document support** including binary files, images, and PDFs